



Schulzentrum HTL HAK Ungargasse

Höhere Lehranstalt für Informationstechnologie

Ausbildungsschwerpunkt Netzwerktechnik

Diplomarbeit



SLOTecH – Smart Life Optimization Technology

Sebastian Rosenkranz

Martin Dobrev

Mateusz Pacyga

Betreuer: Ing. Thomas Gürth

Schuljahr 2023/24

Klasse: 5BHITN

Abgabevermerk: 3U_HI_2024_09

übernommen von:

Datum: 22.03.2024

.....

Eidesstattliche Erklärung

Wir erklären, dass wir die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt, sowie die aus anderen Werken übernommenen Inhalte durch Quellenangaben erkenntlich gemacht haben.

Wien, 22.03.2024

Sebastian Rosenkranz:

Martin Dobrev:

Mateusz Pacyga:

Kurzfassung

Smart Homes sind eine futuristische Idee, die in den letzten Jahren in greifbare Nähe gerückt ist, aber bei weitem noch nicht billig genug, um die Anschaffung attraktiv zu machen. Deshalb hat sich die Diplomarbetsgruppe der 5BHITN aus eigenem Interesse, Neugier und Erfahrung dazu entschlossen, ein eigenes, kostengünstiges und individuell angepasstes Smart Home von Grund auf neu zu entwickeln. Mit der Wirtschaftlichkeit im Vordergrund wusste das Team bereits, was es entwickeln wollte: selbst programmierte Endgeräte aus der Schwesterdiplomarbeit SLOTech Components. Diese Diplomarbeit beinhaltet die Software zur Realisierung einer Smart-Home-Zentrale mit einer Web-Applikation als Benutzerschnittstelle zur Steuerung. Nach reiflicher Überlegung entschied sich das Team für den Namen „Smart Life Optimisation Technology“, kurz SLOTech. Das Diplomarbeitsteam bestand ursprünglich aus sechs Personen, wurde aber in zwei Gruppen aufgeteilt: SLOTech und SLOTech Components mit jeweils drei Personen. Durch diese Aufteilung und die unterschiedlichen Kompetenzen der Mitglieder konnte der Entwicklungsprozess fokussiert und präzise gehalten werden.

Abstract

Smart homes are a futuristic idea that has become more tangible for people in recent years, but they are still far from cheap enough to make them an attractive purchase.

Therefore, a group from the class 5BHITN decided, out of curiosity and experience, to develop their own low-cost, personally customised smart home from scratch. With cost-effectiveness in mind, the team already knew what they wanted to develop for: self-programmed end devices from the sister thesis SLOtecH Components. This work included the software to create a smart home control centre, with a web application as the user interface for control.

After much deliberation, the team decided on the name „Smart Life Optimisation Technology“, or SLOtecH for short.

The team originally consisted of six people, but was split into two groups: SLOtecH and SLOtecH Components, with three people in each. Thanks to this split and the different skills of the members, the development process was able to remain focused and precise.

Inhaltsverzeichnis

Eidesstattliche Erklärung	3
Kurzfassung	4
Abstract	5
1 Allgemein	17
1.1 Ausgangslage	17
1.2 Zielsetzung	17
1.3 Individuelle Themenstellungen	18
1.4 Geplantes Ergebnis	18
1.5 Tatsächliches Ergebnis	18
1.6 Meilensteine	19
1.7 Sprints	20
1.7.1 Die Verwendung künstlicher Intelligenz	20
1.8 Projektziele	21
1.8.1 Hauptziele	21
1.8.1.1 HZ01 - Anfertigung einer Präsentation	21
1.8.1.2 HZ02 - Abgabe des Projektantrags	21
1.8.1.3 HZ03 - Projektmanagement in Jira	21
1.8.1.4 HZ04 - Kommunikationsprotokoll festlegen	21

1.8.1.5	HZ05 - Endgeräte ansteuern	21
1.8.1.6	HZ06 - Schnittstelle zu den Endgeräten	21
1.8.1.7	HZ07 - Schnittstelle zur Web-App	22
1.8.1.8	HZ08 - Datenbank entwerfen	22
1.8.1.9	HZ09 - Messwerte speichern	22
1.8.1.10	HZ10 - Szenen erstellen und ausführen	22
1.8.1.11	HZ11 - Responsive Design	22
1.8.1.12	HZ12 - Darstellen und Steuern von Endgeräten	22
1.8.1.13	HZ13 - Echtzeitkommunikation	22
1.8.1.14	HZ14 - Hinzufügen von neuen Endgeräten	23
1.8.1.15	HZ15 - Benachrichtigungen	23
1.8.1.16	HZ16 - Event Verwaltung	23
1.8.1.17	HZ17 - Diplomarbeitsbuch	23
1.9	Projektteam	24
1.10	Projektressourcen	26
1.10.1	Projektressourcen: Soll - Ist Vergleich	26
1.10.2	Technische Ressourcen und Kosten	26
1.10.3	Hardware Anforderungen der Implementierung dieser Diplomarbeit	26
1.10.4	Personelle Ressourcen (geplant)	27
1.10.5	Personelle Ressourcen (tatsächlich)	27
2	Projektmanagement	31
2.1	Allgemeines	31
2.2	SCRUM	32
2.2.1	SCRUM Prozess	32
2.2.2	Originelles SCRUM-Manifest	33
2.2.3	Angepasstes SCRUM-Manifest	33

2.3	Meilensteine	35
2.4	Leistungstrend der Mitglieder	35
2.5	Motivation	37
2.5.1	Motivation in der Diplomarbeit	37
2.5.2	Motivationstrend während der Entwicklung	37
2.5.3	Methoden zur Förderung der Motivation	38
2.6	Erlernstes	38
3	Raspberry Pi	39
3.1	GPIO-Pins	40
4	Kommunikationsprotokoll	41
4.1	Message Queuing Telemetry Transport Protokoll (MQTT)	41
4.1.1	Publish-Subscribe-Modell	42
4.1.2	Quality of Service (QoS)	43
4.1.3	Retained Messages	44
4.2	Aufbau des Kommunikationsprotokolls mit den ESPs	44
4.3	Kommunikationsprotokoll mit der Web-Anwendung	45
4.3.1	API	45
4.3.1.1	HTTP GET Methoden	45
4.3.1.2	HTTP POST Methoden	46
4.4	Zusammenspiel beider Schnittstellen	46
5	Linux	47
5.1	Betriebssystem Auswahl	47
5.1.1	Installation	50
5.2	Eingesetzt Linux Tools	50
5.3	SSH	52

5.4	Netzwerkkonfiguration Herausforderungen	53
5.4.1	nmcli	54
5.5	Systemd	54
5.5.1	Systemctl	54
6	Broker & Datenbankverwaltung	59
6.1	Allgemein	59
6.1.1	Eigener Bereich	59
6.1.2	Verwendete Technologien	61
6.1.3	Visual Studio 2019	61
6.1.4	Microsoft.Data.Sqlite	62
6.1.5	MQTTnet	63
6.1.6	ADO.NET	65
6.2	Umsetzung vom Broker	68
6.2.1	Was ist ein Broker?	68
6.2.2	Szenen	69
6.2.2.1	Was sind Smart Home Szenen	69
6.2.2.2	Was können Smart Home Szenen	70
6.2.2.3	Wie werden Szenen erstellt	70
6.2.2.4	Wie lassen sich Szenen steuern oder anpassen	71
6.2.3	SlothServer	71
6.2.4	HTTP-POST	72
6.2.4.1	Beispiel	72
6.2.5	HTTP-GET	73
6.2.5.1	Beispiel	74
6.3	Textdateien	76
6.3.1	Deadlocks beim Speichern von Informationen in Dateien	76

6.3.2	Was ist ein Deadlock?	76
6.3.3	Beispiel	77
6.3.4	Warum Textdateien anstatt einer Datenbank?	77
6.3.5	Aufbau der Methoden	78
6.4	Datenbank	81
6.4.1	Entity-Relationship-Diagramm	81
6.4.2	Warum eigentlich eine SQLite Datenbank?	82
6.4.3	Initialisation der Datenbank	83
6.4.4	CRUD - Create, Read, Update, Delete	83
6.4.5	Create (Datensätze einfügen)	84
6.4.6	READ (Datensätze ausgeben)	86
6.4.7	UPDATE (Datensätze aktualisieren)	87
6.4.8	DELETE (Datensätze löschen)	88
6.4.9	Allgemein zu der Syntax von der Datenbank	89
6.4.10	Filtermöglichkeiten	90
6.4.11	Debuggen	92
6.4.12	ADO Mapper	92
7	Webapplikation	97
7.1	Einführung in die Webapplikation	97
7.1.1	Problemstellung	97
7.1.2	Zielsetzung	98
7.2	Überblick	99
7.2.1	Grundlagen	100
7.2.2	Technologien	100
7.2.3	Architektur & Design	100
7.3	Grundlagen & Technologien	103

7.3.1	TypeScript	104
7.3.1.1	Server-Side	105
7.3.1.2	Client-Side	105
7.3.2	Express.js	105
7.3.2.1	NPM	106
7.3.2.2	Server-Side	107
7.3.2.3	Client-Side	107
7.3.2.4	Beispiel	107
7.3.3	Socket.io	108
7.3.3.1	Features	108
7.3.3.2	Socket.io-Client	110
7.3.3.3	Beispiel	110
7.3.3.4	Server-Side & Client-Side	111
7.3.4	Next.Js	113
7.3.4.1	Features	113
7.3.4.2	Server-Side & Client-Side	114
7.3.5	React	116
7.3.5.1	Vorteile von React	118
7.3.5.2	Beispiel	119
7.3.5.3	Server-Side	120
7.3.5.4	Client-Side	120
7.3.6	TailwindCSS	122
7.3.6.1	Beispiel	122
7.3.6.2	Features	123
7.3.6.3	Server & Client	125
7.3.7	Bun.sh	126

7.3.7.1	Bun CLI	126
7.3.7.2	Features	126
7.3.7.3	APIs	128
7.3.7.4	Client & Server	129
7.3.8	Zusammenspiel der Technologien	130
7.4	Server & Client Realisierung und Herausforderungen	131
7.4.1	Server	131
7.4.1.1	Realisierung	131
7.4.1.2	Beispiel	132
7.4.1.3	Herausforderungen und deren Lösungen	132
7.4.2	Client	133
7.4.2.1	Realisierung	133
7.4.2.2	Beispiele	134
7.4.2.3	Herausforderungen und deren Lösungen	138
7.4.2.4	Fazit	141

Allgemeiner Teil

SLOTecH

Kapitel 1

Allgemein

1.1 Ausgangslage

Wir sind äußerst begeistert von den Konzepten und Grundlagen der Smart Home Technologie, mit denen wir uns schon außerhalb der Unterrichtszeit auseinandergesetzt haben, und wollen daher jetzt unsere Begeisterungen im Zuge der Diplomarbeit ausleben lassen. Dafür benutzen wir einen Raspberry PI mit MQTT für die Zentrale des Systems, welche auch eine Web-App als Steuereinheit zur Verfügung stellt.

1.2 Zielsetzung

Das Endprodukt soll ein komplettes Smart Home, inklusive Steuerzentrale, sein. Es soll kostengünstig sein und technikaffinen Benutzern die Möglichkeit geben, Erweiterungen selbst einzufügen. Diese Erweiterungen sollen durch bereitgestellte Templates ermöglicht werden. Die einzelnen Komponenten sollen unter anderem auf C#, mit dem .Net nanoFramework, basieren.

1.3 Individuelle Themenstellungen

- Rosenkranz: Entwicklung der Web-APP, alle Endgeräte sind über die App ansteuerbar, Werte können dargestellt & manipuliert werden, externe Smart Geräte sind ansteuerbar, simple Aktionen können erstellt werden
- Dobrev: Schnittstelle zu SLOTech Components Endgeräten, automatische Messungen, Verfassung der Konfigurationsdateien, Kommunikation der empfangenen Daten zwischen Broker und Endgeräten
- Pacyga: Messwerte in Textdateien speichern, Neue Endgeräte mit DLL-Dateien hinzufügen, Broker wird möglichst energieeffizient betrieben, Vordefinierte Richtwerte können individuell angepasst werden

1.4 Geplantes Ergebnis

SLOTech ist ein Smart Home System. Es setzt sich aus mehreren Teilen zusammen, jedoch ist das Projekt in zwei Gruppen geteilt, wobei diese Gruppe sich mit der Entwicklung des Brokers und der Web-APP beschäftigt. Der Broker empfängt Daten und übergibt diese der Web-APP und den Sensoren, weiters speichert er diese Messwerte. Die Web-APP empfängt die Daten und steuert die Geräte.

1.5 Tatsächliches Ergebnis

Das Ergebnis dieser Diplomarbeit ist eine ISO-Datei mit Software zum Flashen auf einem Raspberry Pi. Die ISO-Datei enthält eine bereinigte Ubuntu Software Distribution, die für Erweiterungen offen ist. Die Software beinhaltet ein Backend-Service, welches eine offene Kommunikationsschnittstelle zu selbst programmierten ESPs und SLOTH Components' Endgeräten bereitstellt. Alle relevanten Informationen werden vom Backend-Service gespeichert und können per Web-App visualisiert werden.

Über die Webanwendung lässt sich das Smart Home System steuern, Szenen können erstellt und verwaltet sowie Geräte und ihre Historie eingesehen und angesteuert werden. Außerdem bleiben alle Elemente live up-to-date und wichtige Informationen können über Benachrichtigungen und eine Benachrichtigungszentrale eingesehen werden.

1.6 Meilensteine

- 25.10.2023 Grundstruktur von der Web-App steht + Basis Konstellation des Projektes.
- 22.11.2023 Web-App ausgebaut, inklusive externe Provider ansteuerbar (z.B. Philips Hue) + mittels C# die Logik bei der Zentrale abwickeln.
- 20.12.2023 Kommunikation zwischen Web-App, Zentrale und Endgeräten steht & funktioniert.
- 24.01.2024 Thesis zu 60% fertig
- 14.02.2024 Eine komplett funktionierende Zentrale, die offen für Erweiterungen ist, mit einer Kommunikationsschnittstelle für eingehenden und ausgehenden Datenverkehr, sowie einer Steuereinheit, die diese Daten manipulieren und anzeigen kann. Einbau von neuen Endgeräten mittels DLL-Dateien möglich.
- 06.03.2024 Thesis vollendet

1.7 Sprints

Nummer	Anfang	Endabgabe	Hinweis	Meilenstein
1	27.09.2023	11.10.2023	-	Meilenstein 1
2	11.10.2023	25.10.2023	-	Meilenstein 1
3	25.10.2023	08.11.2023	-	Meilenstein 2
4	08.11.2023	22.11.2023	-	Meilenstein 2
5	22.11.2023	06.12.2023	-	Meilenstein 2
6	06.12.2023	20.12.2023	-	Meilenstein 3
7	28.12.2023	10.01.2024	Weihnachtsferien	Meilenstein 4
8	10.01.2024	24.01.2024	-	Meilenstein 4
9	24.01.2024	14.02.2024	-	Meilenstein 5
10	28.02.2024	13.03.2024	-	-

Tabelle 1.1: Sprint Daten

1.7.1 Die Verwendung künstlicher Intelligenz

Für diese Arbeit wurden zwei KI-Tools verwendet, um einerseits die Struktur und Ideen zu perfektionieren, und andererseits um den Inhalt und die Rechtschreibung zu überprüfen.

- ChatGPT
 - Verwendung in diesem Dokument: Für das Erstellen und Bearbeiten von der Struktur und fürs Ideen sammeln
 - URL: <https://chat.openai.com/>
- DeepL Write
 - Verwendung in diesem Dokument: Für die Rechtschreib- und Inhaltsprüfung
 - URL: <https://www.deepl.com/write>

1.8 Projektziele

1.8.1 Hauptziele

1.8.1.1 HZ01 - Anfertigung einer Präsentation

Eine PowerPoint die einen groben Überblick über das Projekt und dessen Teilgebiete liefert.

1.8.1.2 HZ02 - Abgabe des Projektantrags

Ein Dokument, dass das zu machende Projekt, Themeneinteilungen und alle Teilgebiete je Projektmitglied enthält.

1.8.1.3 HZ03 - Projektmanagement in Jira

Management des Projektes in Form von einzelnen Arbeitspaketen, Sprints und Meilensteinen.

1.8.1.4 HZ04 - Kommunikationsprotokoll festlegen

Ein einheitliches Kommunikationsprotokoll festlegen für die Kommunikation zw. dem Hub und den Endgeräten.

1.8.1.5 HZ05 - Endgeräte ansteuern

Alle Endgeräte, sowie externe Smart Geräte (Philips Hue Lampen) sind ansteuerbar.

1.8.1.6 HZ06 - Schnittstelle zu den Endgeräten

Kommunikation zwischen den Endgeräten und dem Hub ist gegeben.

1.8.1.7 HZ07 - Schnittstelle zur Web-App

Die erhaltenen Informationen werden an die Web-App weitergeleitet.

1.8.1.8 HZ08 - Datenbank entwerfen

Ein Klassendiagramm entwerfen und dieses dann umsetzen.

1.8.1.9 HZ09 - Messwerte speichern

Die erhaltenen Messwerte in der Datenbank speichern.

1.8.1.10 HZ10 - Szenen erstellen und ausführen

Eine Szene über die Web-App erstellen und wenn die Bedingungen der Szene erfüllt sind, wird eine Aktion gesetzt.

1.8.1.11 HZ11 - Responsive Design

Die Web-App ist auf allen Geräten benutzbar.

1.8.1.12 HZ12 - Darstellen und Steuern von Endgeräten

Die Web-App kann die Endgeräte darstellen und ansteuern.

1.8.1.13 HZ13 - Echtzeitkommunikation

Die Webanwendung kommuniziert in Echtzeit mit dem Broker und bekommt die aktuellen Messwerte.

1.8.1.14 HZ14 - Hinzufügen von neuen Endgeräten

Das Hinzufügen von neuen Endgeräten ist möglich. Beim Hinzufügen kann man Namen und Tags vergeben.

1.8.1.15 HZ15 - Benachrichtigungen

Benachrichtigungen werden an die Web-App gesendet, wenn eine Bedingung erfüllt ist und können in der Web-App eingesehen werden.

1.8.1.16 HZ16 - Event Verwaltung

Die Web-App kann Events erstellen und verwalten.

1.8.1.17 HZ17 - Diplomarbeitsbuch

Ein Diplomarbeitsbuch ist ein Dokument, welches eine ausführliche Dokumentation von jedem Arbeitsschritt manifestiert und als Endabgabe verwendet wird.

1.9 Projektteam

Sebastian Rosenkranz



- **Aufgabenbereich:** Entwicklung der Web-APP, alle Endgeräte sind über die App ansteuerbar, Werte können dargestellt & manipuliert werden, externe Smart Geräte sind ansteuerbar, simple Aktionen können erstellt werden
- **Auflistung der einzelnen Ziele:** HZ01, HZ02, HZ11, HZ12, HZ13, HZ14, HZ15, HZ16, HZ17

Martin Dobrev



- **Aufgabenbereich:** Schnittstelle zu SLOTech Components Endgeräten, Verfassung der Konfigurationsdateien für Linux Tools, Kommunikation der empfangenen Daten zwischen Broker und Endgeräten, Inbetriebnahme vom Raspberry Pi, Kommunikationsprotokoll festlegen
- **Auflistung der einzelnen Ziele:** HZ01, HZ02, HZ03, HZ04, HZ05, HZ06, HZ07, HZ10, HZ17

Mateusz Pacyga



- **Aufgabenbereich:** Broker wird möglichst energieeffizient betrieben, Entwurf und Umsetzung der Datenbank, Schnittstelle zur Webanwendung
-
- **Auflistung der einzelnen Ziele:** HZ01, HZ02, HZ07, HZ08, HZ09, HZ10, HZ17

1.10 Projektressourcen

1.10.1 Projektressourcen: Soll - Ist Vergleich

Know-how	Ist
Bereich Softwareentwicklung	ausreichend
Bereich Projektmanagement	ausreichend

Tabelle 1.2: Projektressourcen

1.10.2 Technische Ressourcen und Kosten

Produkt	Stückzahl	Preis
Raspberry Pi 4B+	1 x	€ 68,90
SanDisk Ultra 32GB Micro SD Karte	1 x	€ 9,90
Summe		€ 78,80

1.10.3 Hardware Anforderungen der Implementierung dieser Diplomarbeit

Für die Installation dieses Diplomarbeitsprojektes steht eine ISO-Datei zur Verfügung, die auf den Raspberry geflasht werden kann.

Ein Raspberry Pi 4 Modell B mit 2GB RAM ist das Minimum für eine reibungslose Implementierung, kombiniert mit einer Micro-SD-Karte mit einer Größe von mindestens 32GB, beziehungsweise 16GB, falls die ISO-Datei nicht verwendet wird, sondern nur die Projekte aufgesetzt werden. Die Lese- und Schreibgeschwindigkeiten sind ebenfalls relevant, daher eine folgende (minimale) Empfehlung:

- Größe: 32 GB
- Lesegeschwindigkeit laut Spezifikation: 98 MB/s
- Schreibgeschwindigkeit laut Spezifikation: 41MB/s, aber bis zu 100MB/s

- Speichertyp: microSDHC
- Geschwindigkeitsklasse: Klasse 10

Bei der Entwicklung wurde eine SanDisk Ultra 32GB Micro SD Karte mit den gleichen Spezifikationen verwendet.

1.10.4 Personelle Ressourcen (geplant)

Teammitglied	Personenstunden
Rosenkranz Sebastian	180 h
Dobrev Martin	180 h
Pacyga Mateusz	180 h
Summe	540 h

Tabelle 1.3: Personelle Ressourcen geplant

1.10.5 Personelle Ressourcen (tatsächlich)

Teammitglied	Personenstunden
Rosenkranz Sebastian	210 h
Dobrev Martin	200 h
Pacyga Mateusz	160 h
Summe	570 h

Tabelle 1.4: Personelle Ressourcen tatsächlich

Projektmanagement & Teil des Brokers

Dobrev Martin

Kapitel 2

Projektmanagement (Dobrev Martin)

2.1 Allgemeines

Die Bedeutung des Projektmanagements für die effiziente Organisation, sorgfältige Planung und termingerechte Durchführung von Projekten ist unbestritten. Ein Projekt zeichnet sich durch seine zeitliche Befristung und inhärente Risiken aus und unterscheidet sich dadurch von den Routinetätigkeiten eines Unternehmens. Die Rolle des Projektmanagers ist daher entscheidend, um die Komplexität zu steuern und den Fortschritt eines Projekts zu koordinieren.

Die erfolgreiche Durchführung eines Projekts kann auf unterschiedliche Weise erreicht werden, was die Existenz verschiedener Projektmanagementmethoden erklärt. Diese Vielfalt spiegelt die Einzigartigkeit jedes Projekts wider. Während einige Projekte von einer traditionellen Managementstruktur profitieren, können andere durch agile Praktiken effizienter gestaltet werden. Entscheidend ist die Wahl der Methode, die den spezifischen Bedürfnissen und Eigenschaften des Projekts am besten entspricht. Für diese Diplomarbeit war die Entscheidung des Teams von Anfang an klar: Agile Softwareentwicklung, speziell mit dem SCRUM-Prozess.

2.2 SCRUM

2.2.1 SCRUM Prozess

Der SCRUM-Prozess ist eine iterative und inkrementelle Softwareentwicklungsmethode, die darauf abzielt, flexibel auf sich ändernde Anforderungen und Prioritäten zu reagieren. Er besteht aus mehreren Hauptkomponenten:

1. Product Backlog: Eine Liste von Anforderungen und Funktionen, die das Produkt enthalten soll. Diese werden in Form von User Stories oder Tasks definiert und priorisiert.
2. Sprint: Ein Zeitraum von ein bis vier Wochen, in dem das Team die ausgewählten Aufgaben aus dem Product Backlog umsetzt. Ziel ist es, am Ende des Sprints ein inkrementelles Produkt oder Feature bereitzustellen.
3. Sprint Planung: Zu Beginn jedes Sprints findet ein Sprint Planning Meeting statt, in dem das Team die Tasks aus dem Product Backlog auswählt, die während des Sprints umgesetzt werden sollen. Das Team schätzt den Aufwand und verpflichtet sich, die ausgewählten Aufgaben innerhalb des Sprints abzuschließen.
4. Daily Stand-up Meetings: Tägliche kurze Besprechungen, in denen jedes Teammitglied über den Fortschritt seiner Arbeit seit der letzten Besprechung, mögliche Hindernisse und die für den nächsten Tag geplante Arbeit berichtet.
5. Sprint Review: Am Ende jedes Sprints findet ein Sprint Review Meeting statt, bei dem das Team die abgeschlossene Arbeit präsentiert und Feedback von den Stakeholdern erhält.
6. Sprint Retrospective: Nach dem Sprint Review trifft sich das Team zur Sprint Retrospective, um den vergangenen Sprint zu reflektieren und Verbesserungsmöglichkeiten für den nächsten Sprint zu identifizieren.

Meilensteine sind Ziele, die das Team im Entwicklungsprozess verfolgt. Sie helfen bei der Motivation und geben einen Überblick über den Entwicklungsfortschritt. Obwohl Meilensteine nicht direkt zum SCRUM-Prozess gehören, sind sie ein Teil der agilen Entwicklung. SCRUM ist eine Spezifikation für agile Softwareentwicklung.[1]

2.2.2 Originelles SCRUM-Manifest

Das SCRUM Manifest verkörpert die grundlegenden Prinzipien und Werte, die die SCRUM Methodik für die Softwareentwicklung definieren. Es konzentriert sich auf vier Kernwerte, die aus technischer Sicht die Essenz erfolgreicher Softwareentwicklung darstellen.[2]

1. Menschen und Interaktionen über Prozesse und Werkzeuge: Dieser Wert betont die Bedeutung der zwischenmenschlichen Interaktion und Zusammenarbeit innerhalb des Entwicklungsteams und mit dem Kunden.
2. Funktionierende Software durch umfassende Dokumentation**: Dieser Wert betont die Priorität der eigentlichen Softwareentwicklung und -lieferung gegenüber der Erstellung einer umfassenden Dokumentation.
3. Zusammenarbeit mit dem Kunden bei Vertragsverhandlungen: Dieser Wert betont die Wichtigkeit einer engen Zusammenarbeit mit dem Kunden, um seine Bedürfnisse besser zu verstehen und zu erfüllen.
4. Auf Veränderungen reagieren, indem man einem Plan folgt: Dieser Wert unterstreicht die Bedeutung von Anpassungsfähigkeit und Flexibilität während des Entwicklungsprozesses.

2.2.3 Angepasstes SCRUM-Manifest

In dieser Diplomarbeit hat sich das Team für die Verwendung von SCRUM entschieden. Es wird betont, dass Anpassungsfähigkeit und Flexibilität während des gesamten Entwicklungsprozesses von entscheidender Bedeutung sind.

Das Team ist sich von Anfang an bewusst, dass das Zeitmanagement in der 5. Klasse HTL eine kritische Rolle spielt und dass die Zeit ein entscheidender Faktor ist. Das

Diplomarbeitsteam entschied sich für ein angepasstes SCRUM Manifest und einen SCRUM Prozess, um eine bessere Arbeitsmethode für diese Zwecke zu entwickeln, bei der die Zeit für die Planung effizient genutzt wird und mehr Zeit in den Unterrichtseinheiten für das Testen und Arbeiten an der Funktionalität bleibt. Um dies zu erreichen, entschied sich das Team, einen eigenen Prozess für die Entwicklung dieser Diplomarbeit zu erfinden bzw. das originale SCRUM Manifest zu adaptieren. Tägliche Stand-Ups wurden am Anfang durchgeführt, aber mit der Zeit stellte sich heraus, dass die Gruppe ohne diese effizienter arbeitete. Da sich die Gruppe nur einmal pro Woche zum Arbeiten trifft, macht es wenig Sinn zu erklären, was an diesem Tag gemacht wird, da es sich meistens entweder um User Stories aus dem aktuellen Sprint oder um Tests mit den anderen Teammitgliedern handelt, was allen klar ist.

Die Sprintstruktur und das Sprint-Reporting sind wie im Original-Manifest geblieben, was auch Sinn macht, da alle im Team sehr gerne strukturiert mit User-Stories arbeiten. Diese Diplomarbeit startete nicht mit einem Product Backlog bestehend aus User Stories, sondern mit einer Vision für das Projekt und einer grundlegenden Planung für die zu implementierende Funktionalität und wie diese implementiert werden soll. Dabei wurde das Kommunikationsprotokoll definiert und später im Laufe der Entwicklung erweitert. Im Nachhinein kann man sagen, dass ein Product Backlog sinnvoll ist, da man keine Zeit mit Überlegungen verliert, sondern sich auf die Entwicklung konzentrieren kann.

Sprints sind der Schlüssel zu einer gut geplanten Entwicklung des Projekts, da man einen Rhythmus definieren kann und somit auch die Meilensteine leichter verfolgen kann. Im Fall dieser Diplomarbeit war dieser Takt ein Zeitraum von zwei Wochen und das Team hatte insgesamt 12 Sprints vom Beginn des Schuljahres bis zur Abgabe der Diplomarbeit. Jede zweite Woche traf sich das Team im Fach „Informationstechnischeprojekte“, um den aktuellen Sprint abzuschließen und den nächsten Sprint zu planen und zu starten. Sprint Reviews gehören zur Lernphase in der Entwicklung.[3]

2.3 Meilensteine

Meilensteine im Entwicklungsprozess sind Schritte, die zu bestimmten Zeitpunkten erreicht werden müssen. Dies sind z.B. bestimmte Funktionalitäten, Implementierungen, Fortschritte in der Dokumentationsarbeit und andere Ziele, die das Team im Entwicklungsprozess erreichen will.[1] In dieser Diplomarbeit hat das Team die folgenden Meilensteine definiert und alle erreicht:

1. Grundstruktur der Web-App steht + Grundkonstellation des Projektes.
2. Web-App erweitert, inkl. externe Anbieter ansteuerbar (z.B. Philips Hue) + mittels C# die Logik in der Zentrale abarbeiten.
3. Kommunikation zwischen Web-App, Zentrale und Endgeräten steht & funktioniert.
4. Diplomarbuchs zu 40% fertiggestellt.
5. Voll funktionsfähige, erweiterbare Zentrale mit Kommunikationsschnittstelle für eingehenden und ausgehenden Datenverkehr sowie einem Controller, der diese Daten manipulieren und anzeigen kann.
6. Diplomarbuchs fertiggestellt.
7. Diplomarbeit fertig gedruckt und gebunden.

Meilensteine haben feste Zeitpunkte, zu denen der Inhalt gegeben sein muss, die auch in dieser Diplomarbeit vom Team definiert wurden, sind aber für die Dokumentation nicht relevant.

2.4 Leistungstrend der Mitglieder

Während des Diplomarbeitsprojektes zeigte sich ein interessanter Leistungstrend bei den Teammitgliedern. Zu Beginn des 8. Semesters war die Produktivität eher gering, was auf die Eingewöhnungsphase und die Anpassung an die Projektanforderungen zurückzuführen war. In dieser Zeit wurde die gewünschte Funktionalität definiert. Mit Beginn des 9. Semesters setzte jedoch eine deutliche Steigerung der Arbeitsleistung ein. Die Teammitglieder arbeiteten konsequent und zeigten ein

bemerkenswertes Engagement für ihre Aufgaben. Insbesondere in den Semesterferien konnte eine erhöhte Produktivität festgestellt werden, da die Mitglieder mehr Zeit für das Projekt aufwenden konnten. Gegen Ende der Bearbeitungszeit war jedoch ein Leistungsabfall zu beobachten. Dies war teilweise auf die erhöhten Anforderungen seitens der HTL zurückzuführen. Dazu gehörten vermehrte Tests und Schularbeiten, die die Zeit und Energie der Teammitglieder in Anspruch nahmen. Trotz dieser Herausforderungen konnte das Team die Diplomarbeit erfolgreich abschließen.

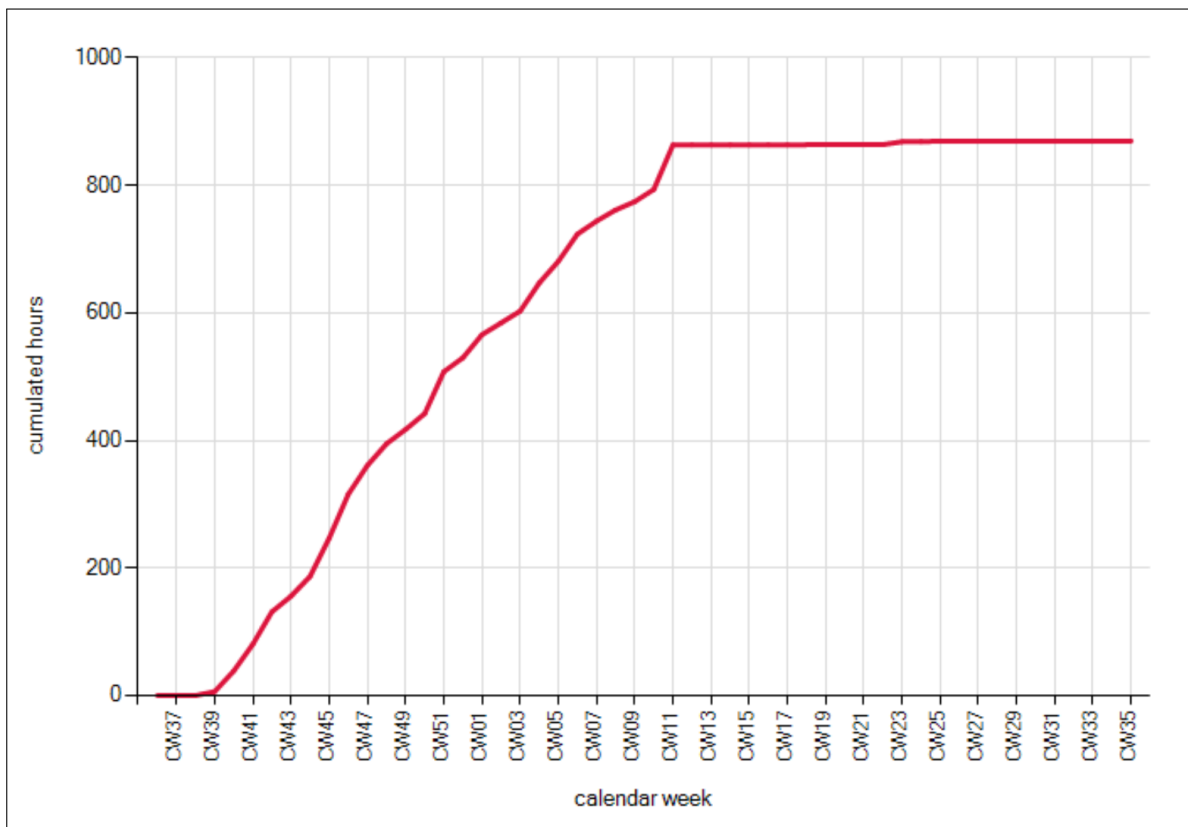


Abbildung 2.1: Statistik aller produktiven Stunden

Betrachtet man die Statistik (inkl. SLOTech und SLOTech Components), so ist zu erkennen, dass die Leistung pro Kalenderwoche relativ konstant blieb und in wichtigen Entwicklungsphasen entsprechend gut anstieg. Alle Teammitglieder erbrachten gute Leistungen und brachten ihre Expertise in den Bereichen ein, in denen sie sich am besten auskannten.

Die gute Zusammenarbeit war entscheidend für den erfolgreichen Abschluss der Entwicklung. Regelmäßige Treffen und eine offene Kommunikation ermöglichten es den Teammitgliedern, Hindernisse zu überwinden, sich gegenseitig zu motivieren und

gemeinsam Lösungen zu finden. Die Aufgabenverteilung war transparent und basierte auf den individuellen Stärken und Vorlieben der einzelnen Teammitglieder.

Die letzten Schritte dieser Diplomarbeit umfassten das Testen der gesamten Funktionalität des Projekts und die nahtlose Integration der Komponenten von SLOTH Components. Nach der erfolgreichen Validierung der Funktionalität begann das Team mit der Fertigstellung des Projekts. Dies beinhaltete die Erstellung einer ISO-Datei und die Fertigstellung der internen Dokumentation im Code.

2.5 Motivation

2.5.1 Motivation in der Diplomarbeit

Motivation spielt in jeder Diplomarbeit eine entscheidende Rolle, insbesondere wenn es um die Leistung und das Engagement der Teammitglieder geht. Sie bezieht sich auf die Energie, den Enthusiasmus und die Ausdauer, um die gesetzten Ziele zu erreichen und Herausforderungen zu meistern. In einer Teamumgebung ist es wichtig, dass alle Mitglieder motiviert sind, effektiv zusammenzuarbeiten und das Projekt erfolgreich abzuschließen.

2.5.2 Motivationstrend während der Entwicklung

Die Motivation der Teammitglieder schwankte im Laufe der Zeit. Während der Schulzeit war überraschenderweise eine hohe Motivation bei allen Teammitgliedern zu spüren, insbesondere wenn weniger schulische Verpflichtungen wie Tests und Schularbeiten anstanden. In den Winterferien hingegen erlebte das Team ein Motivationstief, da vor und nach den Ferien schulische Verpflichtungen wie Prüfungen anstanden und sich alle mehr auf ihre schulischen Leistungen konzentrierten. In der mittleren Entwicklungsphase zwischen November und Januar war die Motivation im

Team konstant hoch. Alle motivierten sich gegenseitig und erbrachten gemeinsam Leistungen. Ab Januar richtete sich die Aufmerksamkeit aller auf die Vorbereitung der Abgabe der Diplomarbeit. Das Team arbeitete intensiv an der Erstellung und Fertigstellung des Projektes.

2.5.3 Methoden zur Förderung der Motivation

Es gibt verschiedene Möglichkeiten, die Teammitglieder während einer Diplomarbeit zu motivieren. Ein wichtiger Ansatz ist es, klare Ziele zu setzen und Meilensteine zu definieren, um den Fortschritt besser verfolgen zu können. Offene Kommunikation und gegenseitige Unterstützung im Team tragen ebenfalls dazu bei, die Motivation aufrechtzuerhalten. Eine gemeinsame Vision und das Verständnis für die Bedeutung des Projekts können dazu beitragen, dass sich alle Mitglieder mit dem Projekt identifizieren und sich engagieren. Letztendlich erleichtert eine gemeinsame Motivation nicht nur die Zusammenarbeit, sondern hilft dem Team auch, Hindernisse zu überwinden und das Projekt erfolgreich abzuschließen.

2.6 Erlerntes

Während der Projektentwicklung wurden wertvolle Erfahrungen gesammelt, die für zukünftige Projekte von Bedeutung sein können. Es wurde ein klares Verständnis für die Bedeutung einer strukturierten Planung von Anfang an entwickelt. Dadurch konnten potenzielle Herausforderungen frühzeitig erkannt und bewältigt werden. Die enge Zusammenarbeit im Team machte deutlich, wie wichtig eine offene Kommunikation und eine klare Aufgabenverteilung sind, um Effizienz und Produktivität zu steigern. Flexibilität und Anpassungsfähigkeit an sich ändernde Anforderungen sind entscheidende Fähigkeiten für eine erfolgreiche Projektentwicklung. Diese Erkenntnisse dienen als wertvolle Grundlage für zukünftige Projekte, sei es in der Arbeitswelt oder in weiteren Schulprojekten, die mit gestärktem Selbstvertrauen und einem besseren Verständnis der Anforderungen und Prozesse angegangen werden können.

Kapitel 3

Raspberry Pi (Dobrev Martin)

Der Raspberry Pi ist ein kompakter Einplatinencomputer, der sich ideal für die Entwicklung von IoT-Projekten (Internet of Things) eignet. Durch seine geringe Größe und seine leistungsfähige Hardware bietet er zahlreiche Möglichkeiten für die Umsetzung verschiedenster Softwareprojekte. Die Vielseitigkeit und Anpassungsfähigkeit des Raspberry Pi machen ihn zu einem wichtigen Werkzeug für Entwickler, die eine kostengünstige und dennoch leistungsstarke Plattform suchen. Er unterstützt eine Vielzahl von Programmiersprachen und Betriebssystemen, darunter C#, C++, Java und andere, so dass Entwickler ihre Projekte in ihrer bevorzugten Umgebung umsetzen können. Das Betriebssystem Ubuntu Server 22.04 LTS bietet eine stabile und zuverlässige Plattform für den Einsatz auf dem Raspberry Pi. Die LTS (Long Term Support) Version garantiert langfristigen Support und Updates für Sicherheitspatches und Bugfixes, was besonders für den Einsatz in Produktionsumgebungen oder langfristigen Projekten wichtig ist. Die geringen Ressourcenanforderungen von Ubuntu Server ermöglichen eine effiziente Nutzung der begrenzten Hardware des Raspberry Pi, während gleichzeitig eine breite Palette von Serveranwendungen und -diensten unterstützt wird.[4]

3.1 GPIO-Pins

Die GPIO-Pins (General Purpose Input/Output) des Raspberry Pi ermöglichen die Interaktion mit der physikalischen Umgebung durch externe Hardware. Sensoren, Aktoren und andere Peripheriegeräte können angeschlossen und gesteuert werden, wodurch Projekte im Bereich Embedded Systems und IoT realisiert werden können. In dieser Diplomarbeit wurde überlegt, mit den Pins zu arbeiten, aber eine große Reichweite konnte nur mit drahtlosen Technologien realisiert werden.

Die Pins sind für Erweiterungen offen gelassen, falls sie bei der Umsetzung dieser Diplomarbeit oder deren Erweiterung benötigt werden. Das Gehäuse von SLOTech Components, welches auch als 3DDruckmodell verfügbar ist, hält die Pins sowie den Kameraanschluss frei.[5]

Kapitel 4

Kommunikationsprotokoll (Dobrev Martin)

Das in dieser Diplomarbeit verwendete Kommunikationsprotokoll spielt eine zentrale Rolle für die Interaktion zwischen dem Broker und den Endgeräten sowie zwischen dem Broker und der Webapplikation. Es definiert die Struktur der übertragenen Nachrichten sowie die Handhabung dieser Nachrichten während des gesamten Kommunikationsprozesses.

4.1 Message Queuing Telemetry Transport Protokoll (MQTT)

Im Rahmen dieser Diplomarbeit wird das Message Queuing Telemetry Transport (MQTT) Protokoll zur Datenübertragung verwendet. Der Broker dient als zentraler Knoten dieses Protokolls. Er empfängt die Daten von den Clients, speichert diese und stellt sie über die REST API der Webapplikation zur Verfügung. Eine direkte Kommunikation zwischen den Clients findet nicht statt, stattdessen verbinden sich die Clients mit dem Broker. Dieses Modell unterstützt eine effiziente und skalierbare Datenübertragung und wird auch als Publish-Subscribe-Modell bezeichnet.[6]

4.1.1 Publish-Subscribe-Modell

Zwei grundlegende Konzepte sind für das MQTT-Protokoll von Bedeutung: Publish und Subscribe. Beim Publizieren (Publishing) von Nachrichten senden Clients ihre durch das Kommunikationsprotokoll vordefinierten Nachrichten an den Broker. Dabei werden Topics verwendet, um die Nachrichten, die beim Broker ankommen, zu gruppieren. Beispielsweise können Sensoren Nachrichten an bestimmte Topics senden, die auch hierarchisch aufgebaut sein können. Beispielsweise könnte ein Temperatursensor im Badezimmer Nachrichten an ein Topic mit dem Pfad „Haus/Obergeschoss/Bad/Temperatur“ senden, während ein CO₂-Sensor Nachrichten an ein anderes Topic mit dem entsprechenden Pfad senden würde.

Diese hierarchische Struktur wird in der MQTT-Implementierung dieser Diplomarbeit ausgenutzt. Die Topics, die für die Kommunikation zwischen den ESPs von SLOTH Components und dem Broker verwendet werden, sind die folgenden:

- GUID des Endgerätes (nicht hierarchisch)
- Status/Info-, Warnungs- oder FehlerTopic
- Configuration/GUID des Endgerätes

MQTT benötigt aber auch Subscriber. Das Abonnieren (Subscribing) von Topics bezieht sich auf das Empfangen von Nachrichten. Wenn ein Client ein Topic abonniert, erhält er jede Nachricht, die in diesem Topic veröffentlicht wird.

Beispielsweise kann ein Aktor ein Topic abonnieren und bei bestimmten Nachrichten entsprechende Aktionen ausführen. Eine wichtige Eigenschaft des MQTTProtokolls ist, dass Topics automatisch erstellt werden, wenn sie noch nicht existieren. Dies ist für die Zwecke dieser Implementierung sehr nützlich, da wir nicht alle Topics im Voraus definieren müssen. Beim Abonnieren eines Topics muss der Name des Topics bereits bekannt sein, aber mit Hilfe eines Platzhalters kann ein Host/Gerät alle Topics abonnieren, die existieren, oder auf der entsprechenden hierarchischen Ebene.

Dieser Platzhalter ist das Zeichen '#', das dies ermöglicht. Es ist auch möglich, das Topic 'HierarchicalLevelEins/#' zu abonnieren, um alle Sub-Topics unter HierarchicalLevelEins zu abonnieren. Dies macht der Broker und ist der entscheidende Punkt für die MQTT Implementierung dieser Diplomarbeit.[6]

4.1.2 Quality of Service (QoS)

MQTT basiert auf dem TCP-Übertragungsprotokoll und bietet drei verschiedene Qualitätsstufen (Quality of Service, QoS) zur Steuerung der Datenübertragung:

- QoS 0 (At most once): Auch bekannt als „Fire and Forget“. Hier wird eine Nachricht genau einmal gesendet und dann vergessen. Da hier nicht auf eine Empfangsbestätigung gewartet wird, können Nachrichten verloren gehen oder unvollständig ankommen. Diese Qualitätsstufe verursacht den geringsten Netzwerkverkehr.
- QoS 1 (At least once): Der sendende Host sendet seine Nachricht mehrmals, um sicherzustellen, dass sie vom Zielhost empfangen wird. Dies verursacht mehr Verkehr bei der Übertragung der Nachricht, aber es wird sichergestellt, dass die Nachricht mindestens einmal ankommt. Es kann jedoch vorkommen, dass eine Nachricht mehrmals ankommt.
- QoS 2 (Exactly Once): Um sicherzustellen, dass eine Nachricht nur einmal beim Broker ankommt, wird ein Four-Way Handshake verwendet. Dies erhöht den Datenverkehr etwas, garantiert aber eine hohe Zuverlässigkeit. Der Sensor sendet seine Pakete, bis der Broker sie bestätigt. Durch ein Flag in der Nachricht kann der Ziel-Host aufgefordert werden, die empfangenen Nachrichten auf eine zu reduzieren, indem er die restlichen löscht.

In dieser Diplomarbeit wird QoS 2 (Exactly Once) verwendet, da alle beim Broker eintreffenden Nachrichten in einer SQLite Datenbank gespeichert werden und daraus auf der Webapplikation eine Anzeige mit den bisher gemessenen Werten erzeugt wird. Dies kann bei späterer Entwicklung leicht geändert werden, ist also für Änderungen erweiterbar, aber nicht empfehlenswert, außer wenn die Anzeige der Statistikhistorie wegfallen soll.[7]

4.1.3 Retained Messages

Retained Messages sind spezielle Nachrichten, die auf dem Broker gespeichert bleiben, bis ein neuer Client das Topic abonniert. Dies sind normale MQTT Nachrichten, aber mit dem Retain Flag gesetzt. Dies bewirkt, dass diese Nachrichten nicht einfach vom Broker weitergeleitet und nach dem Empfang gelöscht werden, sondern gespeichert und an jedes neu abonnierte Gerät gesendet werden. Dadurch kann im Fall dieser Diplomarbeit sichergestellt werden, dass auch wenn ein SLOTH Components Gerät zu einem bestimmten Zeitpunkt inaktiv ist, es die Nachricht erhält.

4.2 Aufbau des Kommunikationsprotokolls mit den ESPs

Die Kommunikation zwischen dem Broker und den ESP Mikrocontrollern von SLOTH Components besteht ausschließlich aus MQTT Nachrichten. Die Struktur dieser Nachrichten ist sehr klar und eindeutig definiert und auch entscheidend für die Funktionalität der geschriebenen Software. Die Struktur ist wie folgt und muss unbedingt eingehalten werden:

- Topic: „GUID des Endgerätes“
 - Nachricht: „GUID;Raum;Value;Unit;TypeOfDevice“
- Thema: „Status/Info-, Warning- oder ErrorTopic“
 - Nachricht: „GUID;PlainText“

- Thema: „Konfiguration/GUID des Endgeräts“
 - Nachricht(nur vom Broker): „GUID;Configuration-options“

Die Interpretation dieser Nachrichten ist im Code beider Software (Web-Applikation & Broker) fest programmiert und die Erweiterung des Protokolls für die Schnittstelle zu den Endgeräten ist genau zu implementieren.

4.3 Kommunikationsprotokoll mit der Web-Anwendung

Im Broker-Projekt dieser Diplomarbeit werden alle Daten der Endgeräte gespeichert und der Web-Applikation zur Verfügung gestellt, um eine entsprechende Funktionalität zu gewährleisten. Dies beinhaltet nicht nur die Speicherung von Nachrichten, sondern auch die Verwaltung von Gerätekonfigurationen und anderen relevanten Informationen. Zur effizienten Verwaltung dieser Daten wird eine Datenbank erstellt, die als zentraler Datenspeicher dient und eine schnelle Verfügbarkeit der Daten gewährleistet.

4.3.1 API

Um die gespeicherten Daten der Web Applikation zur Verfügung zu stellen ist eine API eingerichtet, welche es der Web-App ermöglicht, mittels HTTP GET Methoden, Daten aus der Datenbank anzufordern. Diese API stellt eine Vielzahl von Endpunkten bereit, über die die Web-App Daten abrufen, speichern, modifizieren kann und auch diverse Nachrichten verschicken kann. Die API wird durch die Nutzung von .NET Rest API realisiert und integriert sich nahtlos in das MQTT Broker Back- End Teil.

4.3.1.1 HTTP GET Methoden

Die HTTP GET-Methode ist ein grundlegendes Element von HTTP, das für den Austausch von Daten im Internet verwendet wird. Diese Methode wird häufig verwendet, um Informationen von einem Server abzurufen, sowie in dieser Diplomarbeit. Bei einer HTTP GET-Anfrage sendet der Client eine Anfrage an den

Server, um bestimmte Ressourcen wie Webseiten, Bilder, Dokumente oder Daten anzufordern. Die GET-Anfrage enthält normalerweise eine URL (Uniform Resource Locator), die die Adresse der gewünschten Ressource angibt. Um die URL für alle im lokalen Netzwerk erreichbar zu machen, wird eine Reverse-Proxy eingesetzt, welche mittels der Web-Server Anwendung Nginx realisiert wird.

4.3.1.2 HTTP POST Methoden

Die HTTP POST-Methode ist ein weiteres wichtiges Element von HTTP, das für den Austausch von Daten im Internet verwendet wird. Im Gegensatz zur GET-Methode, die hauptsächlich für das Auffordern des Austauschs an Informationen verwendet wird, wird die POST-Methode hauptsächlich für das Senden von Daten an einen Server verwendet. Diese Methode wird in dieser Diplomarbeit insbesondere für das Auslösen von Aktionen wie das Versenden von Nachrichten an ESPs verwendet.

4.4 Zusammenspiel beider Schnittstellen

Um eine vollständige Kommunikation und Funktionalität der Web Applikation zu gewährleisten, müssen beide Schnittstellen zusammenspielen. Die Schnittstelle zu den SLOTH Components Endgeräten ist optimiert und effizient, um alle Nachrichten aller Endgeräten bearbeiten zu können. Das Broker Back-End leitet empfangene Nachrichten der Web Applikation zur Anzeige, sowie eventueller Abspeicherung.

Kapitel 5

Linux (Dobrev Martin)

5.1 Betriebssystem Auswahl

Im Bezug auf dem Broker hat sich unsere Diplomarbeitsgruppe mit mehreren Betriebssystemen auseinander gesetzt, bis die endgültige Entscheidung gekommen ist. Am Anfang wurde das native Betriebssystem eines Raspberry Pis probiert, nämlich Raspberry Pi OS, aber auch als Raspbian bekannt. Das Betriebssystem stellt sich aber jedoch als Over-Kill für die Zwecke dieses Projektes, da sie zu viel „Bloatware“ vorinstalliert hat.

Der größte Grund, wieso dieses Betriebssystem sich als Over-Kill darstellt, ist weil sie ein Desktop Environment hat, sowie auch ein Window Manager. Das bedeutet im Prinzip, dass man eine bekannte Desktop Oberfläche hat, welche aber sehr viele Ressourcen verschwendet. Am Ende des Projektes stellt es sich auch heraus, dass diese Entscheidung eine der wichtigsten ist, da sehr hohe RAM Auslastung mit beiden Backends gegeben ist (eine der Webapp, die andere der ESP-Schnittstelle). Mit dieser Probe am Anfang konnte das Team die Anforderungen für das Betriebssystem definieren. Sie sind folgende:

- So wenig Speicherplatzauslastung(Bloatware) wie möglich
 - Dies ist weil wir SD-Karten mit niedrigen Speicherkapazitäten verwenden wollen

- So wenig RAM-Auslastung wie möglich, vom Betriebssystem aus
 - Hier behalten wir beide Backends Hinterkopf, da sie sehr hohe Auslastung mit sich bringen
- .NET Unterstützung
 - Unsere ESP-Schnittstelle wird mit C# entwickelt, weswegen wir .NET Support brauchen, um das Projekt ausführen zu können
- LTS Support
 - LTS Support war uns schon wichtig, aber es wäre kein Show-Stopper, hätten LTS nicht. LTS steht für Long-Term Support, was bedeutet, dass das Betriebssystem Updates für die, vom Entwickler, vorgesehene Zeit Updates bekommen würde

Alleine wegen der .NET Unterstützung haben sich Debian-basierende Linux Distributionen angeboten, da sie alle den „apt“ Package Manager vorinstalliert haben. Der apt Package Manager ist eine zentrale Softwareverwaltungstool, unter Linux auch Pakete genannt, für Debian-basierte Linux-Distributionen wie Debian selbst oder Ubuntu. Der Name „apt“ steht für „Advanced Package Tool“. Mit diesem Package Manager können Softwarepakete gesucht, installiert, aktualisiert und entfernt werden, ohne sich um Abhängigkeiten und andere technische Details kümmern zu müssen, da apt es für einen macht. .NET hat von Natur aus sehr viele Package Abhängigkeiten, aber apt erleichtert die Installation deutlich.

Der apt Package Manager bietet auch Funktionen zur Verwaltung von Repositories, aus denen Softwarepakete bezogen werden können. Dies ermöglicht es, verschiedene Softwarequellen hinzuzufügen oder zu entfernen. Dies war ein entscheidender Punkt am Anfang, für den Fall, dass man keine .NET Unterstützung von apt Repositories bekommen hat, könnte man Third Party Repositories einbinden, um eine ältere Version herunterzuladen. Dies stellt sich später in der Diplomarbeit als nicht notwendig.

Ubuntu hat sich als nächste Entscheidung angeboten und nach ein bisschen Recherche wurde Ubuntu 22.04 ausprobiert, ohne Desktop Environment, somit auch kein Window Manager. Wir waren hierbei zufrieden, aber Ubuntu als Betriebssystem ist

bekannt für Hintergrundprozesse, die das Verhalten einem erleichtern. Hier hat sich dann Ubuntu Server 22.04, auch ohne Desktop Environment, als eine passende Wahl erwiesen. Ubuntu Server ist eine spezielle Variante von Ubuntu, die für den Einsatz als Server optimiert ist. Im Gegensatz zur Desktop-Version verzichtet Ubuntu Server auf grafische Benutzeroberflächen und richtet sich stattdessen auf die Bereitstellung von Diensten wie Webserver, Datenbanken und mehr über die Kommandozeile aus.

Aber wieso ist Ubuntu Server so eine gute Wahl? Ubuntu Server 22.04 ist perfekt für den Einsatz in die Diplomarbeit aus folgenden Gründen:

- Jede geradzahlige Version von Ubuntu (und auch vielen anderen Linux Distributionen) ist eine LTS Version
 - LTS wurde priorisiert, um das Ergebnis der Diplomarbeit nachhaltig und zukunftsorientiert zu halten.
- Ubuntu Server hat das Terminal als Benutzeroberfläche
 - Das Verzicht auf eine GUI hilft dabei, Ressourcen zu sparen, im Sinner von RAM und Speicherplatz. Die Verwendung des Terminals ermöglicht eine direkte Interaktion mit dem System, ohne zusätzliche Overheads durch eine GUI.
- Ubuntu Server bietet eine breite Unterstützung an Serveranwendungen
 - In den Zügen der Diplomarbeit stellt es sich herausgestellt, dass man Nginx als Proxy einsetzen muss, um die ESP-Schnittstelle für die Web-App Implementierung auf konstanten Ports zugreifbar macht. Weitere Tools für das Monitoring der Hardware Ressourcen sind auch durchaus notwendig. Die Softwareunterstützung von Ubuntu Server ist sehr hilfreich dabei.
- Vertrautheit mit Debian-Derivaten
 - Aus der HTL-Ausbildung besteht Vertraut mit der Linux Distribution Debian, worauf Ubuntu Server basiert. Dies hat die Konfiguration um einiges erleichtert.

5.1.1 Installation

Mit der Entscheidung für das richtige Betriebssystem, kann die Installation dieses anfangen. Ein Betriebssystem auf einem Raspberry Pi zu installieren ist mit dem „Raspberry Pi Imager“ leicht gemacht. Der Raspberry Pi Imager ist eine Software von der Raspberry Pi Foundation, welches sich pur mit der Installation und Vorkonfigurationen der diversen Linux Distributionen auf dem Raspberry Pi beschäftigt. Die Installation von Ubuntu Server auf dem Raspberry Pi 4 mit 4GB RAM besteht aus folgenden Schritten:

1. Installation von Raspberry Pi Imager.
2. Darin das Betriebssystem auswählen. - Ubuntu Server 22.04.3 LTS (64-Bit)
3. SD-Karte auswählen, die sich auf dem Gerät befindet, wo der Imager läuft.
4. In den erweiterten Einstellungen SSH aufsetzen, mit Passwort Authentifizierung.
5. Hostname für das erkennen im lokalen Netzwerk setzen.
6. User anlegen.
7. WLAN SSID und WLAN Passwort angeben.

Mit dem Aussetzen von SSH und der Verbindung zum lokalen Netz, kann an dem Raspberry Pi gearbeitet werden, ohne Bildschirm Anzeige auf einem Monitor. Per SSH verbindet man sich zum kompakten Computer und die Terminal Session ist schon bereit für die Arbeit.

5.2 Eingesetzt Linux Tools

Auf Linux bezogen, werden ein paar Tools gebraucht, um die Entwicklung der Diplomarbeit zu erleichtern. Diese Tools sind alle über den apt Package Manager verfügbar zu installieren und können alle mit folgendem Kommando installiert werden:

Die Tools die wir verwendet haben sind folgende:

- **btop**: Ein Tool zur Überwachung der Systemressourcen, das eine benutzerfreundliche Oberfläche bietet und es ermöglicht, CPU-, Speicher- und Netzwerkauslastung, gemeinsam, in Echtzeit zu überwachen.[8]
- **dotnet-runtime-7.0**: Bietet die Laufzeitumgebung für .NET-Anwendungen, die unter .NET 6.0 entwickelt wurden. Enthält die erforderlichen Komponenten für die Ausführung von SSH Anwendungen, jedoch nicht für die Entwicklung.
- **dotnet-sdk-7.0**: Das Software Development Kit (SDK) für die Entwicklung von .NETAnwendungen unter .NET 6.0. Enthält Compiler, Laufzeitumgebung, Entwicklertools und Bibliotheken für die Entwicklung, Tests und Veröffentlichung von .NET-Anwendungen. Wird für das Kompilieren und Build-en des ESP-Schnittstelle-Projektes.[9]
- **neovim**: Ein erweiterbarer Texteditor für die Kommandozeile, der eine moderne Alternative zum klassischen Vim-Editor darstellt. Neovim bietet Funktionen wie Syntaxhervorhebung und automatische Vervollständigung, was die Entwicklung und Bearbeitung von Textdateien auf dem Raspberry Pi erleichtert. Wird für das Modifizieren von Teilen des Projektes verwendet zum Zeitpunkt des Debugging.[10]
- **ranger**: Ein Dateimanager für die Kommandozeile, der es ermöglicht, Dateien und Verzeichnisse zu durchsuchen, zu kopieren, zu verschieben und zu bearbeiten, alles über die Kommandozeile. Ranger bietet eine intuitive Benutzeroberfläche für die Dateiverwaltung auf dem Raspberry Pi.[11]
- **svn**: Das Paket „svn“ ist ein Versionskontrollsystem, das für die Verwaltung von Dateien und Verzeichnissen in einem Projekt verwendet wird. Es ermöglicht Benutzern Änderungen zu verfolgen, verschiedene Versionen von Dateien zu speichern und Zusammenarbeit in einem Team zu erleichtern.

- – Subversion, oft als „svn“ abgekürzt, ist ein Open-Source-Versionierungssystem, das zur Verfolgung von Änderungen an Dateien und Verzeichnissen in einem Projekt verwendet wird. Es bietet Funktionen wie das Einchecken, Auschecken, Aktualisieren und Vergleichen von Dateien sowie das Verwalten von Zweigen und Zusammenführungen für die Versionskontrolle.[12]
- nginx: Nginx ist ein beliebter Webserver unter Linux, der aber auch als Reverse Proxy eingesetzt werden kann. In dieser Diplomarbeit wird er als Reverse Proxy eingesetzt, um die Schnittstelle für die Web-Applikation, von der Seite des MQTT Broker Projektes, im lokalen Netz erreichbar zu machen. Dies ermöglicht spätere Skalierung des Projektes, indem die Web- Applikation außerhalb des Raspberry Pis bereitgestellt wird.[13]

SSH (Secure Shell) ist ein verbreitetes Protokoll für sichere Remote-Verbindungen mit entfernten Computern über ein Netzwerk. Es ermöglicht Benutzern eine verschlüsselte Verbindung herzustellen, um auf einen entfernten Computer zuzugreifen und Befehle auszuführen oder Dateien zu übertragen. SSH wird häufig verwendet, um auf entfernte Server oder Geräte zuzugreifen, sowie auch für die Verwaltung von Linux-Systemen, sowie in dieser Diplomarbeit. SSH bietet zwei bekannte und breit eingesetzte Methoden Zugriff sich zu verschaffen.

5.3 SSH

1. Mittels Username & Passwort: Die Authentifizierung hierbei ist nicht sehr optimal, vor allem wenn es über Serververwaltung geht, für diese Diplomarbeit aber durchaus gut einsetzbar und optimal, da es Zugriffe aus diversen Hosts geben wird.
2. Public Key Authentifizierung: Diese Methode ist sehr sicher, da Schlüssel-Paare immer eindeutig und eigenartig sind, weswegen sich ein Host nur dem Schlüssel Paar anmelden darf. Diese Authentifizierungsmethode ist für die Entwicklung dieser Diplomarbeit aber nicht zwingend nötig.

Es ist wichtig anzumerken, dass SSH für das Endprodukt irrelevant sein sollte, außer der Benutzer will sich per SSH auf dem Raspberry Pi verbinden. Hierfür wird beim Endprodukt ein SSH User vorangelegt sein, um dies zu ermöglichen.

5.4 Netzwerkkonfiguration Herausforderungen

Die Administration über den Raspberry Pi war im Rahmen der Diplomarbeit kein Problem, die einzige Herausforderung für das Team waren Probleme mit der Konfiguration der Netzwerkschnittstelle unter dem gewählten Betriebssystem Ubuntu Server.

Nach einigen Diskussionen wurde festgestellt, dass bei einer Verbindung über LAN mit einem noch unbekannten Netzwerk die Netzwerkkonfiguration mit folgenden Elementen manuell vorgenommen werden muss, da diese nicht automatisch aufgerufen werden:

- DHCP-Lease-IP-Adresse
- Default Gateway des Netzwerks
- Subnetzmaske des Netzes
- Falls vorhanden, die IP-Adresse des gewünschten DNS-Servers

Für die Verbindung zu einem WLAN-Netzwerk empfehlen wir die Verwendung des Tools 'nmcli'. Mit nmcli wird die Angabe der SSID und des Passwortes für die Authentifizierung wesentlich erleichtert, da das Tool diese Daten selbst in die Konfigurationsdateien schreibt. Die Netzwerkkonfiguration muss dann wie oben beschrieben erneut eingelesen werden, falls nmcli diese Informationen nicht automatisch einliest.

5.4.1 nmcli

Das Linux Terminal Tool nmcli ist ein Netzwerkkonfigurationstool für Linux. Dieses Tool ist mit allen gängigen Paketmanagern unter Linux verfügbar, da es sehr häufig in der Netzwerkadministration verwendet wird.

Mittels folgendem Terminal-Kommando kann man sich mit nmcli zu einem WLAN verbinden:

```
nmcli interface wlan verbinden <SSID> passwort <Passwort>
```

Dabei steht <SSID> für den Namen des WLAN-Netzwerks und <Passwort> für das entsprechende Passwort. Durch Ausführen dieses Befehls wird eine Verbindung zum angegebenen WLAN-Netzwerk hergestellt, wobei die Authentifizierungsinformationen automatisch in die Konfigurationsdateien geschrieben werden.[14]

5.5 Systemd

Systemd ist ein System- und Servicemanager für Linux-Betriebssysteme, der eine effiziente Verwaltung von Systemdiensten ermöglicht. In dieser Diplomarbeit wird Systemctl verwendet, um einen Daemon für das C#-Programm zu erstellen und zu steuern. Ein Daemon ist ein Hintergrundprozess, der ohne direkte Benutzerinteraktion läuft. Durch die Verwaltung des Daemons mit Systemd wird sichergestellt, dass der Daemon bei jedem Systemstart automatisch gestartet wird und kontinuierlich im Hintergrund läuft. Systemd und systemctl sind auf Ubuntu Server vorinstalliert und vorkonfiguriert, um alle notwendigen Daemons des Betriebssystems zu verwalten.[15]

5.5.1 Systemctl

Systemctl ist ein mächtiges Dienstverwaltungswerkzeug, das Teil des systemd-Systems ist. Es ermöglicht die Verwaltung von Diensten, Sockets, Slices, Targets und anderen Systemkomponenten. In dieser Diplomarbeit wird Systemctl verwendet, um einen Daemon für das C#-Programm zu erstellen und zu steuern. Durch die Verwendung von Systemctl wird sichergestellt, dass unser Programm als Hintergrundprozess, also als Daemon, ohne direkte Benutzerinteraktion ausgeführt wird.[15]

Um dies zu erreichen, wird eine Servicedatei erstellt. Diese enthält die Systemd-Konfiguration für den Dienst und definiert, wie der Dienst gestartet, gestoppt, neu gestartet und verwaltet werden soll. Anschließend wird der Dienst mit Systemctl aktiviert und gestartet. Dadurch wird sichergestellt, dass der Dienst beim Systemstart automatisch gestartet wird und im Hintergrund läuft.

Durch die Integration von Systemctl in die Projektentwicklung wird eine zuverlässige Startfunktionalität für das C#-Programm auf dem Raspberry Pi gewährleistet. Dies ermöglicht eine reibungslose und automatisierte Ausführung des Programms als Hintergrundprozess, was für die Anwendung im Rahmen der Diplomarbeit von entscheidender Bedeutung ist.

Mit dem Terminal-Tool „journalctl“ unter Linux können die Logs und verschiedenste Informationen in Form von Logs eingesehen werden. Mit diesem Befehl können die Logs der C#-Applikation in einem bestimmten Zeitintervall angezeigt werden:

```
journalctl -u slothBackEnd.service --since <Beginn des Zeitintervalls> --until <Ende des Zeitintervalls>
```


Teil des Brokers & Datenbankverwaltung

Pacyga Mateusz

Kapitel 6

Broker & Datenbankverwaltung (Mateusz Pacyga)

6.1 Allgemein

6.1.1 Eigener Bereich

Meine Aufgabe in dieser Arbeit war es, ein Datenbank-Modell zu entwerfen und dies dann umzusetzen, den Broker soweit es geht möglichst effizient zu halten, sowie die Hilfe bei der Abwicklung Kommunikationsschnittstelle zwischen den Sensoren und der Web-App.

- Datenbank: Mit Hilfe des Tools Visual Paradigm habe ich ein Entity-Relationship-Diagramm (ERD) entworfen und in SQLite umgesetzt. Da diese Datenbank mit Testdaten gefüllt war, die eine Imitation von echten Sensordaten darstellen sollten, habe ich im Laufe des Projekts die Datenbank mit neuen Inhalten erweitert und aufgetretene Fehler soweit möglich behoben. Die meiste Arbeit bestand darin, die richtigen Datensätze zu filtern, die vom Webserver angefordert werden. Dies erfordert nicht nur SQL-Kenntnisse,

sondern auch Kommunikations- und Kommunikations- und Teamfähigkeit, da in diesem Bereich die Datenbank eng mit der Webanwendung verknüpft ist. Der Benutzer soll eine klare Übersicht haben und die benötigten Informationen mit einem Klick abrufen bzw. bearbeiten können.

- Schnittstelle Sensoren/Web-App: Die Kommunikation erfolgt über eine MQTT-Verbindung. Sobald der MQTT-Client eine Verbindung mit dem MQTT-Broker aufgebaut hat, kann der Client entweder Nachrichten veröffentlichen, bestimmte Nachrichten abonnieren oder beides tun. Durch bestimmte Aktionen des Benutzer über die Web-Applikation werden bestimmte Aktionen ausgelöst. So wird z.B. beim Erstellen einer neuen Szene über die Web-App eine Nachricht an den Broker gesendet, der die neue Szene in der Datenbank speichert. Beim Empfang neuer Nachrichten wird überprüft, ob die Wenn dies der Fall ist, wird eine Aktion gesetzt (eine Nachricht, die dem Sensor mitteilt, was er zu tun hat). Mit der MQTTx App kann man einen MQTT Client emulieren. Dies ist nützlich, um das eigene Programm während der Entwicklung gründlich zu testen. [6]

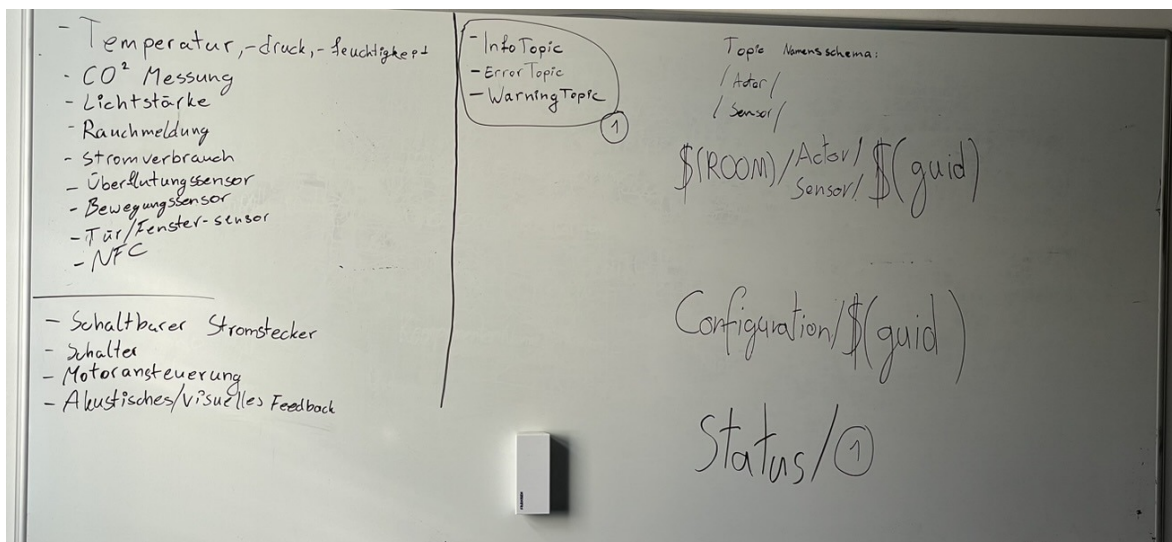


Abbildung 6.1: Das Bild veranschaulicht unsere Definition von den verschiedenen Topics

6.1.2 Verwendete Technologien

Während des Projekts habe ich Visual Studio 2022 für die Entwicklung der Schnittstelle und die Erstellung und Konfiguration der Datenbank verwendet. Um mit der Datenbank in Visual Studio 2022 arbeiten zu können, war eine Erweiterung (SQLite and SQL Server Compact Toolbox) notwendig. Das Entity-Relationship-Diagramm habe ich mit Hilfe von Visual Paradigm erstellt. Ich habe mich für dieses Modellierungswerkzeug entschieden, weil ich bereits einige Erfahrungen damit im Unterricht gesammelt habe.

6.1.3 Visual Studio 2019

Visual Studio ist eine leistungsstarke integrierte Entwicklungsumgebung (IDE), die von Microsoft entwickelt wurde. Sie bietet eine umfangreiche Palette von Funktionen, um Softwareprojekte zu entwickeln, zu debuggen und zu verwalten.

Die IDE enthält einen robusten Code-Editor mit Funktionen wie Syntaxhervorhebung, automatischer Codevervollständigung (IntelliSense), Refactoring-Werkzeugen und Codevergleich.

Projekte werden in Visual Studio in Lösungen organisiert. Eine Lösung kann verschiedene Projekte wie Bibliotheken, Anwendungen oder Tests enthalten. Die Projekte werden in speziellen Dateien (.sln und .csproj für C#-Projekte) gespeichert.

Visual Studio bietet umfassende Debugging-Tools, mit denen Entwickler ihren Code Schritt für Schritt ausführen, Variablenwerte überwachen und Fehler in ihrer Anwendung identifizieren können.

Visual Studio ermöglicht durch Integration mit verschiedenen Versionsverwaltungssystemen wie Git, SVN und TFS das direkte Verwalten, Comitten, Pullen und Pushen von Projekten aus der IDE heraus.

Für die Erstellung von Anwendungen mit grafischer Benutzeroberfläche bietet Visual Studio Designer für verschiedene Plattformen wie Windows Forms, WPF (Windows Presentation Foundation) und Xamarin für plattformübergreifende Anwendungen.

Visual Studio ist eine sehr erweiterbare IDE, die die Integration von Erweiterungen und Plugins aus dem Visual Studio Marketplace unterstützt, um die Funktionalität zu erweitern.

Mit Visual Studio können Projekte kompiliert, getestet und bereitgestellt werden. Es bietet auch Unterstützung für Continuous Integration und Continuous Deployment (CI/CD) über Azure DevOps und andere Dienste.

Visual Studio unterstützt eine Vielzahl von Programmiersprachen wie C#, VB.NET, C++, F#, Python und JavaScript und ermöglicht die Entwicklung von Anwendungen für verschiedene Plattformen wie Windows, das Web, mobile Geräte und die Cloud.[16]

6.1.4 Microsoft.Data.Sqlite

Die Erweiterung „Microsoft.Data.Sqlite“ ist eine Bibliothek, die es Entwicklern ermöglicht, SQLite-Datenbanken in ihren .NET-Anwendungen zu nutzen, unabhängig von der Art der Anwendung (Desktop, Web oder Mobil). Hier sind einige wichtige Aspekte dieser Erweiterung:

- Die Unterstützung von SQLite: Diese leichtgewichtige und serverlose SQL-Datenbank-Engine ist ideal für Embedded-Anwendungen, mobile Apps und kleinere Desktopanwendungen, da sie ohne separate Serverinstallation arbeitet. „Microsoft.Data.Sqlite“ erleichtert die Verwendung von SQLite in .NET-Anwendungen.
- .NET-Integration: Speziell für .NET-Anwendungen entwickelt, bietet die Erweiterung eine nahtlose Integration mit dem .NET-Ökosystem. Sie kann in Projekte eingebunden werden, die auf .NET Framework, .NET Core oder .NET 5/6 basieren.
- Plattformübergreifend: „Microsoft.Data.Sqlite“ ist plattformübergreifend und unterstützt die Entwicklung von Anwendungen für Windows, macOS und Linux.

- Einfache API: Die Bibliothek bietet eine einfache und intuitive API für die Arbeit mit SQLite-Datenbanken in .NET-Anwendungen. Sie ermöglicht das Herstellen von Datenbankverbindungen, das Ausführen von Abfragen, das Verwalten von Transaktionen und die Datenbankverwaltung.
- Leistungsfähigkeit: Obwohl SQLite eine leichtgewichtige Datenbank-Engine ist, bietet sie dennoch gute Leistung und Skalierbarkeit. „Microsoft.Data.Sqlite“ optimiert die Interaktion mit SQLite für eine effiziente Datenbanknutzung.
- NuGet-Paket: Die Erweiterung wird als NuGet-Paket bereitgestellt, das einfach zu Ihrem Projekt hinzugefügt werden kann, um die Installation und Aktualisierung zu vereinfachen.
- Open Source: „Microsoft.Data.Sqlite“ ist Open Source und wird auf GitHub unter der MIT-Lizenz gehostet. Dadurch haben Sie Zugriff auf den Quellcode, können Beiträge leisten und Fehler melden.

Insgesamt ist „Microsoft.Data.Sqlite“ eine nützliche Erweiterung für Entwickler, die SQLite-Datenbanken in ihren .NET-Anwendungen nutzen möchten. Sie bietet eine einfache Integration, gute Leistung und plattformübergreifende Unterstützung für die Anwendungsentwicklung.[17]

6.1.5 MQTTnet

Die Erweiterung „MQTTnet“ ist eine Bibliothek, die es Entwicklern ermöglicht, das MQTT (Message Queuing Telemetry Transport) Protokoll in ihren .NET-Anwendungen zu implementieren.

MQTT ist ein leichtgewichtiges, zuverlässiges und effizientes Protokoll, das häufig in IoT (Internet of Things), Messaging-Anwendungen und anderen verteilten Systemen verwendet wird. Hier sind einige wichtige Aspekte dieser Erweiterung:

- Die Unterstützung von MQTT: MQTTnet erleichtert die Verwendung des MQTT-Protokolls in .NET-Anwendungen. Es implementiert die Spezifikation des MQTT-Protokolls und bietet Funktionen für die Verbindung zu MQTT-Brokern, das Abonnieren und Veröffentlichen von Nachrichten sowie die Verwaltung von QoS (Quality of Service) Ebenen.
- .NET-Integration: Die Erweiterung ist speziell für die Verwendung in .NET-Anwendungen entwickelt und bietet eine nahtlose Integration mit dem .NET-Ökosystem. Sie können sie in Projekte einbinden, die auf .NET Framework, .NET Core oder .NET 5/6 basieren.
- Plattformübergreifend: MQTTnet ist plattformübergreifend und unterstützt die Entwicklung von Anwendungen für Windows, macOS und Linux.
- Einfache API: Die Bibliothek bietet eine einfache und intuitive API für die Arbeit mit MQTT in .NET-Anwendungen. Sie können MQTT-Clients erstellen, Verbindungen zu MQTT-Brokern herstellen, Nachrichten abonnieren und veröffentlichen sowie andere MQTT-bezogene Aufgaben ausführen.
- Leistungsfähigkeit: MQTTnet ist darauf ausgelegt, eine gute Leistung und Skalierbarkeit zu bieten. Sie können MQTTnet verwenden, um zuverlässige und effiziente IoT- oder Messaging-Anwendungen zu entwickeln.
- Open Source: MQTTnet ist Open Source und wird auf GitHub unter der MIT-Lizenz gehostet. Das bedeutet, dass Sie den Quellcode einsehen, an Beiträgen arbeiten und Fehler melden können.
- NuGet-Paket: Die Erweiterung wird als NuGet-Paket bereitgestellt, das Sie einfach zu Ihrem Projekt hinzufügen können. Dadurch wird die Installation und Aktualisierung der Bibliothek vereinfacht.

Insgesamt ist „MQTTnet“ eine nützliche Erweiterung für Entwickler, die das MQTT-Protokoll in ihren .NET-Anwendungen verwenden möchten. Sie bietet eine einfache Integration, gute Leistung und plattformübergreifende Unterstützung für die Entwicklung von IoT-, Messaging- und anderen verteilten Anwendungen.[18]

6.1.6 ADO.NET

ASP.NET ist ein Framework für die Webentwicklung, das von Microsoft entwickelt wurde und auf der Programmiersprache C# basiert. Es ermöglicht die Erstellung leistungsstarker und dynamischer Webanwendungen sowie Websites.

ASP.NET wurde erstmals im Jahr 2002 eingeführt und hat sich seitdem kontinuierlich weiterentwickelt. Es besteht aus einer Reihe von Werkzeugen, Bibliotheken und APIs, die Entwicklern helfen, robuste und skalierbare Webanwendungen zu erstellen.

Ein Kernkonzept von ASP.NET ist das Model-View-Controller (MVC) Muster, das eine klare Trennung von Datenmodell, Präsentation und Geschäftslogik ermöglicht. Dies fördert eine modulare und gut strukturierte Codebasis, die leichter zu warten und zu erweitern ist.

ASP.NET bietet verschiedene Technologien zur Erstellung von Webanwendungen, darunter:

- ASP.NET Web Forms: Eine Technologie, die es Entwicklern ermöglicht, Webanwendungen mithilfe von Ereignishandlern und serverseitigen Steuerelementen zu erstellen. Web Forms bietet eine ähnliche Entwicklungserfahrung wie die Windows Forms-Entwicklung, was es Entwicklern erleichtert, traditionelle Desktop-Anwendungen in das Web zu übertragen.
- ASP.NET MVC: Ein Framework, das auf dem MVC-Muster basiert und eine strukturierte Entwicklung von Webanwendungen ermöglicht. Mit MVC können Entwickler die Anwendungslogik von der Benutzeroberfläche trennen und somit die Wartbarkeit und Testbarkeit des Codes verbessern.
- ASP.NET Web API: Eine Technologie zur Erstellung von HTTP-basierten Diensten, die von einer Vielzahl von Clients, einschließlich Webbrowsern und mobilen Anwendungen, konsumiert werden können. Web API ermöglicht die Entwicklung von RESTful Web Services, die Daten in verschiedenen Formaten wie JSON und XML zurückgeben können.

- **ASP.NET Core:** Eine plattformübergreifende Version von ASP.NET, die auf Linux, macOS und Windows ausgeführt werden kann. ASP.NET Core wurde entwickelt, um die Leistung und Skalierbarkeit von ASP.NET-Anwendungen zu verbessern und eine flexiblere Bereitstellung zu ermöglichen. Es unterstützt auch moderne Entwicklungstechniken wie Dependency Injection und Middleware-Pipeline.
- **State Management:** ASP.NET bietet Mechanismen zur Verwaltung des Zustands von Webanwendungen, einschließlich serverseitiger und clientseitiger Zustandsspeicherung sowie der Verwendung von Sitzungen und Cookies.
- **Datenzugriff:** ASP.NET bietet verschiedene Datenzugriffstechnologien, darunter das Entity Framework, das eine objektorientierte Schnittstelle für die Interaktion mit Datenbanken bereitstellt, sowie ADO.NET für die direkte Datenbankinteraktion.
- **Sicherheit:** ASP.NET bietet eine umfassende Palette von Sicherheitsfunktionen, einschließlich Authentifizierung, Autorisierung, HTTPS-Unterstützung, Cross-Site Request Forgery (CSRF) Schutz und Anti-SQL-Injection-Maßnahmen.
- **Testbarkeit:** ASP.NET wurde mit der Testbarkeit im Hinterkopf entwickelt und unterstützt verschiedene Testframeworks und -methoden, einschließlich Unit-Tests, Integrationstests und UI-Tests.
- **Internationalisierung und Lokalisierung:** ASP.NET bietet Unterstützung für die Internationalisierung und Lokalisierung von Webanwendungen, einschließlich Mehrsprachigkeitsunterstützung, Datums- und Zahlenformatierung sowie Ressourcenverwaltung.

Bei der Entwicklung von ASP.NET-Anwendungen stehen Entwicklern eine Vielzahl von Entwicklungstools zur Verfügung, darunter Visual Studio, das eine umfassende integrierte Entwicklungsumgebung (IDE) für die Entwicklung von ASP.NET-Anwendungen bereitstellt. Visual Studio bietet Funktionen wie Syntaxhervorhebung, Intellisense, Debugging-Tools und leistungsstarke Projektverwaltungsfunktionen.

Darüber hinaus bietet Microsoft eine Vielzahl von Ressourcen und Support für ASP.NET-Entwickler, darunter Dokumentation, Tutorials, Schulungen und eine aktive Entwicklergemeinschaft. Entwickler können auch von einer Vielzahl von Drittanbieterkomponenten und -erweiterungen profitieren, die die Entwicklung von ASP.NET-Anwendungen erleichtern.

Insgesamt bietet ASP.NET eine leistungsstarke und flexible Plattform für die Entwicklung von Webanwendungen, die es Entwicklern ermöglicht, schnell und effizient hochwertige Webanwendungen zu erstellen. Durch seine umfangreichen Funktionen, Entwicklungstools und Unterstützung ist ASP.NET eine beliebte Wahl für die Entwicklung von Unternehmensanwendungen, E-Commerce-Websites, sozialen Netzwerken und vielem mehr.[19] [20]

6.2 Umsetzung vom Broker

6.2.1 Was ist ein Broker?

Ein Smart Home Broker ist ein Server, der verschiedene Smart Home Geräte und -Systeme integriert und ihnen die Kommunikation untereinander ermöglicht. Der Broker fungiert als Vermittler zwischen den verschiedenen Geräten und ermöglicht es ihnen, miteinander zu interagieren, Informationen auszutauschen und Befehle zu empfangen und zu senden.

Smart-Home-Broker werden typischerweise eingesetzt, um eine einheitliche Benutzeroberfläche bereitzustellen, über die Benutzer ihre Geräte steuern können. Verschiedene Smart-Home-Geräte können unabhängig vom Hersteller oder Protokoll gesteuert werden. Dadurch wird die Benutzerfreundlichkeit erhöht und die Kompatibilität zwischen den Geräten verbessert.

Ein Smart-Home-Broker kann auch Sicherheits- und Datenschutzfunktionen bieten, um sicherzustellen, dass die zwischen den Geräten ausgetauschten Daten geschützt sind. Es ist wichtig, dass ausgetauschte Daten geschützt sind und nicht von unbefugten Dritten abgefangen oder manipuliert werden können.

Ein Smart Home Broker spielt eine entscheidende Rolle bei der Integration und Koordination verschiedener Smart Home Geräte und Systeme. Seine Hauptaufgabe besteht darin, als Vermittler zu fungieren, der die Interoperabilität zwischen den Geräten ermöglicht, unabhängig von ihrem Hersteller oder den verwendeten Kommunikationsprotokollen.

Der Broker erleichtert in erster Linie die nahtlose Kommunikation zwischen verschiedenen Smart Home Geräten. Durch die Bereitstellung einer gemeinsamen Plattform oder eines Kommunikationskanals können diese Geräte miteinander interagieren, Daten austauschen und Anweisungen empfangen und senden. Auf diese Weise können Benutzer ihre Smart Home Systeme ganzheitlich steuern.

Ein Smart Home Broker kann auch Sicherheits- und Datenschutzfunktionen bereitstellen, da Smart Home Systeme sensible persönliche und häusliche Daten verarbeiten. Es ist von entscheidender Bedeutung, dass diese Informationen angemessen geschützt werden. Der Broker implementiert Verschlüsselungstechnologien und Sicherheitsprotokolle, um sicherzustellen, dass die Kommunikation zwischen den Geräten sicher ist und nicht von Dritten abgefangen oder manipuliert werden kann.

Ein weiterer Aspekt der Aufgaben eines Smart Home Brokers ist die Unterstützung von Automatisierung und Szenariensteuerung. Der Broker richtet Regeln und Programme ein, damit Benutzer komplexe Abläufe und Szenarien definieren können, die auf bestimmte Bedingungen reagieren. Dadurch können Smart Home Systeme intelligenter und autonomer werden, was den Komfort und die Effizienz für die Benutzer verbessert.

Der Smart Home Broker ist entscheidend für die Entwicklung eines vernetzten und ganzheitlichen Smart Home Ökosystems. Er bietet Interoperabilität, zentrale Steuerung, Sicherheit und Automatisierung, um die Benutzererfahrung zu verbessern und das Potenzial der Smart Home Technologien zu maximieren.

6.2.2 Szenen

6.2.2.1 Was sind Smart Home Szenen

Smart Home Szenen sind eine Funktion, die dabei hilft, die Automatisierung eines intelligenten Zuhauses entsprechend individuellen Vorlieben einzurichten. Sie ermöglichen es, spezifische Abläufe und Routinen zu definieren. Zum Beispiel könnten morgens zu bestimmten Zeiten die Jalousien geöffnet und die Kaffeemaschine eingeschaltet werden. Oder es könnte jeden Abend die Raumtemperatur im Schlafzimmer auf 16 Grad senken und das Licht allmählich dimmen.

6.2.2.2 Was können Smart Home Szenen

Der Hauptzweck von Smart Home Szenen besteht darin, die Technologie so einzurichten, dass sie automatisch funktioniert und dadurch Zeit spart sowie den Wohnkomfort erhöht. Durch die Integration spezifischer Logiken und Routinen in das Smart Home sind kaum noch manuelle Eingriffe erforderlich. Stattdessen übernimmt das Zuhause automatisch die anfallenden Aufgaben. Die Szenen können jederzeit problemlos angepasst werden. Hier sind einige mögliche Anwendungsfälle: Energie sparen kann durch das Einstellen der Heizung auf die gewünschte Temperatur je nach Tageszeit und das Absenken der Temperatur in Abwesenheit erreicht werden. Kurz vor der Rückkehr sollte die Temperatur wieder erhöht werden. Um den Tag angenehm zu beginnen, kann eine Szene erstellt werden, die das Licht allmählich erhellt, sanfte Musik abspielt und möglicherweise motivierende Nachrichten oder Zitate auf dem Smart Display anzeigt. Dazu gehört beispielsweise das Dimmen des Lichts im Wohnzimmer, das Einschalten des Nachtlights im Flur und das Vorbereiten des Schlafzimmers. Automatisierte Abläufe für den Abend können dazu beitragen, eine entspannte Atmosphäre zu schaffen.

6.2.2.3 Wie werden Szenen erstellt

Die Programmierung von Szenen erfolgt unkompliziert über die Smart Home Web-App. Um eine Szene zu erstellen, müssen mindestens eine Bedingung (z.B. 'Wenn es in der Küche 18 Grad hat'), ein eindeutiger Name und eine Aktion (z.B. 'auf 20 Grad erwärmen') angegeben werden, die ausgeführt werden soll, wenn die Bedingung eintritt. Nach dem Erstellen einer Szene durch den Benutzer erhält der Broker eine Nachricht von der Web-App mit einer Liste von Bedingungen und einer Liste... von Aktionen und den Namen der Szene. Die Szene wird zunächst in der Datenbank gespeichert. Mit jeder ankommenden Nachricht von den Geräten wird geprüft, ob die Bedingung eingetroffen ist, wenn ja, werden die Aktionen ausgeführt, ansonsten läuft das Smart Home System normal weiter.

6.2.2.4 Wie lassen sich Szenen steuern oder anpassen

Um programmierte Szenen zu ändern, das Licht spontan ein- oder auszuschalten oder nicht mehr benötigte Szenen zu löschen, kann auch die Smart Home Web-App genutzt werden. Dort können Änderungen einfach vorgenommen werden und es besteht die Möglichkeit, neue Geräte und Aktionen hinzuzufügen. [21]

6.2.3 SlothServer

Die Methode `HandleApplicationMessageReceivedAsync` ist eine asynchrone Methode, die eingehende MQTT-Anwendungsnachrichten verarbeitet.

Zunächst wird die Methode `ConsoleLogMessageInformation` der `SlothLogger`-Klasse aufgerufen, um die Informationen über die empfangene Nachricht in der Konsole zu protokollieren. Anschließend wird ein neuer Task gestartet, um die Verarbeitung der Nachricht in einem separaten Thread auszuführen, damit die Hauptausführung nicht blockiert wird.

Verschiedene Informationen aus der empfangenen Nachricht werden innerhalb des neuen Tasks extrahiert. Dazu gehören das Nachrichtenthema, der Payload, die Client-ID des sendenden Clients und die ID des Bestimmungsorts der Nachricht. Dazu gehören das Nachrichtenthema, der Payload, die Client-ID des sendenden Clients und die ID des Bestimmungsorts der Nachricht. Dazu gehören das Nachrichtenthema, der Payload, die Client-ID des sendenden Clients und die ID des Bestimmungsorts der Nachricht. Es wird überprüft, ob die Nachricht an den 'Broker' gerichtet ist. Wenn dies der Fall ist, wird die Nachricht weiterverarbeitet, ansonsten wird sie ignoriert.

Wenn die Nachricht an den Broker gerichtet ist, wird überprüft, ob das Nachrichtenthema 'Status/Info', 'Status/Warning' oder 'Status/Error' lautet. Falls ja, wird die Statusnachricht erstellt, protokolliert, in die Datenbank eingefügt und an den Next.js-Client gesendet.

Wenn das Nachrichtenthema nicht auf einen Status hinweist, wird die empfangene CSV-Nachricht in ein Message-Objekt umgewandelt und protokolliert. Anschließend wird diese Nachricht an den... Die Statistik wird an den Next.js-Client gesendet und

anschließend wird überprüft, ob die empfangene Nachricht die Bedingungen für eine Szene erfüllt. Dazu werden die Szenen und Bedingungen durchlaufen. Wenn eine Bedingung erfüllt ist, wird die entsprechende Szene ausgeführt und protokolliert.

Schließlich wird die Methode beendet, ohne auf das Ergebnis des gestarteten Tasks zu warten. Dies liegt daran, dass der Task im Hintergrund läuft und die Hauptausführung nicht blockieren soll.

6.2.4 HTTP-POST

Die HTTP-POST-Methode ermöglicht das Senden von Daten von einer Webanwendung an einen Webserver. Wenn ein Formular ausgefüllt und auf 'Senden' geklickt wird, verwendet der Browser normalerweise die POST-Methode, um die eingegebenen Daten an den Server zu übermitteln. Die Anfrage wird mit der POST-Methode erstellt und die zu sendenden Daten werden im Nachrichtenrumpf der Anfrage eingefügt. Diese Daten können Formulardaten, JSON-Daten oder andere strukturierte Daten sein. Die POST-Anfrage wird an die URL des Servers gesendet, der die Anfrage verarbeiten soll. Der Server empfängt die POST-Anfrage und extrahiert die Daten aus dem Nachrichtenrumpf. Anschließend werden die Daten entsprechend dem Zweck der Anfrage verarbeitet. Nach der Verarbeitung sendet der Server normalerweise eine Antwort an den Client zurück. Die Form der Antwort hängt von der Implementierung der Serverseite und dem Inhalt der Anfrage ab. Die POST-Methode ist eine grundlegende Methode des HTTP-Protokolls und ermöglicht die Übertragung von Daten von einer Webanwendung an einen Server. Sie wird in vielen verschiedenen Szenarien verwendet, um Informationen zwischen einer Webanwendung und einem Server auszutauschen.[22]

6.2.4.1 Beispiel

```
1      public class RunSceneController : ControllerBase
2      {
3          SceneManager _sceneManager = Factory.CreateSceneManager();
4          SlothLogger _logger = Factory.CreateSlothLogger();
5          DatabaseHelper _database;
```

```
6
7      [HttpPost]
8      public ActionResult<string> Post(string nameOfScene)
9      {
10          _database = Factory.CreateDatabaseHelper();
11
12          if (nameOfScene != null && nameOfScene != "")
13          {
14              _sceneManager.RunScene(nameOfScene);
15              return "Ok";
16          }
17          return "Invalid";
18      }
19  }
```

Listing 6.1: Führt eine Szene aus

Die vorliegende Methode ist ein Controller in einer ASP.NET Web API-Anwendung, der für den Endpunkt verantwortlich ist, auf den HTTP-POST-Anfragen gesendet werden. Es wird erwartet, dass ein Parameter mit dem Namen 'nameOfScene' übergeben wird, welcher den Namen einer Szene darstellt. Der Controller, welcher den Namen 'RunSceneController' trägt, initialisiert ein SceneManager-Objekt und ein SlothLogger-Objekt über eine Factory-Methode. Außerdem wird ein DatabaseHelper-Objekt erstellt. In der Post-Methode wird überprüft, ob der übergebene Szenenname null oder leer ist. Wenn dies der Fall ist, wird die Methode RunScene des SceneManager-Objekts aufgerufen, um die Szene auszuführen, und es wird 'Ok' zurückgegeben. Andernfalls wird 'Invalid' zurückgegeben.

6.2.5 HTTP-GET

Die HTTP GET-Methode wird genutzt, um Daten von einem Webserver abzurufen. Sie ist eine der grundlegenden Methoden im Hypertext Transfer Protocol (HTTP), welches zur Übertragung von Daten über das Internet verwendet wird.

Die GET-Methode wird durch das Senden einer Anfrage an einen bestimmten URI (Uniform Resource Identifier) ausgelöst, welcher eine Ressource auf dem Server identifiziert. Die Anfrage kann zusätzliche Parameter in der URL enthalten, um spezifische Daten anzufordern. Diese Parameter werden üblicherweise als Teil der Query-String-Komponente der URL angegeben.

Die GET-Anforderung ist idempotent, was bedeutet, dass mehrere aufeinanderfolgende GET-Anfragen den gleichen Serverzustand nicht ändern sollten. Daher sollte sie verwendet werden, um Daten abzurufen, ohne den Serverzustand zu ändern.

Ein typisches Beispiel für eine GET-Anforderung ist das Abrufen von Informationen von einer bestimmten Ressource auf einem Webserver. Zum Beispiel können Benutzerdaten von einer API, Produktinformationen von einer E-Commerce-Website oder Suchergebnisse von einer Suchmaschine abgerufen werden.

Die GET-Methode ist eine grundlegende und häufig verwendete Methode, um Daten von einem Webserver abzurufen und gehört zu den wichtigsten Funktionen im HTTP-Protokoll.[22]

6.2.5.1 Beispiel

```
1    /// <summary>
2    /// Returns the history of a specific GUID to the WebApp
3    /// </summary>
4    [Route("api/history")]
5    [ApiController]
6    public class HistoryController : ControllerBase
7    {
8        [HttpGet]
9        public ActionResult<String> Get(string guid)
10       {
11           DatabaseHelper databaseHelper = Factory.
12               CreateDatabaseHelper();
```

```
13         List<object> lastValues = databaseHelper.  
            GetMessageObjectDataForCertainGUIDFromDatabase  
            (guid);  
14         SlothLogger.ConsoleLogMessageInformation($"Last Values for  
            {guid} sent");  
15         return JsonConvert.SerializeObject(lastValues);  
16     }  
17 }
```

Listing 6.2: Liefert die History von einem Sensor

Die vorliegende Methode ist eine HTTP-GET-Methode in einer ASP.NET Web API-Anwendung. Sie erwartet einen Parameter 'guid' im URL-Routing und nutzt diesen, um spezifische Daten aus einer Datenbank abzurufen. Zunächst wird eine Instanz der Klasse 'DatabaseHelper' erstellt, um auf die Datenbank zuzugreifen. Anschließend wird eine Methode aufgerufen, um Daten basierend auf dem übergebenen 'guid' abzurufen. Schließlich wird diese Aktion protokolliert. Die abgerufenen Daten werden anschließend in das JSON-Format serialisiert und als Antwort zurückgegeben.

6.3 Textdateien

6.3.1 Deadlocks beim Speichern von Informationen in Dateien

Deadlocks stellen eine gängige Herausforderung in der Informatik dar, die entstehen kann, wenn mehrere Prozesse um Ressourcen konkurrieren und dabei auf die Freigabe anderer Ressourcen warten. Dieses Problem kann sich auch beim Speichern von Informationen in Dateien zeigen, insbesondere in Umgebungen mit mehreren Benutzern, in denen parallele Zugriffe auf dieselben Dateien erfolgen. [23]

6.3.2 Was ist ein Deadlock?

Ein Deadlock tritt auf, wenn zwei oder mehr Prozesse auf Ressourcen warten, die von anderen Prozessen gehalten werden, während sie selbst Ressourcen halten, auf die andere Prozesse warten. Dadurch kommt es zu einer Blockade, da keiner der Prozesse die benötigten Ressourcen freigeben kann, um die anderen Prozesse voranzutreiben. [23]

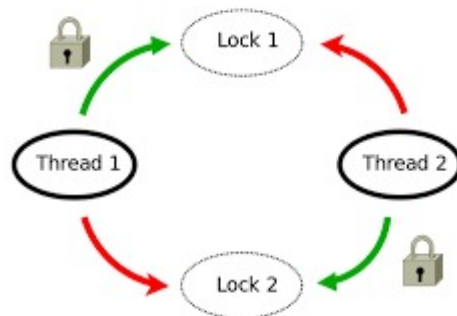


Abbildung 6.2: Ein Beispiel zu Deadlocks

6.3.3 Beispiel

Angenommen, Prozess 1 hält Datei X (Lock 1) geöffnet und versucht auf Datei Y (Lock 2) zuzugreifen, während Prozess 2 Datei Y (Lock 2) geöffnet hat und auf Datei X (Lock 1) zugreifen möchte. Wenn beide Prozesse gleichzeitig versuchen, auf die jeweils von ihnen gehaltenen Dateien zuzugreifen, können sie in einem Deadlock stecken bleiben. Prozess 1 kann nicht auf Datei Y zugreifen, weil Prozess 2 sie hält, und Prozess 2 kann nicht auf Datei X zugreifen, weil Prozess 1 sie hält.

6.3.4 Warum Textdateien anstatt einer Datenbank?

Am Anfang vom Projekt war uns nicht bewusst, welche Daten wir alles speichern wollen und wie stark sich diese Daten im Laufe der Zeit vergrößern würden. Zu Beginn erschien es uns klar, dass die Verwendung einer Datenbank eine unnötige Zeitverschwendung wäre. Daher haben wir uns zunächst für die Speicherung unserer Daten in Textdateien entschieden. Zunächst haben wir lediglich die eingehenden Nachrichten gespeichert, die an uns gesendet wurden. Doch im Verlauf des Projekts begannen wir, weitere Daten zu erfassen, wie beispielsweise die Informationen über die verbundenen Clients und andere relevante Metriken. Mit der Zeit stiegen die Anforderungen an unsere Datenverwaltung und wir fanden uns am Ende mit fünf verschiedenen Textdateien wieder, die jeweils unterschiedliche Datensätze enthielten. Diese fragmentierte Datenspeicherung führte jedoch schnell zu einem Problem: dem Verlust des Überblicks. Es wurde zunehmend schwieriger, die relevanten Informationen zu finden und Zusammenhänge zwischen den verschiedenen Dateien herzustellen. Die Textdateien erwiesen sich als unpraktisch für die Skalierung und Strukturierung unserer Daten, und wir erkannten, dass eine effizientere Lösung erforderlich war. Dies war der Wendepunkt, der uns dazu brachte, über eine Datenbanklösung nachzudenken. Wir erkannten, dass eine Datenbank nicht nur eine bessere Strukturierung und Organisation unserer Daten ermöglichen würde, sondern auch eine flexiblere und leistungsfähigere Verwaltung ermöglichen könnte. Angesichts

der wachsenden Komplexität unseres Projekts und der stetig zunehmenden Datenmengen wurde klar, dass die Migration zu einer Datenbank eine sinnvolle Entscheidung war, um unsere Effizienz zu steigern und den Überblick über unsere Daten zu behalten.

6.3.5 Aufbau der Methoden

```
1    /// <summary>
2    /// Removes the Scene from the File and rewrites the File
3    /// </summary>
4    /// <param name="scene"></param>
5    internal void RemoveSceneFromFile(Scene scene)
6    {
7        List<Scene> list = GetScenesFromFile();
8        List<Scene> list = _databaseHelper.GetScenesFromDatabase();
9
10       foreach (Scene item in list.ToList())
11       {
12           if (item.Name == scene.Name)
13           {
14               list.Remove(item);
15           }
16       }
17       using (FileStream fs = File.Open(FilePaths.PathOfScenesFile,
18           FileMode.OpenOrCreate, FileAccess.ReadWrite))
19       {
20           lock (fs)
21           {
22               fs.SetLength(0);
23               fs.Close();
24
25               foreach (var item in list)
26               {
27                   _databaseHelper.AddDataTo_Scenes(item.Name, item.
28                       Raw);
29               }
30
31               WriteScenesToFile(list);
32           }
33       }
```

```
30         }
31     }
32 }
```

Listing 6.3: Löschen von Scenes

```
1  /// <summary>
2  /// Saves the incoming status message into: /home/pi/SlothLogs/
   slothLogs.txt
3  /// </summary>
4  /// <param name="statusMessage"></param>
5  public void SaveStatusMessageToFile(StatusMessage statusMessage)
6  {
7      lock (fileWritelock)
8      {
9          ShortenFile(FilePaths.PathOfLogFile);
10
11         List<string> fileLines = new List<string>(File.ReadAllLines(
            FilePaths.PathOfLogFile));
12
13         fileLines.Insert(0, $"{DateTime.Now}: {statusMessage.ToCSV()
            }");
14
15         File.WriteAllLines(FilePaths.PathOfLogFile, fileLines);
16     }
17 }
```

Listing 6.4: Speichern von StatusMessages

Diese Codeausschnitte zeigen verschiedene Methoden, um Daten in verschiedene Dateien zu speichern oder aus ihnen zu lesen. Sie verwenden Dateioperationen wie das Lesen von Zeilen aus einer Datei, das Schreiben von Zeilen in eine Datei, das Hinzufügen von Zeilen zu einer Datei und das Entfernen von Zeilen aus einer Datei. Alle diese Methoden verwenden locks, um sicherzustellen, dass nur ein Thread auf eine Datei Zugriff hat und Datenintegrität gewährleistet ist. Einige der Methoden verarbeiten auch spezifische Datenstrukturen wie Nachrichten oder Szenen und führen entsprechende Operationen wie das Konvertieren dieser Daten in ein geeignetes Format vor dem Schreiben in die Datei durch. Zusätzlich gibt es eine Methode, die dazu dient, eine Datei zu verkürzen, wenn sie eine maximale Anzahl von Zeilen

überschreitet. Dies ist nützlich, um zu verhindern, dass eine Datei unendlich groß wird und möglicherweise zu Leistungsproblemen führt oder zu viel Speicherplatz belegt. Um die Datei zu verkürzen, wird eine temporäre Datei erstellt, in die die erste Hälfte der Zeilen aus der ursprünglichen Datei kopiert wird, während die andere Hälfte verworfen wird. Genau wie die anderen Methoden benutzt auch die wieder.

6.4 Datenbank

6.4.1 Entity-Relationship-Diagramm

Entity-Relationship-Diagramm Ein ERD (Entity-Relationship-Diagramm) ist eine grafische Darstellungsmethode, die häufig in der Datenmodellierung verwendet wird. Es zeigt die Beziehungen zwischen verschiedenen Entitäten (oder Objekten) in einem System auf. Diese Diagramme können bei der Visualisierung und Klärung der Struktur von Daten und der Art und Weise, wie sie miteinander verbunden sind, hilfreich sein. Ein ERD (Entity-Relationship-Diagramm) bietet eine Vielzahl von Vorteilen, darunter:

- Es hilft bei der Identifikation von Redundanzen und inkonsistenten Daten, indem es die Beziehungen zwischen verschiedenen Entitäten darstellt und es einfach macht, diese zu überprüfen.
- Man kann bei der Analyse und dem Design von Geschäftsprozessen hilfreich sein, indem es die Beziehungen zwischen verschiedenen Aktivitäten und Daten im System darstellt.
- Es ermöglicht es, die Funktionsweise eines Systems zu beschreiben und zu dokumentieren, indem es die Beziehungen zwischen verschiedenen Entitäten darstellt und zeigt, wie sie miteinander interagieren.

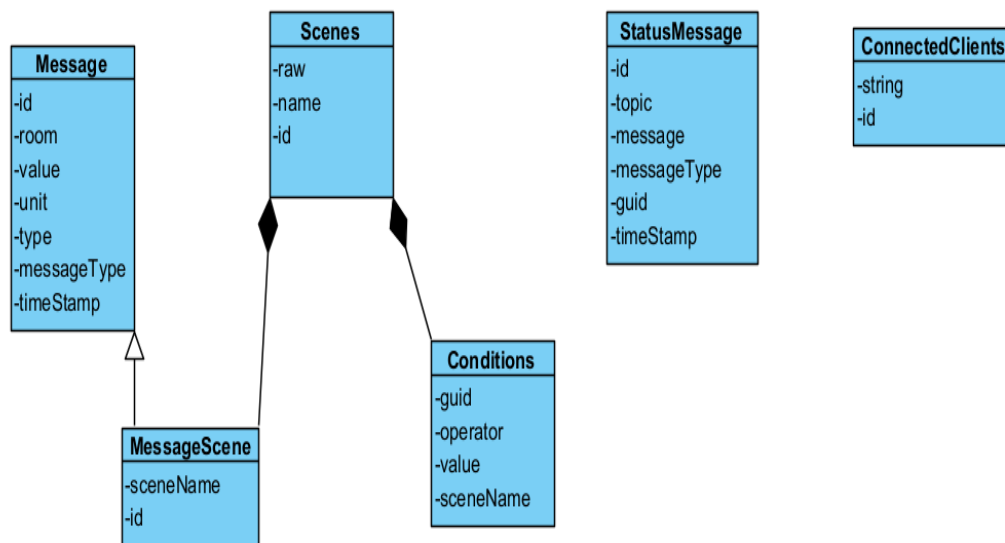


Abbildung 6.3: UML Diagramm zur Veranschaulichung der Datenbank

6.4.2 Warum eigentlich eine SQLite Datenbank?

Erstens sollten wir uns die Frage stellen, warum wir eine Datenbank verwenden, obwohl wir unsere Nachrichten und andere Daten in Textdateien speichern. Das Speichern von neuen Nachrichten in einer Datenbank bietet nicht nur eine höhere Leistung, sondern ist sogar verständlicher und nachvollziehbarer im Vergleich zu Textdateien. Dies ist besonders wichtig für SLOTech, da wir eine einfache Erweiterbarkeit für technikaffine Benutzer anstreben. Der Grund, warum wir uns für eine SQLite-Datenbank entschieden haben, liegt darin, dass unser Broker asynchron läuft und wir eine Thread-sichere Lösung benötigen. SQLite bietet eine solide Thread-Sicherheit, was für unsere Anforderungen passend ist. Zusätzlich dazu ist SQLite leichtgewichtig, was bedeutet, dass keine separate Serverinstallation erforderlich ist. Die gesamte Datenbank ist in einer einzigen Datei gespeichert, was sie einfach zu transportieren und zu verwalten macht. Dies ist besonders praktisch für unsere Benutzer, da sie die gesamte Datenbank einfach sichern, verschieben oder kopieren können, ohne sich um komplexe Serverkonfigurationen kümmern zu müssen.

6.4.3 Initialisation der Datenbank

Die InitializeDatabase-Methode wird verwendet, um sicherzustellen, dass die benötigte Datenbank und ihre Tabellen erstellt werden. Die Methode überprüft zunächst, ob die Datenbankdatei vorhanden ist. Falls die Datenbankdatei nicht existiert, werden die Tabellen für die Klassen Messages, Scenes, MessagesScenes, Conditions, StatusMessages und ConnectedClients erstellt. Dabei werden die entsprechenden SQL-Anweisungen verwendet.

Die Verwendung von parameterisierten Abfragen hilft dabei, SQL-Injektionen zu verhindern und die Sicherheit der Anwendung zu erhöhen. Die Protokollierung mit SlothLogger ermöglicht das Überwachen von Datenbankoperationen und dient der Fehlersuche. Es ist wichtig sicherzustellen, dass die Verbindungen zur Datenbank ordnungsgemäß geöffnet und geschlossen werden, um Ressourcenlecks zu vermeiden.

6.4.4 CRUD - Create, Read, Update, Delete

CRUD steht für die vier grundlegenden Operationen, die auf Daten in einer Datenbank ausgeführt werden können: Create (Erstellen), Read (Lesen), Update (Aktualisieren) und Delete (Löschen). Jede dieser Operationen hat eine bestimmte Funktion und ermöglicht es Anwendern, Daten in einer Datenbank effektiv zu verwalten und bearbeiten.

- Create (Erstellen) bezieht sich auf das Hinzufügen von Daten in eine Tabelle. Mit einer Create-Operation können neue Datensätze in einer Tabelle erstellt werden, um das Datenbestand zu erweitern.
- Read (Lesen) bezieht sich auf das Abrufen von Daten aus einer Tabelle. Mit einer Read-Operation kann man bestehende Daten aus einer Tabelle abfragen, um Informationen zu erhalten oder zu überprüfen.
- Update (Aktualisieren) bezieht sich auf das Ändern von Daten in einer Tabelle. Mit einer Update-Operation kann man bestehende Daten in einer Tabelle ändern, um sicherzustellen, dass die Daten aktuell und korrekt sind.

- Delete (Löschen) bezieht sich auf das Entfernen von Daten aus einer Tabelle. Mit einer Delete-Operation kann man unerwünschte oder veraltete Daten aus einer Tabelle entfernen, um den Datenbestand zu verkleinern.[24]

6.4.5 Create (Datensätze einfügen)

```
1  /// <summary>
2  /// Adds an entry into Message
3  /// </summary>
4  /// <param name="identifier"></param>
5  /// <param name="room"></param>
6  /// <param name="value"></param>
7  /// <param name="unit"></param>
8  /// <param name="type"></param>
9  public void AddDataTo_Message(string identifier, string room, int value
    , string unit, string type, string messageType, DateTime? timeStamp)
10 {
11     using (SqlConnection connection = new SqlConnection(
        connectionString))
12     {
13         connection.Open();
14
15         using (SqlCommand command = new SqlCommand())
16         {
17             command.Connection = connection;
18             command.CommandText = @"INSERT INTO Messages (identifier,
                room, value, unit, type, timeStamp, messageType)
19                 VALUES (@identifier, @room, @value,
                @unit, @type, @timeStamp, @messageType);";
20
21             command.Parameters.AddWithValue("@identifier", identifier);
22             command.Parameters.AddWithValue("@room", room);
23             command.Parameters.AddWithValue("@value", value);
24             command.Parameters.AddWithValue("@unit", unit);
25             command.Parameters.AddWithValue("@type", type);
26             if (timeStamp == null)
27             {
28                 timeStamp = DateTime.MinValue;
```



```
29         }
30         command.Parameters.AddWithValue("@timeStamp", timeStamp);
31         command.Parameters.AddWithValue("@messageType", messageType);
32         );
33         // Execute the command
34         command.ExecuteNonQuery();
35         command.Parameters.Clear();
36         SlothLogger.ConsoleLogDatabaseAdd("Data added to Message");
37     }
38 }
39 }
```

Listing 6.5: AddDataToMessages

Beim Erstellen einer Datenbank ist wo möglich Create das erste, was zum Einsatz kommt. Wie auch schon vorher erwähnt kann man Datensätze in Tabellen mithilfe von INSERT INTO einfügen.

```
1     string createMessagesTable = @"
2         CREATE TABLE IF NOT EXISTS Messages (
3             id INTEGER PRIMARY KEY AUTOINCREMENT,
4             identifier VARCHAR(255),
5             room VARCHAR(255),
6             value INTEGER,
7             unit VARCHAR(50),
8             type VARCHAR(50),
9             timeStamp DATETIME,
10            messageType VARCHAR(255)
11        );";
```

Listing 6.6: Erstellt eine Message Tabelle

In diesem Beispiel werden die Spalten gezielt angegeben: Messages (Spaltenname1, Spaltenname2, ...), da die anderen Spalten in dieser Tabelle automatisch einen Wert kriegen. Wie man am Bild sieht, können die Werte auch Parameter sein. Diese werden verwendet, wenn die Daten nicht von der Datenbank bestimmt werden, sondern extern wie in unserem Fall von den Sensoren kommen. Auf der Website gibt es die

Möglichkeit, einen neuen Sensor hinzuzufügen. Es wird eine GUID, ein Raum, ein Wert, eine Einheit und ein Sensor-Typ angegeben. Wenn der Broker also eine neue Nachricht erhält wird die `AddDataTo_Messages` aufgerufen und ein neuer Datensatz wird in die Tabelle `Messages` hinzugefügt.

6.4.6 READ (Datensätze ausgeben)

Diese Funktion wird in unserer Arbeit häufig beim Darstellen der Messdaten auf der Website verwendet. Da Sloth mehrere Tabellen hat wurden mehrere Abfragen erstellt, um die Messdaten einfach und leserlich auf der Website anzeigen zu können:

```
1  /// <summary>
2  /// Returns a list of the current messages in the database
3  /// </summary>
4  /// <returns></returns>
5  public List<Message> GetMessagesFromDatabase()
6  {
7      List<Message> messages = new List<Message>();
8
9      using (SqlConnection connection = new SqlConnection(
10         connectionString))
11      {
12         connection.Open();
13
14         using (SqlCommand command = new SqlCommand())
15         {
16             command.Connection = connection;
17             command.CommandText = @"SELECT * FROM Messages";
18
19             using (SqlDataReader reader = command.ExecuteReader())
20             {
21                 while (reader.Read())
22                 {
23                     string identifier = reader.GetString(reader.
24                         GetOrdinal("identifier"));
25                     string room = reader.GetString(reader.GetOrdinal("
26                         room"));
```

```
24         int value = reader.GetInt32(reader.GetOrdinal("
           value"));
25         string unit = reader.GetString(reader.GetOrdinal("
           unit"));
26         string type = reader.GetString(reader.GetOrdinal("
           type"));
27         string messageType = reader.GetString(reader.
           GetOrdinal("messageType"));
28
29         messages.Add(Factory.CreateMessage(identifier, room
           , value, unit, type, messageType, null));
30     }
31 }
32 SlothLogger.ConsoleLogDatabaseAdd("Returned Messages from
           Database");
33 }
34 }
35 return messages;
36 }
```

Listing 6.7: GetMessageFromDatabase liefert alle Nachrichten von der Datenbank

Die Prozedur zeigt alle Antworten der Tabelle MessagesScenes für eine bestimmte Szene an. Dies erkennt man an der Bedingung WHERE ms.sceneName = @sceneName. Das wichtige beim SELECT ist es nach jeder Antwort die in der Tabelle gespeichert zu suchen und dann auszugeben. Dies wird mit einer Schleife realisiert, wo die id mit jedem Durchlauf um eins erhöht, wird sodass jede Antwort in die Liste messages gespeichert wird und am Ende zurückgegeben wird.

6.4.7 UPDATE (Datensätze aktualisieren)

UPDATE wird häufig dazu verwendet, um vorhandene Datensätze in einer Tabelle zu überarbeiten bzw. editieren. Dazu benötigt man ein UPDATE und ein SET.

6.4.8 DELETE (Datensätze löschen)

Mit Delete kann man Daten aus einer Tabelle entfernen. Es ist wichtig zu beachten, dass das Löschen von Datensätzen irreversible Folgen hat. Daher sollte man die Delete-Anweisung mit Vorsicht verwenden.

```
1 public void DeleteDataFrom_Message(string identifier, string room, int value, string
   unit, string type, string messageType)
2 {
3     using (SqliteConnection connection = new SqliteConnection(connectionString))
4     {
5         connection.Open();
6
7         using (SqliteCommand command = new SqliteCommand())
8         {
9             command.Connection = connection;
10            command.CommandText = $"DELETE FROM Messages
11                                   WHERE identifier = @identifier AND room = @room AND value =
                                   @value AND unit = @unit AND type = @type AND messageType = @messageType";
12
13            command.Parameters.AddWithValue("@identifier", identifier);
14            command.Parameters.AddWithValue("@room", room);
15            command.Parameters.AddWithValue("@value", value);
16            command.Parameters.AddWithValue("@unit", unit);
17            command.Parameters.AddWithValue("@type", type);
18            command.Parameters.AddWithValue("@messageType", messageType);
19
20            command.ExecuteNonQuery();
21
22            command.Parameters.Clear();
23
24            SlothLogger.ConsoleLogDatabaseDelete("Data deleted from Message");
25        }
26    }
27 }
```

Listing 6.8: DeleteDataFromScenes löscht Szenen aus der Datenbank

Die Methode `DeleteDataFrom_Scenes` hat einen Parameter `name`, der genau eine Szene in der Tabelle referenziert. Mit einer `WHERE`-Bedingung wird nach dem Namen gefiltert und der Datensatz wird gelöscht. Anschließend da es die Szenen nicht mehr gibt müssen auch die Bedingungen und Antworten von der jeweiligen Szene gelöscht werden.

6.4.9 Allgemein zu der Syntax von der Datenbank

```
1 public void Test(string test)
2 {
3     using (SqliteConnection connection = new SqliteConnection(connectionString))
4     {
5         connection.Open();
6
7         using (SqliteCommand command = new SqliteCommand())
8         {
9             command.Connection = connection;
10            command.CommandText = @"INSERT INTO Messages (test)
11                                   VALUES (@test);";
12
13            command.Parameters.AddWithValue("@test", test);
14
15            // Execute the command
16            command.ExecuteNonQuery();
17            command.Parameters.Clear();
18            SlothLogger.ConsoleLogDatabaseAdd("Data added to Message");
19        }
20    }
21 }
```

Listing 6.9: Beispiel SQL-Statement

Um SQL-Statements schreiben zu können benötigen man eine Verbindung zu der Datenbank, als nächstes müssen wir einen neues SQLite Kommando erstellen. Dem SQL-Statement müssen wir dann eine connection hinzufügen, das passiert durch die Eigenschaft `Connection`. Dann befüllen wir die Eigenschaft `CommandText` mit der gewünschten SQL-Anweisung. Wenn zum Beispiel die Tabelle einen neuen Datensatz

bekommen soll müssen wir die Parameter befüllen, dies wird durch die Methode `AddWithValue()` realisiert. Wenn alle Parameter befüllt wurden, muss das Statement ausgeführt werden. Dies geschieht durch die Methode `ExecuteNonQuery()`. Zu guter Letzt müssen wir die Parameter löschen.

6.4.10 Filtermöglichkeiten

Um die Website besser zu gestalten haben wir uns überlegt ein paar Filtermöglichkeiten hinzuzufügen, die der Website helfen soll, mehrere Features zu unterstützen. Dazu haben wir uns paar Möglichkeiten überlegt, wie der User die verschiedenen Sensoren Filtern könnte. Einer dieser Feature ermöglicht es die History des Sensors anzuzeigen. Da das vielleicht in dem Interesse des User ist sich die letzten Messwerte des Sensors anschauen zu wollen, dachten wir uns, dass das eine gute und sinnvolle Filtermöglichkeit ist.

```
1  /// <summary>
2  /// Returns the last Message for each guid
3  /// </summary>
4  /// <returns></returns>
5  public List<object> GetLastMessageObjectDataForAllGUIDsFromDatabase()
6  {
7      List<object> values = new List<object>();
8
9      using (SqliteConnection connection = new SqliteConnection(
10         connectionString))
11     {
12         connection.Open();
13
14         using (SqliteCommand command = new SqliteCommand())
15         {
16             command.Connection = connection;
17             // Corrected SQL query to retrieve the latest message for
18             // each identifier
19             command.CommandText = @"SELECT identifier, room, value,
20                                     unit, type, messageType, timeStamp
21                                     FROM Messages
22                                     WHERE (identifier, timeStamp) IN
```

```
20                                     (SELECT identifier, MAX(timestamp) as
timestamp
21                                     FROM Messages
22                                     GROUP BY identifier);"
23
24     using (SqliteDataReader reader = command.ExecuteReader())
25     {
26         while (reader.Read())
27         {
28             string identifier = reader.GetString(reader.
                GetOrdinal("identifier"));
29             string room = reader.GetString(reader.GetOrdinal("
                room"));
30             int value = reader.GetInt32(reader.GetOrdinal("
                value"));
31             string unit = reader.GetString(reader.GetOrdinal("
                unit"));
32             string type = reader.GetString(reader.GetOrdinal("
                type"));
33             string timeStamp = reader.GetDateTime(reader.
                GetOrdinal("timestamp")).ToString("yyyy-MM-dd_HH
                -mm-ss");
34             string messageType = reader.GetString(reader.
                GetOrdinal("messageType"));
35
36             values.Add(new { Id = identifier, Room = room,
                Value = value, Unit = unit, Type = type,
                Timestamp = timeStamp, MessageType = messageType
                });
37             Console.WriteLine($"All: {identifier};{room};{value
                };{unit};{type};{timeStamp};{messageType}");
38         }
39     }
40 }
41 }
42
43 SlothLogger.ConsoleLogDatabaseAdd("Returned Messages from Database"
    );
44
```

```
45     return values;  
46 }
```

Listing 6.10: Gibt die letzte Nachricht von jedem Sensor

Hierbei wird die letzten Nachrichten von einem Sensor in der Tabelle selektiert und es wird nach dem Sensor gefiltert den der User eingibt. Die eingabe des Users wird als Parameter gespeichert und mit den Nachrichten in der Nachrichten-Tabelle verglichen.

6.4.11 Debuggen

Logs wurden nach jeder erfolgreichen Ausführung der SQL-Statements auf die Konsole geschrieben um den Code nachvollziehbarer und leichter fürs Debuggen zu machen. Anfangs wurden über das Program.cs vorgetäuschte Messwerte in die Datenbank gespeichert. Nach erfolgreichem testen wurden über die schon vorher erwähnte MQTTx App wieder dummy Messwerte verschickt, um einen möglichst realistischen Ablauf der dann später ohne Probleme abgewickelt werden sollte vorzutäuschen.

6.4.12 ADO Mapper

ADO Mapper sind Werkzeuge, die in der Softwareentwicklung verwendet werden, um die Kommunikation zwischen einer Anwendung und einer Datenbank zu erleichtern. Sie dienen dazu, Datenobjekte aus einer Quellstruktur in eine Zielstruktur zu transformieren, wobei sie oft zwischen verschiedenen Datenbankformaten oder -schemas abgleichen. Diese Mapper ermöglichen es Entwicklern, auf einer höheren Abstraktionsebene zu arbeiten, da man sich nicht um die Details der Datenbankkommunikation kümmern muss. Stattdessen konzentrieren sich die Entwickler eher auf die Logik ihrer Anwendung und die Mapper zu verwenden, um die Daten entsprechend zu verarbeiten.

Wenn es darum geht, einen ADO Mapper auszutauschen, geschieht dies in der Regel durch das Ersetzen des aktuellen Mapper-Codes oder -Pakets durch ein neues. Dies erfordert oft eine Anpassung der Konfiguration, um sicherzustellen, dass der neue Mapper ordnungsgemäß mit der Anwendung zusammenarbeitet. In einigen Fällen können auch Tests durchgeführt werden, um sicherzustellen, dass der Austausch keine unerwarteten Auswirkungen auf die Funktionalität der Anwendung hat. Insgesamt bietet der Austausch eines ADO Mappers die Möglichkeit, die Effizienz und Leistungsfähigkeit der Datenbankkommunikation in einer Anwendung zu verbessern.[25]

Webanwendung

Rosenkranz Sebastian Alexander Maximilian

Kapitel 7

Webapplikation

7.1 Einführung in die Webapplikation

In der heutigen Zeit wird immer mehr Technik in Haushalte integriert. Ob es sich um Heizungen, Jalousien, Beleuchtung oder Haushaltsgeräte handelt, diese Geräte werden immer intelligenter und leistungsfähiger. Allerdings sind die meisten Geräte nicht miteinander verbunden und können nicht direkt kommunizieren. Hierfür ist eine Zentrale erforderlich, die alle Geräte miteinander verbindet. Ebenso benötigt man eine Benutzeroberfläche, um diese Zentrale zu steuern.

7.1.1 Die Problemstellung

Hierfür wird eine Steuerzentrale benötigt, die sämtliche Daten sammelt, verarbeitet und die Geräte steuert sowie überwacht. Dieses Smart Home System umfasst verschiedene Geräte, die miteinander vernetzt werden müssen. Zudem ermöglicht sie die Kommunikation untereinander. Um die Steuerzentrale bedienen zu können, ist eine benutzerfreundliche Oberfläche erforderlich, die alle Funktionen des Smart Home Systems zur Verfügung stellt. In anderen Smart Home Systemen wurde dafür oft eine Applikation entwickelt, die nativ auf bestimmten Geräten läuft, wie zum Beispiel

Apple's HomeKit. Dieses läuft auf iPhones, Macintoshes, iPads und anderen Apple-Geräten. Dadurch ist das Smart Home jedoch auf Apple-eigene Geräte beschränkt und kann nicht auf Non-Apple-Produkten, wie zum Beispiel einem Windows-PC, gesteuert werden.

Das Smart Home System soll nicht auf eine bestimmte Anzahl von Geräten beschränkt sein, sondern auf fast allen Geräten verwendet werden können. Aus diesem Grund wurde beschlossen, eine Webapplikation zu entwickeln, da diese auf allen Geräten läuft, die einen Webbrowser besitzen.

7.1.2 Die Zielsetzung

Die Zielsetzung der Webapplikation ist es, eine zentrale Benutzeroberfläche für das Smart Home System bereitzustellen. Im nächsten Absatz werden weitere Ziele wie Events, Live-Refresh, Auto-Discovery, Responsive-Design, eine Notifikationszentrale sowie Sortier- und Filtermechanismen erläutert.

Die Webapplikation soll eine einfache Bedienung und eine Übersicht über alle Geräte bieten. Zusätzlich soll sie die Möglichkeit bieten, die Geräte anzusteuern und zu überwachen. Weitere Ziele sind das Hinzufügen von Events, die auf bestimmte Bedingungen reagieren und dann Aktionen ausführen. Die Web-App soll live aktualisiert werden, wenn sich der Status eines Gerätes ändert. Es soll automatisch erkannt werden, wenn ein neues Gerät im System verfügbar ist, damit es sofort in die Applikation integriert werden kann. Außerdem soll es möglich sein, die Webapplikation auf einem mobilen Gerät ohne Probleme zu nutzen. Des Weiteren soll es möglich sein, sich die Historie der Geräte anzeigen zu lassen, um zu sehen, was in der Vergangenheit passiert ist. Es soll ein Benachrichtigungssystem implementiert werden, um den Benutzer über wichtige Ereignisse zu informieren und auf dem Laufenden zu halten. Außerdem soll ein Sortier- und Filtermechanismus implementiert werden, um die Übersicht über alle Geräte zu verbessern. Es soll auch

möglich sein, ausgewählte externe Smart Home Anbieter zu integrieren, um das Smart Home System zu erweitern. Und schließlich sollte es möglich sein, alle Endgeräte individuell zu konfigurieren, beispielsweise den Intervall, in dem ein Sensor Daten sendet.

7.2 Überblick

Die Webapplikation ist die zentrale Benutzeroberfläche des gesamten Systems. Sie bietet eine Übersicht über alle Endgeräte, die bereits dem Smart Home System hinzugefügt wurden. Die Hauptansicht der Webapplikation ist in Abbildung 7.1 zu sehen.

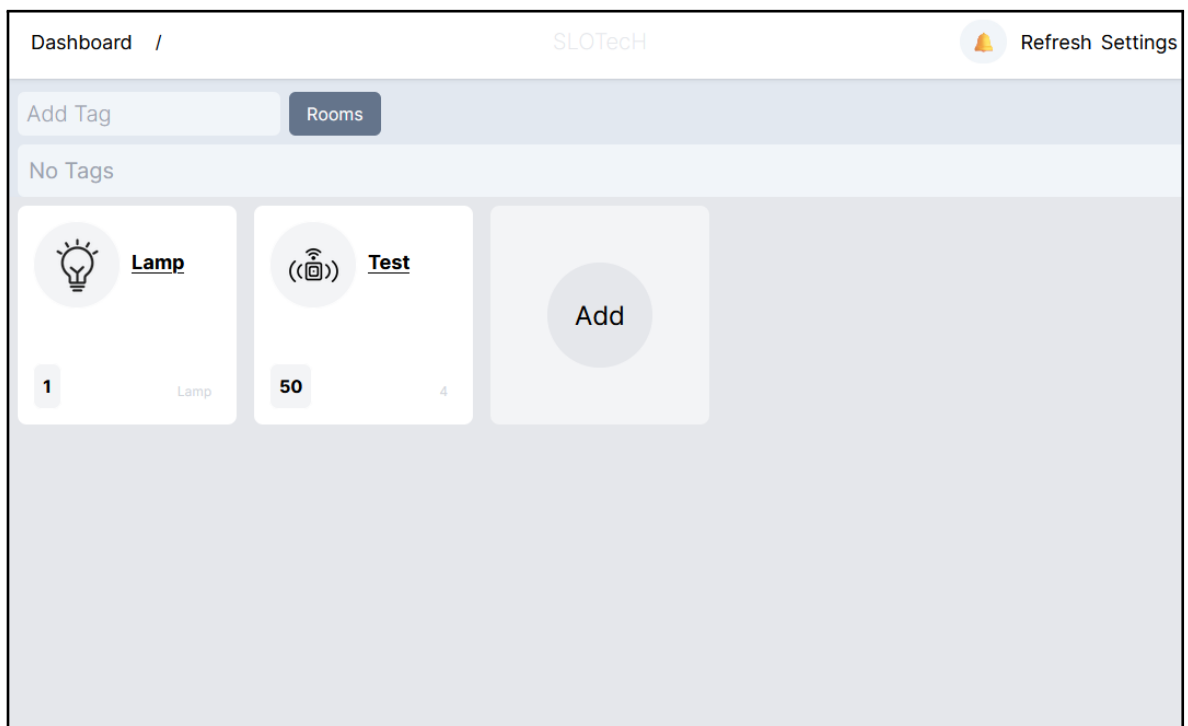


Abbildung 7.1: Hauptansicht der Webapplikation

7.2.1 Grundlagen

Die Webanwendung zur Steuerung des Smart Homes basiert auf den Technologien HTML, CSS und JavaScript. Allerdings wurden zur Entwicklung Frameworks und Bibliotheken verwendet, die auf diesen Technologien aufbauen. Die Anwendung ist in Client- und Server-Seite unterteilt.

7.2.2 Technologien

Der Server-Side-Teil ist in TypeScript geschrieben und nutzt Express.js und Socket.io. Er ist hauptsächlich für die Live-Kommunikation zwischen den Clients und dem Server verantwortlich. Der Client-Side-Teil ist in TypeScript mit den Frameworks Next.js und React.js sowie den Bibliotheken TailwindCSS und Socket.io-Client geschrieben. Dieser Teil ist hauptsächlich für die Darstellung und Interaktion der Benutzeroberfläche sowie für die aktive Kommunikation mit dem Smart Home System zuständig. Beispielsweise kann man damit eine Lampe ein- oder ausschalten oder den Intervall für das Senden eines Sensors ändern.

In Kapitel 7.3 wird genauer auf die Grundlagen und Technologien eingegangen.

7.2.3 Architektur & Design

Die Webapplikation ist so konzipiert, dass sie auf den meisten Geräten einwandfrei funktioniert. Das Design ist einfach und übersichtlich gehalten, um die Bedienung so einfach wie möglich zu gestalten. Die gesamte Anwendung basiert auf dem TreeView-Prinzip, wodurch alle Web-Komponenten in einer Ordnerstruktur aufgebaut sind. Dies verbessert die Übersichtlichkeit und Wartbarkeit der Anwendung und macht sie zudem modular und leicht erweiterbar.

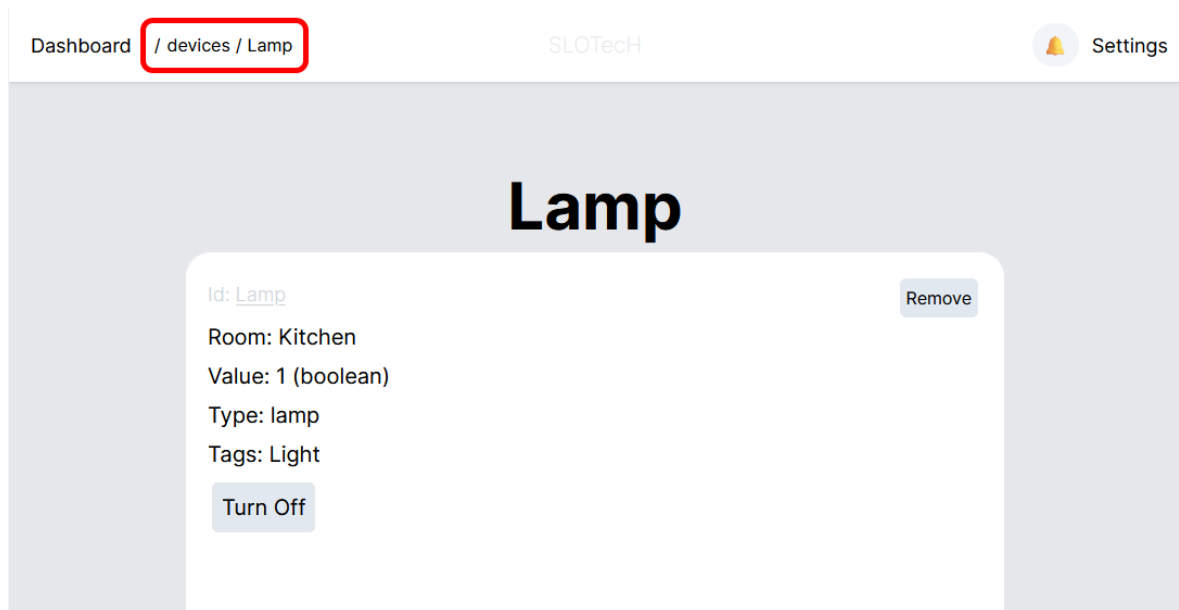


Abbildung 7.2: BreadCrumb Navigation

Es ist leicht nachvollziehbar, wie die Anwendung aufgebaut ist und wie sie funktioniert, da die App grafisch nach demselben Prinzip aufgebaut wurde. Zur Verdeutlichung gibt es in der Anwendung auch eine Breadcrumb Navigation. (Wie in Abbildung 7.2 zu sehen ist)

/ devices / Lamp

Abbildung 7.3: BreadCrumb Navigation (Zoom)

Abbildung 7.3 zeigt die vergrößerte Breadcrumb Navigation, welche die aktuelle Position in der Anwendung anzeigt und es ermöglicht, zu vorherigen Positionen zurückzukehren.

Der Server ist nur für die Live-Kommunikation zuständig. Die Anwendung wurde so konzipiert, dass sie größtenteils auf dem Client ausgeführt wird. Beim ersten Aufruf der Anwendung lädt der Client die gesamte Anwendung und holt sich dann die benötigten Daten von der Zentrale. Der Server sendet nur dann neue Daten an den Client, wenn sich etwas ändert. Alle vom Client modifizierten Daten werden direkt an die Zentrale übermittelt und von dort aus an die Endgeräte weitergeleitet, um den Netzwerkverkehr zu minimieren.

7.3 Grundlagen & Technologien

Die Webanwendung basiert auf den Grundtechnologien HTML, CSS und JavaScript. Diese Entscheidung wurde schnell getroffen, da andere Programmiersprachen zwar auch die Möglichkeit bieten, Webanwendungen zu entwickeln, jedoch werden sie meistens erst in JavaScript übersetzt. Dadurch entsteht unnötige Komplexität und möglicherweise auch Performanceverlust. Unser Team verfügt bereits über Erfahrung mit den genannten Technologien, wodurch die Einarbeitungszeit minimiert wird.

Dennoch wird für die Entwicklung von SLOTH nicht direkt auf diese Technologien zurückgegriffen, sondern es werden JavaScript-Frameworks und -Bibliotheken verwendet. Eine Übersicht über diese Technologien ist in Abbildung 7.4 zu finden.



Abbildung 7.4: Alle Technologien im Überblick

Die Web-App wird in Client- und Server-Side unterteilt. Einige Technologien werden nur im Frontend oder Backend verwendet, während andere in beiden Teilen zum Einsatz kommen.

7.3.1 TypeScript

TypeScript ist eine Programmiersprache, die auf JavaScript aufbaut. Sie ermöglicht die Verwendung von statischer Typisierung, was die Fehlerrate in der Entwicklung minimiert. Darüber hinaus unterstützt TypeScript die neuesten JavaScript-Features und erleichtert somit die Entwicklung moderner Webanwendungen.[26]

Grundsätzlich wird TypeScript in beiden Teilen der Web-App verwendet. Es gibt mehrere Gründe dafür. Zum einen ist TypeScript eine Erweiterung von JavaScript, wodurch JavaScript-Code in TypeScript-Dateien verwendet werden kann. Zum anderen bietet TypeScript die Möglichkeit, Interfaces zu verwenden, die in JavaScript nicht vorhanden sind. Das bedeutet, dass es grundsätzlich auf Objekten aufbaut. Jedes Objekt kann ein anderes Objekt erweitern und verwendet dieses als Prototyp. JavaScript ist eine prototype-basierte Sprache. TypeScript hingegen ist eine objektorientierte Sprache.

Grundsätzlich führt TypeScript Erweiterungen wie Namespaces, Interfaces, Generics und statische Typisierung ein. Allerdings können Browser TypeScript-Code noch nicht direkt ausführen, weshalb der Code in JavaScript transpiliert werden muss. Der TypeScript-Compiler (TSC) wandelt den TypeScript-Code in JavaScript-Code um.[27]

Im Backend gibt es jedoch auch die Möglichkeit, TypeScript-Code direkt auszuführen. Technologien wie Bun.sh oder Node.js ermöglichen dies. Mit Node.js gibt es zum Beispiel eine Erweiterung (ts-node), welche den TypeScript-Code transpiliert und dann ausführt. Dadurch werden die zwei Schritte zu einem verbunden. Da bei der Webapplikation im Frontend sowie im Backend kein Node.js verwendet wird, wird hier nicht weiter darauf eingegangen.[28]

7.3.1.1 Server-Side

Der Server-Side Teil ist komplett in TypeScript geschrieben und nutzt Express.js und Socket.io. Er ist hauptsächlich für die Live-Updates verantwortlich, die auftreten, wenn sich ein Wert auf einem Endgerät ändert. Es ist wichtig, dass er so performant wie möglich ist und Typensicherheit sowie Ausfallsicherheit bietet, was durch TypeScript gewährleistet wird.

7.3.1.2 Client-Side

Der Client-Side Teil wurde vollständig in TypeScript mit den Frameworks Next.js und React.js sowie den Bibliotheken TailwindCSS und Socket.io-Client geschrieben. TypeScript wurde verwendet, um die Entwicklung zu erleichtern und die Typensicherheit zu gewährleisten. Es verbessert auch das objektorientierte Programmieren, das bereits in JavaScript möglich ist, durch die Verwendung von Funktionen wie Interfaces, die in JavaScript nicht vorhanden sind.

7.3.2 Express.js

Express.js, auch einfach nur 'Express' genannt, ist ein serverseitiges Web-Framework für JavaScript. Es wird in den meisten Fällen mit Node.js verwendet und ist ein minimalistisches, schnelles und flexibles Framework, das sich hervorragend für die Entwicklung von Webapplikationen und APIs eignet. Express ist ein Open-Source-Projekt, das von TJ Holowaychuk ins Leben gerufen wurde. Das Framework ist bei vielen großen Unternehmen wie IBM, Uber, PayPal und Fox Sports sehr beliebt und wird auch für Node.js genutzt.[29]

Express wird oft als Middleware bezeichnet, da es eine Reihe von Funktionen bereitstellt, die auf Anfragen und Antworten zugreifen können. Es kann auch für Routing verwendet werden, um Anfragen an die entsprechenden Endpunkte weiterzuleiten.[30]

Express ist ein leistungsstarkes Framework mit einer Vielzahl von Anwendungsmöglichkeiten. Es ist eine flexible und schnelle Bibliothek, die gut skaliert und effizient Anfragen und Antworten verarbeitet. Zudem ist Express.js weit verbreitet und wird von vielen Entwicklern genutzt.[31]

Über die HTTP-Methoden (GET, POST, PUT, DELETE, etc.) können Anfragen an den Server gesendet werden, welche von Express verarbeitet werden. Es können auch Parameter in der URL übergeben werden, um beispielsweise eine spezifische Ressource abzurufen.[32]

7.3.2.1 Node Package Manager (NPM)

Die meisten für JavaScript entwickelten Frameworks und Bibliotheken sind über das NPM-Register online verfügbar. NPM steht für 'Node Package Manager' und ist ein Paketmanager für JavaScript. Es wird hauptsächlich zur Verwaltung von Node.js-Paketen verwendet, aber auch für die Verwaltung von Frontend-Paketen.[33]

Express ist auch über das NPM-Register verfügbar und kann mit dem Befehl **npm install express** installiert werden.[34]

Es gibt jedoch auch andere Paketmanager wie beispielsweise 'Yarn' oder 'pnpm', die zur Verwaltung von JavaScript-Paketen genutzt werden können. In diesem Projekt wird jedoch Bun.sh verwendet, zu dem im weiteren Verlauf des Dokuments weitere Informationen gegeben werden. (Unterkapitel 7.3.7)

Um zu demonstrieren, wie Express.js mittels Bun.sh installiert werden kann, hier der entsprechende Befehl:**bun add express** [35]

7.3.2.2 Server-Side

Da Express ein serverseitiges Framework ist, wird es in diesem Projekt ausschließlich auf der Serverseite verwendet. Konkret wird es in Kombination mit Socket.io eingesetzt, um eine dauerhafte Verbindung zwischen Server und Client aufrechtzuerhalten und alle Änderungen, die von der Zentrale geschickt werden, an alle Clients weiterzuleiten, damit diese stets auf dem neuesten Stand sind. (Unterkapitel 7.3.3)

Im Rahmen dieses Projekts wird Express.js nicht für das Routing genutzt, da Next.js, ein weiteres auf React.js basierendes Framework, für das Routing zuständig ist. (Unterkapitel 7.3.4)

7.3.2.3 Client-Side

In dieser Arbeit wird auf der Clientseite kein Express.js verwendet. Stattdessen wird lediglich per Socket.io-Client mit dem Server kommuniziert, welcher hingegen Express.js verwendet. (Unterkapitel 7.3.3.2)

7.3.2.4 Beispiel

Die Abbildung 7.5 zeigt ein Beispiel für die Verwendung von Express.js. In diesem Fall wird ein Server erstellt, der auf dem Port 3000 lauscht und bei einer Anfrage auf der Route „*IP-Address* : 3000/*hello_world*“ die Worte „Hello World“ zurückgibt.[36]

```
const app = Express();

app.get("/hello_world", (req, res) => {
  ...res.send("Hello World");
});

app.listen(3000, () => {
  ...console.log("✓ Server listening on port 3000");
});
```

Abbildung 7.5: Express.js

7.3.3 Socket.io

Socket.io ist eine Bibliothek, die eine dauerhafte Verbindung zwischen Server und Client aufrechterhält. Sie ermöglicht eine bidirektionale Kommunikation zwischen den beiden Parteien und ist daher ideal für Echtzeitanwendungen geeignet, bei denen Ereignisse von beiden Seiten gesendet und empfangen werden können.[37]

Socket.io wurde ursprünglich von Guillermo Rauch entwickelt und ist eine Open-Source-Bibliothek.[38]

Es ist jedoch mehr als nur eine Bibliothek, es ist auch ein Protokoll, das auf WebSockets basiert, aber auch andere Technologien wie „Polling“ und „Long-Polling“ unterstützt.[39]

Socket.io ist eine sehr mächtige Bibliothek, welche nicht nur für Node.js, sondern auch für andere Plattformen, wie z.B. Java, Swift, C++ und vielen mehr, verfügbar ist. Sie baut auf der Engine.io-Bibliothek auf, welche die Grundlage für die Kommunikation zwischen Server und Client bildet. In weiterer Hinsicht bietet Socket.io auch noch Features wie „Auto-reconnection“, „Disconnection-detection“, „Binary support“, „Multiplexing support“ und viele mehr. [40]

7.3.3.1 Features

Wie bereits erwähnt, bietet Socket.io viele Features, die für Echtzeit-Anwendungen sehr nützlich sind.

Auto-reconnection: Wenn die Verbindung zwischen Server und Client unterbrochen wird, versucht Socket.io automatisch die Verbindung wiederherzustellen. Dies ist besonders nützlich, wenn die Verbindung nicht stabil ist, wie z.B. bei mobilen Geräten. Dies versucht der Client so lange, bis der Server wieder zur Verfügung steht und die Verbindung wiederhergestellt ist.

Disconnection-detection: Socket.io erkennt automatisch, wenn die Verbindung zwischen Server und Client unterbrochen wird und feuert ein Event, welches dann weiterverarbeitet werden kann. Dies ist besonders nützlich, um den Benutzer zu informieren, dass die Verbindung unterbrochen wurde. Dies wird über einen Timer realisiert, der überprüft, ob die Verbindung noch besteht, durch das Senden von Pings in einem bestimmten Intervall.

Binary support: Socket.io unterstützt auch das Senden von Binärdaten, wie z.B. Bilder, Videos, Audiodateien und viele mehr. Grundsätzlich jegliche serialisierbare Daten können übertragen werden, dies wird durch die Verwendung von „ArrayBuffer“ und „Blob“ im Browser, sowie auch „ArrayBuffer“ und „Buffer“ in Node.js, realisiert.

Multiplexing support: Socket.io unterstützt auch das Multiplexing, was bedeutet, dass mehrere Sockets über eine einzige Verbindung laufen können. Dies ist besonders nützlich, um die Anzahl der Verbindungen zu reduzieren und somit auch die Performance zu verbessern. Dies wird durch das Verwenden von „namespaces“ und „rooms“ realisiert, wobei „namespaces“ eine Art von „Channel“ sind, und „rooms“ eine Art von „Sub-Channel“ sind. Sockets können sich in „rooms“ eintragen und somit nur Nachrichten von anderen Sockets in der gleichen „room“ empfangen.

Reliability: Socket.io ist sehr zuverlässig und skaliert sehr gut über Zeit. Es ist auch sehr effizient mit den Anfragen und Antworten und ist gleichzeitig sehr flexibel und schnell. Verbindungen werden trotz Proxies, Load-Balancern, sowie personalisierten Firewalls ohne Probleme hergestellt.

Das sind nicht einmal alle Features, die Socket.io bietet, es gibt noch viele mehr, welche in der offiziellen Dokumentation, sowie auf der GitHub-Seite von Socket.io zu finden sind. [37] [40]

7.3.3.2 Socket.io-Client

Grundsätzlich ist Socket.io-Client die Client-Seite von Socket.io, welche in den meisten Fällen im Browser verwendet wird, kann jedoch schon auch bei anderen Plattformen, wie z.B. React Native, Swift, Java und vielen mehr, verwendet werden. Jedoch wird hier nicht viel weiter auf die Unterschiede zwischen Client und Server eingegangen, da die Implementierungen der jeweiligen APIs sehr ähnlich sind. [38]

7.3.3.3 Beispiel

In Abbildung 7.6 ist ein Beispiel zu sehen, wie Socket.io am Server verwendet werden kann. In diesem Fall wird eine Verbindung aufgebaut und auf ein Event „connection“ gelauscht, welches dann weiterverarbeitet wird. Dieses Event wird gefeuert, wenn ein Client sich mit dem Server verbindet. [41]

```
const io = new Server(3000, {  
  ... /* options */  
});  
  
io.on("connection", (socket) => {  
  ... // ...  
});
```

Abbildung 7.6: Socket.io Server Beispiel

In Abbildung 7.7 ist ein Beispiel zu sehen, wie Socket.io am Client verwendet werden kann. In diesem Fall wird eine Verbindung zum Server aufgebaut und auf ein Event „connect“ gelauscht, welches dann weiterverarbeitet wird. Dieses Event wird gefeuert, wenn der Client sich mit dem Server verbindet. [42] [43]

```
const socket = io();

// client-side
socket.on("connect", () => {
  console.log(socket.id); // x8WIV7-mJelg7on_ALbx
});

socket.on("disconnect", () => {
  console.log(socket.id); // undefined
});
```

Abbildung 7.7: Socket.io Client Beispiel

7.3.3.4 Server-Side & Client-Side

In diesem Projekt wird Socket.io am Server verwendet, um eine dauerhafte Verbindung zwischen Server und Client aufrechtzuerhalten, um neue Informationen, wie z.B. wenn ein Sensorwert sich ändert, an den Client zu senden. Damit dieser auf dem neuesten Stand ist und nicht manuell die Seite aktualisieren muss. Bei dem Weiterleiten von den Informationen, von der Zentrale an die Clients, wird einfach ein Event gefeuert (Das „update“ Event), welches dann von den Clients weiterverarbeitet wird. Wodurch der Server einfach nur es umleitet und nicht als erstes verarbeiten muss.

In Abbildung 7.8 ist ein Ausschnitt aus dem Projekt zu sehen, wie Socket.io am Server verwendet wird. Immer wenn ein Aufruf von der Zentrale kommt, auf die Route „/update“, wird ein Event „update“ gefeuert, welches den Body von der POST Request übergibt an die Clients von denen es in weiterer Folge, verarbeitet wird. (Das Objekt „app“ ist in diesem Fall eine Instanz von Express.js, welches in Abschnitt 7.3.2 erklärt wird.)

```
io.on("connection", (socket) => {
  ... console.log("client connected : " + socket.id + " " + socket.handshake.query.id);
  ... socket.emit(`connected`, `${socket.id}`);
  ... socket.on("client-ready", () => {});

  ... app.post("/update", async (req, res) => {
  ...   socket.broadcast.emit("update", req.body);
  ...   res.send("OK");
  ... });

  ... socket.conn.on("close", () => {
  ...   prevConnection = socket.id;
  ...   socket.conn.close();
  ...   console.log("client disconnected : " + socket.id);
  ...   socket.broadcast.emit("disconnected", `${prevConnection}`);
  ... });
});
```

Abbildung 7.8: Socket.io Server-Side Ausschnitt aus dem Projekt

In Abbildung 7.9 ist ein Ausschnitt aus dem Projekt zu sehen, wie Socket.io am Client verwendet wird. Immer wenn ein Event „update“ gefeuert wird, wird eine Lambda-Funktion ausgeführt, welche dann den Body, verarbeitet und mit der jetzigen Message vergleicht, falls es sich geändert hat, wird das Objekt der alten Message mit dem neuen ersetzt und die Komponente, welche die Message anzeigt, wird neu gerendert. (Das Objekt „Message“ wird in einem späteren Abschnitt erläutert)

```
props.server.on("update", (arg: Message[]) => {
  ... arg.forEach(async (updatedMessage: Message) => {
  ...   if (message.Id === updatedMessage.Id) {
  ...     const oldMessage = message;
  ...     const newMessage = {
  ...       Id: updatedMessage.Id,
  ...       MessageType: updatedMessage.MessageType,
  ...       Room: updatedMessage.Room,
  ...       Type: updatedMessage.Type,
  ...       Unit: updatedMessage.Unit,
  ...       Value: updatedMessage.Value,
  ...       Image: oldMessage.Image,
  ...       TimeStamp: oldMessage.TimeStamp,
  ...       Title: oldMessage.Title,
  ...       Tags: oldMessage.Tags
  ...     } as Message;
  ...     setMessage(newMessage);
  ...     // Replace the existing device
  ...     setValue(newMessage.Value);
  ...   }
  ... });
});
```

Abbildung 7.9: Socket.io Client-Side Ausschnitt aus dem Projekt

7.3.4 Next.Js

Next.js ist ein Framework, welches auf React aufbaut, die Grundidee von Next.js, manchmal auch nur „Next“ genannt, ist das Verwenden von React-Anwendungen mit „Server-Side-Rendering“ (SSR) und „Static-Website-Generation“. Dies ist ein großer Vorteil, da die Webanwendung dadurch schneller geladen wird und somit die Nutzererfahrung verbessert wird. Dieses Framework wird verwendet um Full-Stack Webanwendung mit React zu entwickeln. [44]

Jedoch ist Next.js nicht nur auf SSR und Static-Website-Generation beschränkt, sondern bietet mittlerweile auch viele weitere Features, von Route Handling, über API-Routing, bis hin zu Data-Fetching [45]

Next.js ist ein Open-Source-Projekt und wird von Vercel entwickelt, einem Unternehmen, welches sich in der Entwicklung von ihrer Frontend-Cloud spezialisiert hat. [46]

Diese Cloud bietet Features, wie „Elastic scalability“, oder „Rock-solid security“. Die Vercel-Plattform ist nicht nur auf Next.js beschränkt, sondern unterstützt auch viele andere Frameworks und Technologien, wie zum Beispiel Angular, Vue.js, Astro, Python oder auch Vite. [47]

7.3.4.1 Features

Next.js bietet viele Features, die das Entwickeln von Webanwendungen erleichtern. Einige dieser Features sind:

- **Server-Side-Rendering (SSR):** Next.js bietet die Möglichkeit, Webseiten auf dem Server zu rendern, anstatt auf dem Client. Welche die Last vom Client nimmt und die Webanwendung schneller lädt.
- **Static-Site-Generation:** Dazu kann Next Webseiten statisch generieren, direkt zur Build-Zeit, anstatt zur Laufzeit. Dies ist besonders nützlich, wenn die Daten sich nicht oft ändern und die Webanwendung dadurch schneller geladen wird.

- **Automatic Code Splitting:** Next.js teilt den Code automatisch in kleinere Teile auf, um die Ladezeit der Webanwendung zu verbessern.
- **Hot Module Replacement:** Module können im laufenden Betrieb ausgetauscht werden, ohne die gesamte Anwendung neu zu laden. Somit wird das Deployment von Anwendungen erleichtert und schneller, da Änderungen direkt im Browser sichtbar sind, ohne irgendeine Downtime.
- **Image Optimization:** Next hilft weiters mit dem automatischen Optimieren von Bildern, um die Ladezeit der Webanwendung zu verbessern.
- **Route Handlers:** Mit Next.js ist es genauso möglich, API Endpoints zu erstellen, um sicher und einfach Daten von Third-Party-Services zu holen, oder auf Webhooks zu hören.
- **Server Actions:** Next.js bietet die Möglichkeit, Server-Code rennen zu lassen, direkt mit einem Funktionsaufruf, ganz ohne API.
- **Advanced Routing & Nested Layouts:** Man kann Fortgeschrittene Routen verwenden, sowie verschachtelte Layouts. Alles basierend auf dem Dateisystem.
- **Middleware:** Kann verwendet werden, um Anfragen zu verarbeiten, bevor sie an die Route-Handler weitergeleitet werden.

Das sind nicht alle Features, die Next.js zu bieten hat, aber die größten und wichtigsten, die das Entwickeln von Webanwendungen erleichtern.[45][48]

7.3.4.2 Server-Side & Client-Side

In diesem Projekt wurde Next.js hauptsächlich für den Client verwendet, da der Server auf Express.js basiert und läuft. Wobei Next.js verwendet wurde, um die Webanwendung zu entwickeln und zu rendern.

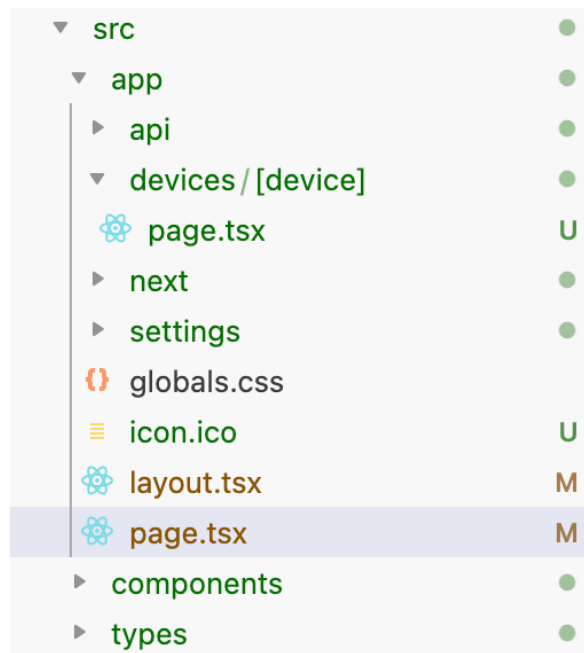


Abbildung 7.10: Dateibaumstruktur von Next.js

In der Abbildung 7.10 sieht man die Dateibaumstruktur von der Webanwendung, welche wie bereits beschrieben auf Next.js aufbaut. Jede Hauptseite eines Next.js Ordners ist eine React-Komponente, und muss „page“ genannt werden. Diese Komponenten werden dann von Next.js gerendert und zur Laufzeit oder zur Build-Zeit, je nachdem, ob es sich um eine statische Seite handelt oder nicht. Alle anderen React Dateien, sind Komponenten, die in den „page“ Dateien verwendet werden.

Wie in der Abbildung 7.11 zu sehen ist, wurde auch eine API in Next.js erstellt, um gewisse Daten vom Next Server zu holen.

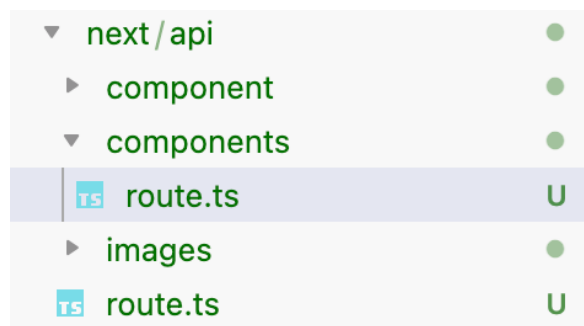


Abbildung 7.11: Dateibaumstruktur von der API in Next.js

Diese API wurde unter dem Ordner „next“ erstellt, in diesen Ordner und Unterordnern befindet sich keine „page“-, genauso wie auch keine anderen React-Dateien. Es befinden sich ausschließlich API-Routen, welche von Next erkannt werden, durch das Schlüsselwort „route“ in den Dateinamen.

In diesen 'route'-Dateien können die HTTP-Methoden wie 'GET', 'POST', 'PUT' oder 'DELETE' definiert werden. Wie in Abbildung 7.12 zu sehen ist, wurde eine Route mit der HTTP-GET-Methode erstellt, um eine Datei einzulesen und die Informationen als JSON-Array zurückzugeben.

```
export async function GET(): Promise<NextResponse<Message[]>> {  
  ...const filePath = path.join(process.cwd() + "/public/components/declaredComponents.json");  
  ...let fileData = fs.readFileSync(filePath, "utf8");  
  ...let dataArray;  
  ...// Check if fileData is not empty and is valid JSON  
  ...if (fileData) {  
    ...dataArray = JSON.parse(fileData);  
  ...}  
  
  ...// If dataArray is not an array, initialize it as one  
  ...if (!Array.isArray(dataArray)) {  
    ...return NextResponse.json([]);  
  ...}  
  
  ...return NextResponse.json(dataArray);  
}
```

Abbildung 7.12: Beispiel aus unserem Projekt, von einer API-Route in Next.js

7.3.5 React

React ist eine JavaScript-Bibliothek, die von Meta und vielen anderen Entwicklern und Unternehmen entwickelt wird. Sie wird zur Entwicklung von Benutzeroberflächen verwendet und basiert auf Komponenten, die in weiteren Komponenten geschachtelt sind. React ist eine der beliebtesten Bibliotheken für die Entwicklung von Webanwendungen und wird von vielen großen Unternehmen wie Netflix, Airbnb, Tesla, Instagram und vielen anderen genutzt.[49][50]

React, auch React.js oder ReactJS genannt, ist eine Open-Source-Bibliothek, die von einer großen Community unterstützt wird. Sie wurde ursprünglich von Meta, damals noch Facebook, entwickelt und im Jahr 2013 veröffentlicht. React.js wird häufig für die Entwicklung von Single-Page-Applications (SPA) und mobilen Anwendungen verwendet. Es wird auch für serverseitiges Rendern und als Basis für andere Frameworks und Bibliotheken wie Next.js (7.3.4) genutzt.[49]

Das Framework ist ausschließlich für das Rendern von Komponenten im DOM (Document Object Model) zuständig. Es ist nicht für die Verwaltung von Daten oder das Abfragen von APIs verantwortlich. Hierfür gibt es andere Bibliotheken und Frameworks, die in Kombination mit React verwendet werden können. Ein Beispiel hierfür ist Express.js (7.3.2), das für das Anfragen von APIs und das Routing verantwortlich ist. Einer der wichtigsten und größten Vorteile von React besteht im Rendern von Komponenten. React rendert, um die Performance der Anwendung zu verbessern und die Ladezeiten zu verkürzen, nur die Komponenten neu, die sich geändert haben, also nicht direkt die gesamte Seite.[49]

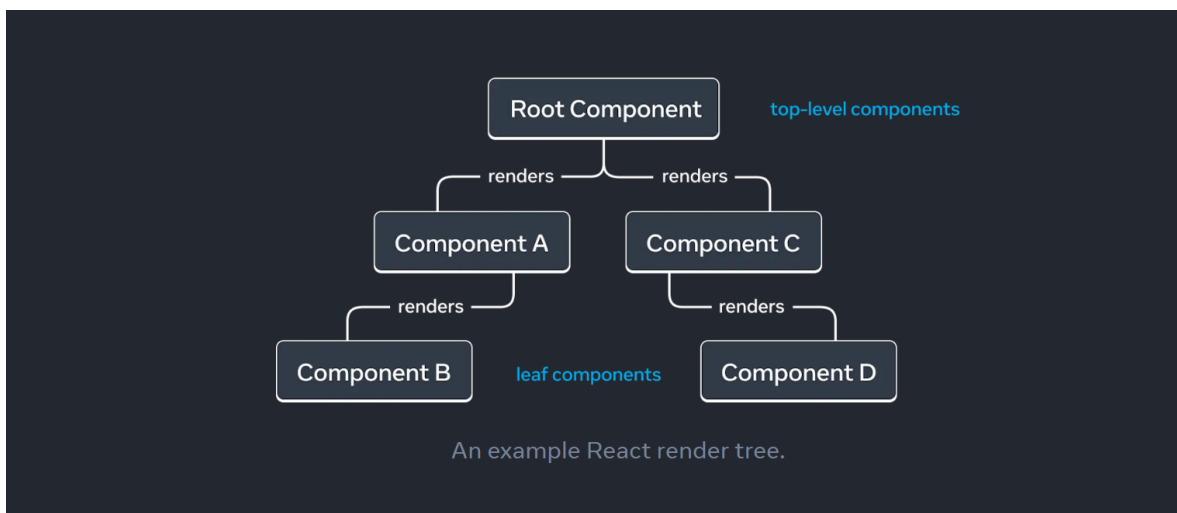


Abbildung 7.13: Beispiel einer React-Komponenten-Struktur

Wie in der Abbildung 7.13 dargestellt, besteht React aus Komponenten, die ineinander verschachtelt sind. Jede Komponente hat ihren eigenen Zustand und Eigenschaften, die sich ändern können und dazu führen, dass die Komponente neu gerendert wird.

Die Eigenschaften einer Komponente können mittels Props von einer anderen Komponente übergeben werden und sind unveränderlich. Komponenten können außerdem eigene Methoden und Funktionen besitzen. Methoden sind Funktionen, die bei Änderung des Zustandes oder der Eigenschaften aufgerufen werden.[49]

7.3.5.1 Vorteile von React

React.js ist eine der beliebtesten Bibliotheken für die Entwicklung von Webanwendungen aufgrund der vielen Vorteile, die sie bietet.

- **Performance:** React rendert nur die Komponenten neu, die sich geändert haben, und nicht die gesamte Seite. Dadurch wird die Performance der Anwendung verbessert und die Ladezeiten verkürzt.
- **Wiederverwendbarkeit:** Komponenten können wiederverwendet und in anderen Komponenten eingebunden werden. Dadurch wird der Code sauberer und übersichtlicher, was sowohl die Wartung als auch die Erweiterung der Anwendung erleichtert. Außerdem können Tests schnell und einfach für einzelne Komponenten geschrieben und durchgeführt werden.
- **SEO-freundlich:** React-Anwendungen können serverseitig gerendert werden, was die Indexierung durch Suchmaschinen erleichtert.
- **Flexibilität:** React kann mit anderen Bibliotheken und Frameworks kombiniert werden. Dazu gehören Express.js, Next.js und viele andere.
- **Virtual DOM:** React verwendet ein virtuelles DOM, das die Performance der Anwendung verbessert und die Ladezeiten verkürzt. Das virtuelle DOM rendert nur die Komponenten neu, die sich geändert haben, indem es den virtuellen DOM mit dem echten DOM vergleicht.
- **Multiplattform:** React kann für die Entwicklung von Web- und mobilen Anwendungen verwendet werden. Mit React Native können native mobile Anwendungen für iOS und Android entwickelt werden, die auf den Grundlagen der Webentwicklung und React.js basieren.

Diese Vorteile machen React zu einer der beliebtesten Bibliotheken für die Entwicklung von Web- und mobilen Anwendungen. Darüber hinaus gilt React als sehr einsteigerfreundliches Framework und eignet sich sowohl für Anfänger als auch für erfahrene Entwickler.[49][51][52]

7.3.5.2 Beispiel

Die Abbildung 7.14 zeigt ein einfaches Beispiel einer React-Komponente. Diese Komponente rendert einen einfachen Text in einem H1-Tag, das sich in einer div-Komponente befindet.

```
2  function App() {  
3    · return (  
4      · · · <div>  
5      · · · | · <h1>Hello World</h1>  
6      · · · </div>  
7    · );  
8  }
```

Abbildung 7.14: React JSX Beispiel

JSX ist im Wesentlichen eine JavaScript-Erweiterung, die React verwendet, um HTML-Elemente in JavaScript zu schreiben. Wie man im oben gezeigten Beispiel gut sehen kann. Eine React-Komponente hat im Normalfall immer ein HTML-Element als Rückgabewert.

In der Abbildung 7.15 wird zunächst eine externe Konstante erzeugt und befüllt. Diese Konstante wird dann mit geschwungenen Klammern in die JSX-Komponente eingebunden, wodurch der Text der Konstanten in der Komponente gerendert wird.

```
2  function App() {  
3    · const message = "Hello World";  
4    · return (  
5    · · · <div>  
6    · · · | · <h1>{message}</h1>  
7    · · · </div>  
8    · );  
9  }
```

Abbildung 7.15: React JSX Beispiel mit einer externer Konstante

7.3.5.3 Server-Side

In diesem Smart Home Projekt wurde React nicht auf dem Server verwendet, da dieser, wie bereits erwähnt, auf Express.js (7.3.2) basiert und läuft. React wurde hauptsächlich für die Entwicklung und das Rendering der Webanwendung, also des Clients, verwendet.

7.3.5.4 Client-Side

Die Webanwendung besteht ausschließlich aus React-Komponenten, von denen viele durch Client-Side-Rendering dargestellt werden, einige aber auch direkt durch Server-Side-Rendering vorkompiliert werden. Da es aber in den meisten Fällen notwendig ist, einen Zustand, z.B. einer Komponente, zu verwalten, mussten wir die Berechnungen dieser Komponenten auf den Client auslagern. Einige Hooks von React, wie `useState`, `useEffect`, etc. können nicht serverseitig gerendert werden, daher war es notwendig, die Komponenten direkt auf dem Client zu rendern.

Da React selbst nicht in der Lage ist, Komponenten serverseitig zu rendern, dafür wird in dieser Arbeit Next.js (7.3.4) verwendet.

```

<div className="h-screen w-screen flex flex-wrap bg-gray-100">
  <Header url={usePathname()} />
  <div className="pt-[5rem] text-black flex flex-col h-full w-full items-center">
    {addDeviceIsVisible && (
      <input
        type="button"
        value="Add Device"
        className="border-2 rounded-md border-slate-200 p-2 bg-slate-200 transition-all hover:p-3 m-2 hover:m-1"
        onClick={() => router.push("settings/new_device")}
      />
    )}
    <input
      type="button"
      value="Add Events"
      className="border-2 rounded-md border-slate-200 p-2 bg-slate-200 hover:
        transition-all hover:p-3 m-2 hover:m-1"
      onClick={() => router.push("settings/events")}
    />
    ...more settings soon to be implemented!
  </div>
</div>

```

Abbildung 7.16: Ausschnitt aus diesem Projekt, eine React-Komponente

In der Abbildung 7.16 ist ein Ausschnitt aus diesem Projekt zu sehen, eine React-Komponente, die grundsätzlich für die Settings-Seite verwendet wird. Da diese relativ übersichtlich ist, wurde sie hier als Beispiel verwendet.

Wie man unschwer erkennen kann, besteht diese Komponente aus vielen anderen Komponenten, sowohl vorgefertigten HTML-Elementen als auch eigenen Komponenten wie „Header“. Bei näherer Betrachtung fällt auf, dass geschwungene Klammern verwendet werden, nicht um eine externe Konstante einzubinden, sondern um den Zustand einer Variable abzufragen und je nachdem, ob dieser wahr oder falsch ist, eine Komponente zu rendern oder nicht. Bei dieser Variable handelt es sich um „addDeviceIsVisible“, diese ist ein Boolean und kann dadurch nur wahr oder falsch sein. Je nachdem, ob diese wahr oder falsch ist, wird ein Input-Feld gerendert.[53]

Geschwungene Klammern können in JSX für viele verschiedene Dinge verwendet werden, aber in den meisten Fällen werden sie verwendet, um Variablen einzufügen oder Variablen abzufragen. Dafür werden sie auf jeden Fall in diesem Projekt am häufigsten verwendet.

Wie bereits erwähnt, bietet React.js viele verschiedene Features. In dieser Arbeit wurden einige davon verwendet, es gibt jedoch noch viele weitere Möglichkeiten, die nicht behandelt wurden. Ein großer Teil dieses Projekts bestand darin, Zustände zu verwalten und Komponenten zu rendern. Hierfür wurden hauptsächlich die React-Hooks 'useState' und 'useEffect' genutzt. Diese Hooks ermöglichen es, den Zustand einer Komponente zu verwalten und bei Änderungen des Zustands eine Funktion aufzurufen. Dazu wurden einige maßgeschneiderte Komponenten erstellt, um die Anwendung übersichtlicher und wartbarer zu gestalten.

Insgesamt bietet React.js viele Vorteile und zählt zu den beliebtesten modernen Web-Frameworks auf dem Markt. Besonders in Kombination mit anderen Bibliotheken wie Next.js (7.3.4) oder Express.js (7.3.2) ist es eine hervorragende Wahl. Aus diesem Grund haben wir uns für React.js in diesem Projekt entschieden.

7.3.6 TailwindCSS

TailwindCSS, manchmal auch einfach Tailwind genannt, ist ein CSS-Framework, das auf Utility-First basiert, dazu später mehr. Es ist ein Open-Source-Projekt, das 2019 mit der Version 1.0.0 sein erstes stabiles Release feierte. Ursprünglich von Adam Wathan, Jonathan Reinink, David Hemphill und Steve Schoger entwickelt, hat Tailwind in den letzten Jahren eine große Community aufgebaut und wird von vielen Entwicklern und Unternehmen genutzt.[54][55]

Vor Version 3 hat Tailwind alle CSS-Styles gesäubert, um alle unbenutzten Styles aus dem fertigen Produktions-Build zu entfernen, damit dieser so klein wie möglich ist. Mit Version 3 wurde das sogenannte PurgeCSS jedoch durch den neuen JIT-Compiler (Just-in-Time Compiler) ersetzt. Dieser kompiliert nur die Styles, die in der Anwendung gerade benötigt werden. Dadurch verkürzt sich die Kompilierzeit und die Anwendung wird schneller.[56]

Was bedeutet es ein Utility-first CSS-Framework zu sein? Damit ist gemeint, dass viele verschiedene Utilities zur Verfügung stehen, und diese frei miteinander kombinierbar sind, um somit das gewünschte Design zu erhalten. Diese sind in CSS-Klassen geschrieben und können somit in allen HTML-Elementen und React-Komponenten verwendet werden. Ein Beispiel ist die Klasse **bg-red-500**, die die Hintergrundfarbe des Elements auf Rot setzt.[55]

Im Gegensatz zu anderen CSS-Frameworks wie Bootstrap oder Foundation, die viele vorgefertigte Komponenten und Designs mit sich bringen, bietet TailwindCSS nur die Utilities. Zum Beispiel bietet Bootstrap mehrere vorgefertigte Styles für HTML-Buttons, während Tailwind sich auf Style-Klassen wie „bold font“, „large text“, etc. spezialisiert. Dies gibt den Entwicklern mehr Freiheit und Flexibilität, um das Design der Anwendung individueller zu gestalten.[55]

7.3.6.1 Beispiel

Abbildung 7.17 zeigt ein Beispiel, wie ein HTML-Element oder eine React-Komponente mit TailwindCSS erweitert werden kann.

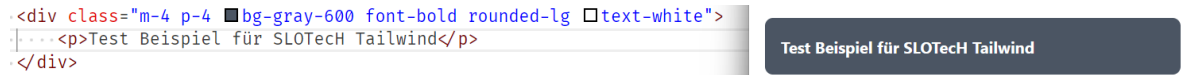


Abbildung 7.17: TailwindCSS Beispiel für eine Simple Komponente

Im Beispiel gibt es ein Paragraph-Element, das von einem Div-Element umschlossen wird. Auf das Div-Element werden einige Tailwind-Klassen angewendet, um der Komponente einen Stil zu geben. Die Klassen „m-4“ und „p-4“ sind für den äußeren Abstand und den inneren Abstand zuständig, „bg-gray-600“ setzt die Hintergrundfarbe auf ein dunkles Grau. Die Klasse „font-bold“ setzt den Text von diesem und von jedem Unterelement auf fett, „rounded-lg“ rundet die Ecken des Elements ab. Schließlich setzt 'text-white' die Textfarbe auf Weiß. Das fertige Design dieser Komponente ist direkt neben dem Code dargestellt.

Auf den ersten Blick mag es ungewohnt sein, so viele Klassen auf einem Element zu sehen. Im Allgemeinen macht es das Design und die Wartung der Anwendung dennoch einfacher und schneller. Als Entwickler muss man sich zu Beginn erst an die Klassennamen und deren Bedeutung gewöhnen. Dies ist jedoch schnell erlernt und wenn man etwas Neues benötigt, gibt es die offizielle Dokumentation, die alle Klassen und deren Bedeutung beinhaltet.[54]

7.3.6.2 Features

Wie bereits erwähnt, bietet Tailwind verschiedenste Utility-Klassen an, die zum Designen von Anwendungen verwendet werden können. Das ist jedoch nicht alles, was Tailwind mit sich bringt. Neben diesen Klassen gibt es jedoch noch viele weitere Features, die das Entwickeln & Stylen von Anwendungen erleichtern und beschleunigen. Einige davon sind:

- **Settings and themes:** TailwindCSS bietet per `tailwind.config.js` die Möglichkeit, das Design der Anwendung zu konfigurieren, sowie vordefinierte Klassen zu verändern oder neue hinzuzufügen. Dies umfasst die Anpassung von Farben, Schriftarten, Abständen, Größen und vielem mehr. Es ist auch möglich,

verschiedene Themes zu erstellen und zu verwenden, um die Programmierrichtlinie DRY (Don't-Repeat-Yourself) einzuhalten und das Factory Pattern, auch für Tailwind-Klassen, nutzen zu können. All das vereinfacht das Ändern des Designs der Anwendung.

- **Utility classes:** TailwindCSS bietet eine Vielzahl von Utility-Klassen, die in der Anwendung verwendet werden können. Diese sind in verschiedene Kategorien unterteilt, wie zum Beispiel Hintergrundfarben, Schriftgrößen, Schriftarten, Abstände, Schatten und viele mehr. Die Klassen können in jedem HTML-Element oder React-Komponenten verwendet werden, um das Design zu verändern. TailwindCSS unterstützt mittlerweile viele verschiedene JavaScript-Frameworks neben React, wie Vue.js, Angular und weitere.
- **Variants:** Diese Varianten ermöglichen es Tailwind, Responsive Design umzusetzen. Dabei wird einer Utility-Klasse ein Präfix hinzugefügt, um das Design für verschiedene Bildschirmgrößen anzupassen. Zum Beispiel wird 'sm:' für kleine Bildschirme, 'md:' für mittlere Bildschirme und 'lg:' für große Bildschirme verwendet. Es gibt auch Varianten, die alle möglichen Zustände abdecken, wie Hover, Focus, Active und Disabled. Alle Varianten können kombiniert werden, um das Design der Anwendung individueller zu gestalten. Zudem sind alle Varianten konfigurierbar, um das Design der Anwendung flexibler zu gestalten.

Die genannten Eigenschaften stellen den Kern von TailwindCSS dar. Es gibt jedoch noch weitere Funktionen, die TailwindCSS bietet, um die Entwicklung und Gestaltung von Anwendungen zu erleichtern und zu beschleunigen. Dazu gehört das Hinzufügen von benutzerdefinierten Utility-Klassen. Neben den bereits erwähnten Funktionen bietet TailwindCSS die Möglichkeit, benutzerdefinierte Werte für manche Klassen zu verwenden. Zum Beispiel kann man, wenn man das vordefinierte Padding von 5 nicht mag ('p-5'), ganz einfach ein selbst definiertes Padding mit eckigen Klammern angeben ('p-[5.5rem]').

Wie zuvor beschrieben, wird seit der Veröffentlichung von Tailwind Version 3.0.0 standardmäßig der JIT-Compiler verwendet. Deswegen ist es wichtig zu erwähnen, sollte aus irgendeinem Grund der veraltete 'Build all and Purge'-Modus benötigt werden, muss eine Tailwind-Version vor 3.0.0 heruntergeladen und genutzt werden.

All das macht TailwindCSS zu einem weitaus mächtigem Werkzeug, um das Design von Anwendungen zu gestalten und zu entwickeln. Es ist eines der größten modernen CSS-Frameworks und wird von vielen Unternehmen, sowie einzelstehenden Entwicklern genutzt.[55]

7.3.6.3 Server & Client

Auf der Serverseite wird TailwindCSS in dieser Arbeit nicht benutzt, umso mehr jedoch auf der Clientseite. Das komplette Design der Webanwendung wird mit TailwindCSS gestaltet. Da in diesem Projekt React (7.3.5) als Frontend-Framework verwendet wird, ist es möglich, Tailwind direkt in den React-Komponenten zu verwenden.

```

..... <div className="w-[12.5em] h-[12.5em] border-[2px] rounded-lg border-none □bg-gray-100 m-2 ■text-black grid grid-cols-1 grid-rows-1
      select-none items-center place-items-center justify-center overflow-hidden cursor-default □hover:bg-gray-50 transition-all">
.....   { " " }
.....   <Link href="/settings/new_device">
.....     <div className="rounded-full □bg-gray-200 w-[6rem] h-[6rem] border border-none m-3 flex justify-center items-center
.....       hover:cursor-pointer □hover:bg-slate-300 hover:h-[20.5rem] hover:w-[20.5rem] transition-all text-2xl hover:text-3xl">
.....         <span className="">Add</span>
.....       </div>{ " " }
.....     </Link>
.....   </div>

```

Abbildung 7.18: TailwindCSS in Benutzung auf der Clientseite

In Abbildung 7.18 ist ein Beispiel zu sehen, aus dieser Arbeit, wie TailwindCSS in einer React-Komponente verwendet wird. Es wird hier nur auf Besonderheiten eingegangen, da es bereits ein Beispiel für eine TailwindCSS-Komponente gibt. Dazu gehört die Nutzung von Varianten oder eigens definierten Werten für Klassen. Zum Beispiel wurde die Klasse 'w-[12.5rem]' verwendet, um die Breite des Elements auf 12.5rem zu setzen. Zusätzlich wurde die Klasse 'hover:bg-gray-50' genutzt, um die Hintergrundfarbe des Elements zu ändern, sobald der Mauszeiger darüber bewegt wird.

7.3.7 Bun.sh

Bun.sh, auch bekannt als Bun, ist ein „all-in-one toolkit“, welches für die Entwicklung von JavaScript, sowie TypeScript Applikationen verwendet wird. Im Kern von Bun.sh befindet sich die Bun-Runtime, welche von Grund auf mit der Idee entwickelt wurde, Node.js durch das Schreiben in der Programmiersprache Zig zu ersetzen, wodurch die Performance, sowie der Speicherverbrauch im Vergleich zu Node.js drastisch reduziert werden konnte.[57]

7.3.7.1 Bun CLI

Zur Runtime, bringt Bun.sh auch ein Command Line Interface (CLI) mit sich. Damit können Projekte erstellt, gestartet und verwaltet werden. Das CLI wird mit dem Befehl **bun** aufgerufen. Mit dem Bun-CLI ist es möglich, sehr einfach Tests, sowie Skripte laufen zu lassen, zusätzlich liefert bun noch einen Node kompatiblen Package Manager mit sich, der es ermöglicht, Packages zu installieren und zu verwalten. Das alles schneller und effizienter als mit Node.js und kann in bestehenden Node.js Projekten verwendet werden.

Die Ziele von Bun sind klar und deutlich in der Dokumentation ersichtlich. Schnelligkeit und Effizienz, native TypeScript & JSX Unterstützung, Kompatibilität mit ESM und CommonJS, Implementierung von Web-Standard-APIs und Node.js-Kompatibilität sind einige der Design-Ziele, die sich Bun.sh gesetzt hat.[57]

7.3.7.2 Features

Bun bietet eine Vielzahl an Features, welche die Entwicklung von JavaScript und TypeScript Applikationen erleichtern. Dazu gehören unter anderem:

- **Bun-CLI:** Eine Kommandozeilenschnittstelle, die die Entwicklung von Anwendungen erleichtert, indem Projekte erstellt und Anwendungen mit einem einzigen Befehl gestartet werden können.

- **Bun-Runtime:** Eine blitzschnelle Laufzeitumgebung um Node.js zu ersetzen. Bei der Entwicklung der Runtime in Zig wurde großer Wert auf Schnelligkeit und Effizienz gelegt.
- **Script-runner:** Skripte können sehr einfach per CLI mit Bun ausgeführt werden. Zum Beispiel kann das Skript „start“ einfach mit dem Befehl **bun run start** ausgeführt werden.
- **Test-runner:** Die Tests können auch einfach über eine Kommandozeilenschnittstelle ausgeführt werden. Der Befehl **bun test** ermöglicht eine einfache Ausführung der Tests.
- **Package Manager:** Bun bietet einen eigenen Package Manager, mit dem Pakete installiert und verwaltet werden können. Dieser ist schneller und effizienter als der Node Package Manager. Je nach Betriebssystem werden Pakete, falls sie bereits installiert sind, nur noch per Link eingebunden. Dies spart Speicherplatz und beschleunigt die Installation.
- **Bundler:** Der Bun-Bundler ermöglicht das Bündeln von JavaScript-Code in einer einzigen Datei. Er ist mit esbuild kompatibel und ermöglicht eine schnelle und effiziente Bündelung von JavaScript-Code.
- **TS & JSX Support:** Bun bietet native Unterstützung für TypeScript und JSX. Daher ist kein zusätzlicher Transpiler erforderlich, um es in JavaScript zu übersetzen.
- **ESM & CommonJS Kompatibilität:** Bun ist sowohl mit ESM als auch mit CommonJS kompatibel. Dies ermöglicht die Verwendung von Modulen, die in ESM geschrieben wurden, in Projekten, die in CommonJS geschrieben wurden und umgekehrt. „Import“ und „require“ Statements werden im selben Projekt, als auch in derselben Datei unterstützt. Im Blog Post vom 8. September 2023, wurde beschrieben: „You can even use import and require(), in the same file. **It just works**“^[58]

- **Web-Standard APIs:** Bun implementiert Web-Standard-APIs, welche im Browser verfügbar sind, wie „fetch“, „Request“, „Response“, „WebSocket“, und „ReadableStream“. Es ist nicht mehr notwendig, Bibliotheken von Drittanbietern wie „node-fetch“ oder „ws“ zu verwenden. Diese Web-Standard-APIs wurden in den Kern von Bun integriert und sind daher schneller und zuverlässiger als Alternativen von Drittanbietern.
- **Node.Js Kompatibilität:** Bun ist mit Node.Js kompatibel. Das bedeutet, dass bestehende Node.Js Projekte einfach auf Bun migriert werden können. Die meisten Node.Js Projekte können ohne Änderungen in Bun ausgeführt werden. Der Package Manager von Bun greift auch auf die Pakete des Node Package Managers zu, wodurch die Kompatibilität gewährleistet ist. Globale Variablen von Node sind auch in Bun verfügbar, ebenso wie Module, die die von Node ersetzen oder erweitern („fs“, „path“, „net“).

Diese Eigenschaften machen Bun zu einem sehr mächtigen Werkzeug für die Entwicklung von JavaScript- und TypeScript-Anwendungen. Die Schnelligkeit und Effizienz, die Bun bietet, machen es zu einer sehr attraktiven Alternative zu Node.Js, weshalb es auch in dieser Arbeit verwendet wurde. Dies sind bei weitem nicht alle Features, die Bun zu bieten hat, es gibt noch viele mehr, wie z.B. „Hot reloading“ etc. Es ist jedoch zu beachten, dass sich Bun.sh noch in einer frühen Entwicklungsphase befindet und daher noch nicht unbedingt für große Produktionsumgebungen geeignet ist.[57][58]

7.3.7.3 APIs

Bun.sh implementiert und stellt viele verschiedene APIs zur Verfügung, die extra optimiert wurden und in den Kern von Bun integriert sind.

Eine Datei kann einfach mit **Bun.file('file.txt')** eingelesen werden. Das Ergebnis dieser Methode ist ein „BunFile“, welches den Webstandard „File“ erweitert. Der Inhalt der Datei kann über verschiedene Methoden geladen werden, als einfacher Text, bis hin zu einem JSON (JavaScript Object Notation) Objekt. Bun.sh kann Dateien bis zu zehnmal schneller lesen als Node.Js.

Auch das Schreiben von Dateien ist mit Bun.sh möglich. Mit der Methode **Bun.write('file.txt', 'Hello World')** kann fast alles in eine Datei geschrieben werden, von einfachem Text bis hin zu HTML-Responses. Bun.sh ist hierbei bis zu dreimal schneller als Node.js.

Das Starten eines einfachen HTTP-Servers mittels Bun ist ebenfalls schnell erledigt. Mit dem Befehl **Bun.serve(port: 3000)** wird ein HTTP-Server gestartet, der auf Port 3000 lauscht. Im Vergleich zu einem Node.js-Server kann dieser Server viermal so viele Anfragen pro Sekunde bearbeiten.

Bun bietet auch weitere APIs, wie beispielsweise ein verbessertes 'sqlite3'-Modul und eine API für Passwörter. Das sind die nennenswertesten APIs, die Bun.sh zum jetzigen Zeitpunkt zu bieten hat.[58]

7.3.7.4 Client & Server

Bun wird in diesem Projekt auf beiden Seiten verwendet, alleinig zum starten von beiden Diensten, bis hin zum Package Management, TypeScript und JSX Support, ist Bun ein wichtiger Bestandteil der Entwicklung von dieser Webanwendung.

Bun wird auf der Serverseite verwendet, um den Server zu starten und zu verwalten. Der Server wird mit dem Befehl **bun run index.ts** gestartet. Bun stellt die Laufzeitumgebung bereit, in der ein Server mit Express.js und Socket.io betrieben wird. Der Server wird auch mit Bun gebündelt, um die Anwendung zu starten.

Auf der Clientseite wird Bun genauso verwendet, um die Anwendung zu starten, mittels dem Befehl **bun run start** gestartet. Die Anwendung wird mit Next.js und React betrieben, wobei Bun, sowie am Server, die Laufzeitumgebung bereitstellt. Das Bündeln der Applikationen wird auch von Bun übernommen.

7.3.8 Integration und Zusammenspiel dieser genutzten Technologien

In Abbildung 7.19 wird das Zusammenspiel der Technologien dargestellt. Die Webanwendung wird von einem Endgerät aufgerufen. Der Server, auf dem die Next.Js- und Express.js-Anwendung laufen, erhält die Anfrage und leitet sie direkt an den Next.Js-Client weiter. Dieser sammelt daraufhin die einzelnen React-Komponenten, die mit TailwindCSS gestyled wurden, zusammen. Anschließend holt der Next.js-Client die neuesten Daten vom Broker und sendet das gebündelte Paket als Antwort an das Endgerät.

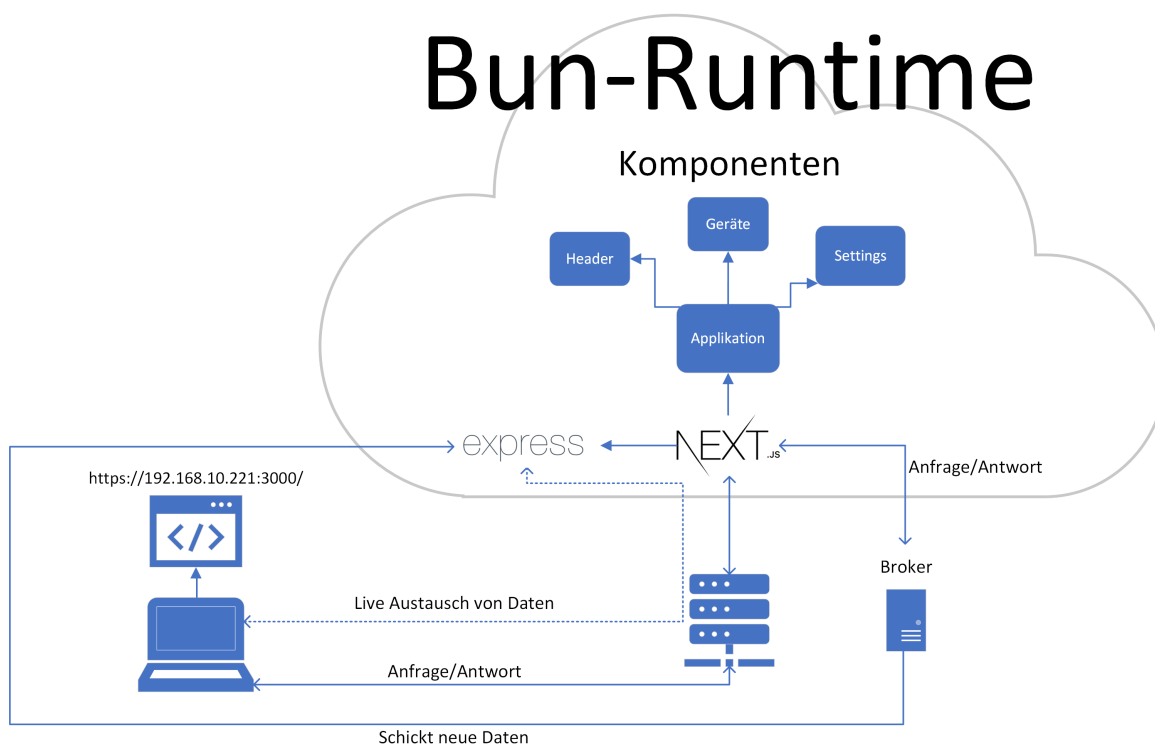


Abbildung 7.19: Zusammenspiel der Komponenten

Anschließend wird die Webanwendung auf dem Endgerät angezeigt und es wird eine Live-Verbindung zum Express.js-Server hergestellt. Dieser Server ist dafür zuständig, die Daten aktuell zu halten. Falls sich die Daten ändern, werden sie direkt vom Broker an den Express.js-Server gesendet und von dort aus an alle verbundenen Endgeräte weitergeleitet, um sicherzustellen, dass alles auf dem neuesten Stand ist.

7.4 Server & Client Realisierung und Herausforderungen

Die Webanwendung ist natürlich auch nicht ganz ohne Probleme entstanden. Es gab Herausforderungen zu meistern und Entscheidungen zu treffen. In diesem Kapitel wird die Realisierung des Servers und des Clients sowie die aufgetretenen Probleme und deren Lösungen behandelt.

7.4.1 Server

Der Server spielt bei der Webanwendung eine zentrale Rolle. Er ist für die konstante Kommunikation zwischen allen Clients und dem Broker verantwortlich und handelt als Vermittler zwischen ihnen. Der Server empfängt neue Werte vom Broker und leitet sie an alle aktiven Clients weiter.

7.4.1.1 Realisierung

Der Server wird ebenso wie der Client in JavaScript entwickelt, genauer gesagt in TypeScript (7.3.1) und mit Bun.sh (7.3.7) direkt ausgeführt, ohne dass es erst transpiliert wird. Der Server wird mit Express.js (7.3.2) realisiert und nutzt Socket.io (7.3.3) für die Echtzeitkommunikation. Über einfache POST-Requests am Server kann der Broker neue Daten übergeben, welche daraufhin an alle verbundenen Socket.io-Clients weitergeleitet werden.

Zu Beginn muss der Befehl **bun run index.ts** ausgeführt werden, da sich das Herz des Servers in der Datei **index.ts** befindet. Dadurch wird die Bun-Runtime gestartet, in der das Programm ausgeführt wird. Anschließend wird der Express.js-Server gestartet und lauscht auf Port 3050. Dieser Port ist festgelegt und kann nicht geändert werden, da der Broker und alle Clients auf diesen Port zugreifen. Nach Abschluss des Startvorgangs wird eine Nachricht in der Konsole ausgegeben, die besagt, dass der Server gestartet wurde und auf eingehende Verbindungen wartet.

7.4.1.2 Beispiel

Wenn ein Nutzer eine Verbindung zur Webanwendung herstellt, wird ein Client gestartet, der sich mit dem Server verbindet. Beim ersten Laden der Webapplikation werden keine weiteren Informationen zwischen dem Client und dem Server ausgetauscht, da der Client die Daten erstmalig selbst vom Broker holt. Erst wenn der Broker neue Daten hat, werden diese an den Server übergeben, welcher sie an alle verbundenen Clients weiterleitet. Ein Beispiel, es ändert sich die Temperatur im Badezimmer von 20°C auf 22°C. Sobald der Broker diese neue Information erhält, leitet er sie an den Server weiter, der alle verbundenen Clients verwaltet und den neuen Wert an alle weiterleitet. Das Objekt, das zu Beginn am Client die Temperatur von 20°C im Badezimmer anzeigte, wird nun auf 22°C aktualisiert. Dieser Vorgang wird bei jedem neuen Wert wiederholt und sorgt dafür, dass alle Clients immer auf dem neuesten Stand bleiben.

7.4.1.3 Herausforderungen und deren Lösungen

Zu Beginn dieser Arbeit war ein Server als Vermittler noch nicht geplant. Erst im Laufe der Entwicklung wurde klar, dass ein solcher Server notwendig ist. Die größte Herausforderung bestand darin, die Echtzeitkommunikation zwischen dem Server und den Clients zu realisieren. Da kein Teammitglied dieses Projekts zuvor Erfahrungen mit Socket.io oder anderen Echtzeitkommunikationstechnologien gesammelt hatte, war es notwendig, sich in diese Thematik einzuarbeiten.

Das Aufsetzen der anderen Teile des Servers, wie TypeScript oder Express.js, verlief aufgrund von Erfahrungen aus anderen Projekten reibungsloser. Es wurde viel Zeit damit verbracht, die Dokumentation von Socket.io zu lesen und zu verstehen. Anhand dieser Beispiele und Tests wurde die Funktionalität überprüft. Nach mehreren Tests und Anpassungen funktionierte die Echtzeitkommunikation zwischen Server und Client. Dadurch konnte der Server seine Aufgabe als Vermittler zwischen Broker und Client erfüllen. Eine größere Herausforderung bestand darin, die Kommunikation auf den Clients aufrechtzuerhalten. Mehr dazu wird im Kapitel 7.4.2 erläutert.

7.4.2 Client

Die meiste Arbeit im Themenbereich der *Webanwendungen* findet auf der Client-Seite statt. Der Client ist die Benutzeroberfläche, die der Nutzer sieht und mit der er interagiert. Er ist für die Darstellung der Daten und die Interaktion mit den Nutzern verantwortlich und bildet somit die Schnittstelle zwischen diesen und dem gesamten Smart Home System.

7.4.2.1 Realisierung

Der Client wird in TypeScript entwickelt und läuft ebenfalls in der Bun-Runtime wie der Server. Zur Darstellung der Benutzeroberfläche wird Next.js mit React genutzt. Next.js ist für das Routen und das Rendern von bestimmten Komponenten zuständig. React ist ein JavaScript-Framework, das für die Darstellung der Benutzeroberfläche verwendet wird und für die Aktualisierung der Komponenten zuständig ist. TailwindCSS übernimmt das Styling aller Objekte. Die Echtzeitkommunikation zwischen Server und Client wird mittels Socket.io realisiert.

Um den Client zu starten, muss der Befehl **bun run start** oder **bun run dev** eingegeben werden. Dadurch wird die Bun-Runtime gestartet und darin der Next.js Client ausgeführt. Der Client lauscht auf Port 3000 und ist darüber frei erreichbar. Nach Abschluss des Startvorgangs wird eine Nachricht in der Konsole ausgegeben, die besagt, dass der Client gestartet wurde und auf eingehende Verbindungen wartet. Wenn ein Nutzer die URL (Uniform Resource Locator) des Next.js-Clients aufruft, werden die vom Nutzer angefragten Routen gerendert und an ihn geschickt. Dadurch kann der Endnutzer die gewünschte Seite anzeigen und mit ihr interagieren. Die Daten werden direkt vom Broker abgerufen und während des Renderings der Webanwendung angezeigt. Gleichzeitig baut der Client eine Verbindung zum Server auf, um die neuesten Daten empfangen zu können.

7.4.2.2 Beispiele

Im Gegensatz zum Server, ist beim Client zu jedem Beispiel ein Bild beigelegt, um die Funktionsweise besser zu veranschaulichen.

Beispiel 1: Hauptseite des Clients

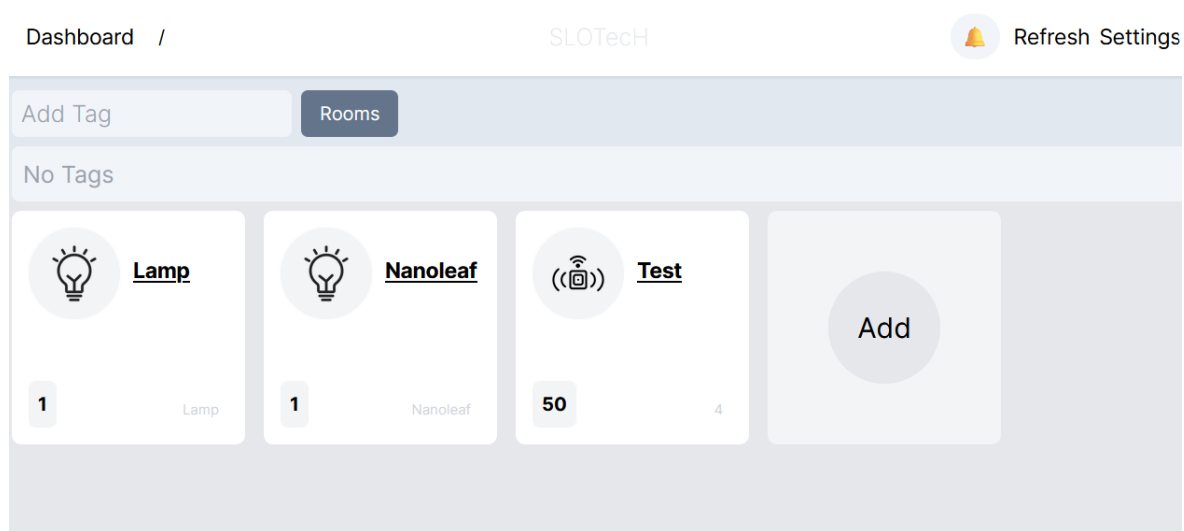


Abbildung 7.20: Beispiel der Hauptseite des Clients

In diesem Beispiel wird die Hauptseite des Clients aufgerufen. Der Nutzer hat die URL der Next.js Anwendung aufgerufen, welche aus einer IP-Adresse und einem Port besteht. In diesem Fall ist die IP-Adresse **192.168.10.221** und der Port **3000**. In Produktivumgebungen würde man hier eine Domain benutzen, aber für diese Beispiele ist eine IP-Adresse ausreichend.

```
▲ Next.js 14.0.1
- Local:      http://localhost:3000

✓ Ready in 1281ms
○ Compiling /page ...
✓ Compiled /page in 740ms (645 modules)
✓ Compiled in 140ms (301 modules)
✓ Compiled /icon.ico/route in 56ms (372 modules)
✓ Compiled /api/allComponents/route in 47ms (390 modules)
✓ Compiled /next/api/components/route in 32ms (392 modules)
```

Abbildung 7.21: Die Konsole des Next.js-Clients beim aufrufen der Hauptseite

Wie in der Abbildung 7.21 dargestellt, werden mehrere verschiedene Routen aufgerufen, um die Hauptseite zu rendern. Die Konsole gibt Auskunft darüber, welche Routen aufgerufen werden und wie lange das Rendern der Seite sowie jeder einzelnen Route gedauert hat. In diesem Fall dauert das Rendern der Hauptseite dreiviertel einer Sekunde, was für das erstmalige Kompilieren der Seite und Rendern der Komponenten eine relativ gute Zeit ist. Jedoch hängt die Leistung des Next.js-Clients auch von der Hardware des Geräts ab, auf dem er ausgeführt wird.

Die Endgeräte, die der Nutzer auf der Hauptseite sieht (Abbildung 7.20), wurden vom Broker abgerufen und an den Client weitergeleitet. Alle Geräte sind interaktiv und können gesteuert oder überwacht werden. Der Nutzer kann auf die einzelnen Geräte klicken und erhält weitere Informationen. Weitere Details dazu folgen in einem späteren Beispiel. Alle Daten auf der Hauptseite können in Echtzeit aktualisiert werden, da bereits eine permanente Verbindung zum Server besteht.

Beispiel 2: Interaktion mit einem Gerät

In diesem Beispiel wird die Interaktion mit einem Gerät beschrieben. Der Nutzer hat auf die Box des Geräts geklickt und wird auf die URL

192.168.10.221:3000/devices/Lamp weitergeleitet. Es handelt sich um ein Licht, das in der Küche installiert ist. Der Nutzer kann das Licht per Webanwendung ein- und ausschalten. Die Seite dieses Gerätes ist in Abbildung 7.22 dargestellt.

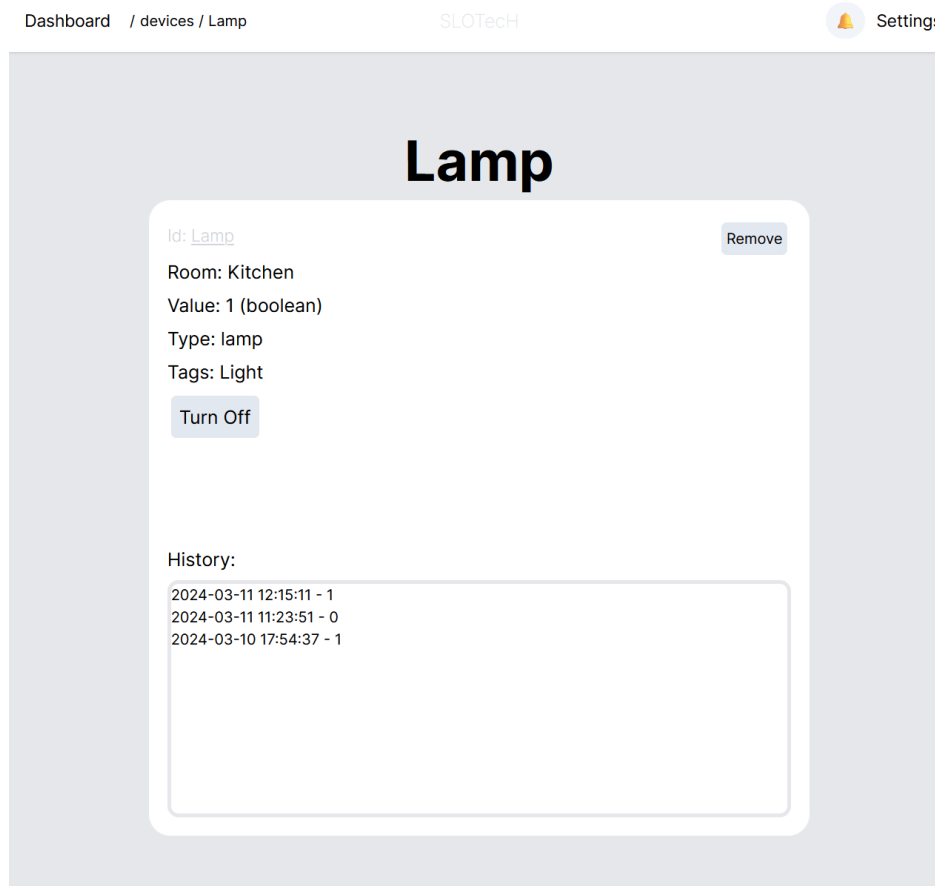


Abbildung 7.22: Beispiel der Interaktion mit einem Gerät

Es wurden neue Routen geladen, die jedoch nicht wesentlich anders aussehen als die Routen auf der Hauptseite. Daher gibt es hier keine zusätzliche Abbildung für die Konsole. Wie bereits erwähnt, kann das Licht einfach per Klick ein- und ausgeschaltet werden. Die gesamte Historie dieses Geräts wurde geladen und angezeigt. Neben der Historie gibt es auch eine Übersicht über den aktuellen Wert sowie den Raum, in dem sich das Gerät befindet. Alle Tags, der Gerätetyp und die ID sind ebenfalls auf dieser Seite zu finden. Schließlich gibt es auch einen Knopf, um das Gerät zu löschen.

Alle Daten auf dieser Seite werden natürlich auf in Echtzeit aktualisiert. Wenn das Licht beispielsweise von einem anderen Client ausgeschaltet wird, wird dies auch auf dieser Seite angezeigt. Der Nutzer kann das Licht auch ausschalten und es wird auf allen anderen Clients und auf dem Broker aktualisiert.

Wenn der Nutzer sich dazu entscheidet, dieses Gerät zu löschen, wird der Eintrag im Next.js-Client gelöscht. Anschließend kann das Gerät ohne weitere Probleme vom Strom getrennt werden.

Beispiel 3: Das Hinzufügen eines neuen Gerätes

In diesem Beispiel wird beschrieben, wie ein neues Gerät hinzugefügt wird. Der Nutzer klickt auf die Schaltfläche 'Add Device' auf der Hauptseite und wird dann auf die URL **192.168.10.221:3000/settings/new_device** weitergeleitet. Auf dieser Seite kann der Nutzer ein neues Gerät hinzufügen. Die Seite ist in Abbildung 7.23 dargestellt.

The screenshot shows a web application interface for adding a new temperature sensor. At the top, there is a breadcrumb trail 'Dashboard / settings / new_device', the 'SLOTech' logo, and a 'Settings' button with a bell icon. The main heading is 'Temperatur Sensor'. Below it, there are two input fields for sensor IDs, with '3' and '5' entered. Underneath these are two icons: a lightbulb and a sensor icon. Below the icons is a button labeled 'Add a new tag'. Then, there are two radio buttons for temperature units: 'Celsius' (selected) and 'Kelvin'. Below that is a large 'Add' button. At the bottom, there is a checked checkbox labeled 'Add Another Device'.

Abbildung 7.23: Beispiel des Hinzufügens eines neuen Gerätes

Wie in den anderen Beispielen werden zuerst alle Routen geladen, die für diese Seite notwendig sind. Nachdem die Seite vollständig geladen wurde, werden alle verfügbaren Geräte, die noch nicht hinzugefügt wurden, per ID angezeigt. In diesem Beispiel entscheidet sich der Nutzer für die ID '3'. Anschließend gibt er den Namen

des Geräts ein, 'Temperatur Sensor', wählt das Sensor-Icon aus und fügt die Tags 'Celsius' und 'Kelvin' hinzu. Um die Aufgabe abzuschließen, muss lediglich auf die Schaltfläche 'Add' geklickt werden. Anschließend wird das Gerät, als JSON-Objekt mit den angegebenen Werten auf dem Next.js-Client in eine Datei geschrieben und auf der Hauptseite angezeigt.

Dies sind einige Beispiele für das Verhalten der Webanwendung und die Interaktion des Benutzers damit. Es gibt viele weitere Funktionen und Features, die in diese Webanwendung integriert wurden. Diese Beispiele sollen nur einen kleinen Einblick in die Funktionsweise der Webanwendung geben.

7.4.2.3 Herausforderungen und deren Lösungen

Im Gegensatz zum Server gab es auf der Client-Seite einige Herausforderungen zu meistern. Um die vielen Features und Funktionen der Webanwendung zu realisieren, war es notwendig, sich in verschiedene Themen einzuarbeiten. Dazu gehören unter anderem die Echtzeitkommunikation, das Server-Side Rendering (SSR) sowie die Funktionsweise von Next.js, insbesondere das Routing und der 'App-Router' von Next.js. Auch die Verwendung von React-Komponenten und -Hooks war zu Beginn eine kleine Herausforderung.

Next.js und Server-Side Rendering (SSR)

Zu Beginn der Planung war es notwendig, sich für ein JavaScript-Framework zu entscheiden. Dabei fiel nach längerem Überlegen die Wahl auf Next.js. Da es von Anfang an klar war, dass React verwendet wird, und durch die enge Zusammenarbeit zwischen dem React Team und Next.js Team, war die Entscheidung, Next.js zu wählen ziemlich schnell getroffen. Durch das gute Auftreten von Next.js, mit allen seinen Features, war es erst recht überraschend, wie viel Zeit und Arbeit es in Anspruch genommen hat, mit dem ganzen Umzugehen und zum finalen Produkt zu kommen.

Nachdem das Grundgerüst von Next.js aufgesetzt wurde, begann die Erstellung der ersten React-Komponenten. Allerdings traten schnell Probleme auf: Die React-Hooks funktionierten nicht in diesen Komponenten. Es stellte sich heraus, dass Hooks in SSR-Komponenten nicht funktionieren, da diese vom Server zu 100% vorgerendert werden. Dennoch waren Hooks in diesen Komponenten notwendig. Zu Beginn wurde nach einer anderen Methode gesucht, um diese Hooks zu ersetzen, jedoch ohne Erfolg. Nach einiger Zeit und langen Recherchen stellte sich heraus, dass die einzige Lösung darin besteht, die Komponenten in Client-Komponenten umzuwandeln. In Next.js kann dies durch ein 'use client' am Anfang der Datei erreicht werden. Dadurch werden die Komponenten erst auf dem Endgerät des Nutzers gerendert und nicht auf dem Server.

Next.js und Routing (App-Router) Nicht nur die SSR-Komponenten waren ein Problem, das mit Next.js zu tun hatte. Eine viel größere Herausforderung war das Umgehen mit dem 'App-Router' von Next sowie das Routing im Allgemeinen und der Verbindungsaufbau mit dem Broker, um an neue Daten zu gelangen.

Alles begann mit dem App-Router, der zum Start der Entwicklung relativ neu war und über den außer in der Dokumentation von Next.js selbst nicht viel zu finden war. Trotzdem wurde sich dazu entschieden, ihn zu verwenden und damit auch die Server-Side-Komponenten. Grundsätzlich funktioniert dieser Mechanismus wie ein Samen, aus dem Wurzeln sprießen. Jede dieser Wurzeln bildet ein neues Verzeichnis. Durch dieses System wird man gezwungen, Dateien nach einem bestimmten Schema zu benennen und zu strukturieren. Leider war dieses Schema in diesem Projekt nicht sehr hilfreich und ist eher zu einem Hindernis geworden.

Eine größere Herausforderung war das Routing zum Broker, der mittels REST-API per ASP.Net realisiert wurde und anfangs nur auf demselben Gerät (Raspberry Pi) auf ASP.Net zugreifen konnte. Der Fehler musste umgangen werden, da die meisten Komponenten am Client gerendert werden und somit alle Aufrufe zum Broker nicht mehr lokal ablaufen, sondern übers Netzwerk, zwischen zwei Geräten.

Um das Problem zu lösen, wurden erneut viele Stunden in die Recherche investiert. Es stellte sich heraus, dass es mittels Next.Js Routing möglich ist, Anfragen auf einer bestimmten URL weiterzuleiten. Daher das die Next.Js Applikation auf dem selben Gerät wie das ASP.Net Backend läuft, konnte die URL des Brokers einfach auf **localhost** umgeleitet werden, mit dem richtigen Port. Dadurch kann der Next.Js Client auf dem Gerät des Nutzers auf das Backend zugreifen und die Daten abrufen.

Es ist wichtig zu erwähnen, dass das ASP.Net-Backend nun für alle Geräte im Netzwerk zugänglich ist und nicht mehr nur auf demselben Gerät läuft. Dies wurde durch die Verwendung von Nginx erreicht.

Next.Js und seine Probleme

Das Problem mit Next.js liegt nicht am Framework selbst, sondern an mangelnder Recherche im Vorhinein, was Next.js eigentlich ist und was es kann. Zu Beginn dieser Arbeit wurde Next.js unterschätzt und es war unklar, wie es funktioniert und wie es sich von anderen Frameworks unterscheidet. Die Webanwendung hatte mit Problemen zu kämpfen, angefangen beim App-Router bis hin zu den SSR-Komponenten. Die Fehler führten zu erheblichen Verzögerungen, die vermieden werden hätten können.

Es ist wichtig zu erwähnen, dass eine Person in diesem Team einen Bekannten hat, der bereits Erfahrungen mit Next.js gesammelt hat. Wodurch die Probleme schneller gelöst werden konnten, allerdings wäre es besser und schneller gewesen, wenn die Recherche im Vorfeld gründlicher durchgeführt worden wäre.

Echtzeitkommunikation (Socket.io)

Für die Realisierung der Echtzeitkommunikation zwischen Broker und Client wurde viel Zeit in Recherche und Tests investiert, um eine Lösung für dieses Projekt zu finden. Socket.io war immer wieder im Gespräch, wenn es um die Suche nach Echtzeitkommunikation ging. Bei den ersten Tests funktionierte jedoch nichts und niemand wusste, woran es lag. Nach einiger Zeit und weiteren Tests stellte sich heraus, dass das Problem beim Next.Js-Client lag, der die Verbindung nicht richtig an die Endgeräte weitergab. Das Problem wurde durch die Umwandlung von SSR-Komponenten in Client-Komponenten, wo es notwendig war, gelöst.

Allerdings traten danach andere Herausforderungen auf, da der Client sich nach jedem Neuladen der Seite neu verbinden musste und alte Socket.io-Clients sich nicht direkt entbunden haben. Dadurch kam es zu einer Vielzahl von Clients, die sich zufällig entbunden haben und den Server belastet haben. Später kam es dazu, dass der Client nach wiederholtem Laden der Seite keine Verbindung mehr zum Server aufbauen konnte. Um dieses Problem zu umgehen, wurden Sessions eingeführt, die mit einzigartigen IDs versehen wurden und an die Clients per Cookies gesendet wurden. Dadurch konnte die Next.js Applikation die Clients identifizieren und die Verbindung aufrechterhalten, ohne dass immer neue Clients erstellt wurden.

Bei jeder Komponente wird die Session einfach als Prop übergeben. Dadurch erfolgt die Anfrage nach der richtigen Session-ID nur einmalig und nicht bei jedem Laden der Komponenten. Nach ausführlichen Tests und Anpassungen funktionierte die Echtzeitkommunikation zwischen Server und Client. Der Server konnte dadurch seine Aufgabe als Vermittler zwischen Broker und Client erfüllen.

7.4.2.4 Fazit

Im Bereich der Webanwendung gab es viele Herausforderungen zu meistern. Einige dieser Herausforderungen waren auf unzureichende Recherche im Vorfeld zurückzuführen, während andere unerwartet auftraten und nur durch Tests und Anpassungen gelöst werden konnten. Trotz dieser Schwierigkeiten konnte die Webanwendung erfolgreich realisiert werden.

Es ist wichtig, aus diesen Erfahrungen zu lernen, dass es ratsam ist, im Voraus über die Technologien nachzudenken und sie vor der Integration in ein Projekt bereits durch Tests und Beispiele zu erproben.

Nichtsdestotrotz war es eine lehrreiche Erfahrung im Team zu arbeiten und sich mit den Technologien zur Webentwicklung auseinanderzusetzen. Das Problemlösen und die Zusammenarbeit im Team hat viel Spaß gemacht und war eine willkommene Abwechslung und Challenge zum normalen Unterricht.

Abbildungsverzeichnis

2.1	Statistik aller produktiven Stunden	36
6.1	Das Bild veranschaulicht unsere Definition von den verschiedenen Topics	60
6.2	Ein Beispiel zu Deadlocks	76
6.3	UML Diagramm zur Veranschaulichung der Datenbank	82
7.1	Hauptansicht der Webapplikation	99
7.2	BreadCrumb Navigation	101
7.3	BreadCrumb Navigation (Zoom)	101
7.4	Alle Technologien im Überblick	103
7.5	Express.js	107
7.6	Socket.io Server Beispiel	110
7.7	Socket.io Client Beispiel	111
7.8	Socket.io Server-Side Ausschnitt aus dem Projekt	112
7.9	Socket.io Client-Side Ausschnitt aus dem Projekt	112
7.10	Dateibaumstruktur von Next.js	115
7.11	Dateibaumstruktur von der API in Next.js	115
7.12	Beispiel aus unserem Projekt, von einer API-Route in Next.js	116

7.13 Beispiel einer React-Komponenten-Struktur	117
7.14 React JSX Beispiel	119
7.15 React JSX Beispiel mit einer externer Konstante	119
7.16 Ausschnitt aus diesem Projekt, eine React-Komponente	120
7.17 TailwindCSS Beispiel für eine Simple Komponente	123
7.18 TailwindCSS in Benutzung auf der Clientseite	125
7.19 Zusammenspiel der Komponenten	130
7.20 Beispiel der Hauptseite des Clients	134
7.21 Die Konsole des Next.js-Clients beim aufrufen der Hauptseite	134
7.22 Beispiel der Interaktion mit einem Gerät	136
7.23 Beispiel des Hinzufügens eines neuen Gerätes	137

Tabellenverzeichnis

1.1	Sprint Daten	20
1.2	Projektressourcen	26
1.3	Personelle Ressourcen geplant	27
1.4	Personelle Ressourcen tatsächlich	27

Literatur

- [1] Motion. *Understanding Project Milestones and Their Role in Success*. [Online; accessed 29-September-2023]. 2024. URL: [**https://www.usemotion.com/blog/project-milestones**](https://www.usemotion.com/blog/project-milestones).
- [2] Ward Cunningham. *Manifesto for Agile Software Development*. [Online; accessed 29-September-2023]. 2001. URL: [**https://agilemanifesto.org/**](https://agilemanifesto.org/).
- [3] Scrum.org. *What is Scrum?* [Online; accessed 29-September-2024]. 2024. URL: [**https://www.scrum.org/resources/what-scrum-module**](https://www.scrum.org/resources/what-scrum-module).
- [4] Opensource.com. *What is a Raspberry Pi?* [Online; accessed 12-Dezember-2023]. 2024. URL: [**https://opensource.com/resources/raspberry-pi**](https://opensource.com/resources/raspberry-pi).
- [5] Raspberry Pi. *Raspberry Pi Documentation - Raspberry Pi hardware*. [Online; accessed 12-Dezember-2023]. 2024. URL: [**https://www.raspberrypi.com/documentation/computers/raspberry-pi.html**](https://www.raspberrypi.com/documentation/computers/raspberry-pi.html).
- [6] Wikipedia contributors. *MQTT (Protokoll)*. [Online; accessed 15-March-2024]. 2024. URL: [**https://www.opc-router.de/was-ist-mqtt/**](https://www.opc-router.de/was-ist-mqtt/).
- [7] HiveMQ. *What is MQTT Quality of Service*. [Online; accessed 29-November-2023]. 2024. URL: [**https://www.hivemq.com/blog/mqtt-essentials-part-6-mqtt-quality-of-service-levels/**](https://www.hivemq.com/blog/mqtt-essentials-part-6-mqtt-quality-of-service-levels/).
- [8] aristocratos. *btop: A monitor of resources*. [Online; accessed 30-November-2023]. 2024. URL: [**https://github.com/aristocratos/btop**](https://github.com/aristocratos/btop).
- [9] Microsoft. *Download .NET 7.0 (Linux, macOS, and Windows)*. [Online; accessed 20-Oktober-2023]. 2024. URL: [**https://dotnet.microsoft.com/en-us/download/dotnet/7.0**](https://dotnet.microsoft.com/en-us/download/dotnet/7.0).

- [10] Neovim. *Home - Neovim*. [Online; accessed 30-November-2023]. 2024. URL: <https://neovim.io/>.
- [11] ranger. *ranger: A VIM-inspired filemanager for the console*. [Online; accessed 30-November-2023]. 2024. URL: <https://github.com/ranger/ranger>.
- [12] Apache Subversion. *Subversion Documentation*. [Online; accessed 10-Oktober-2023]. 2024. URL: <https://subversion.apache.org/docs/>.
- [13] NGINX. *Advanced Load Balancer, Web Server, & Reverse Proxy - NGINX*. [Online; accessed 02-Februar-2024]. 2024. URL: <https://www.nginx.com/>.
- [14] GNOME. *nmcli: NetworkManager Reference Manual*. [Online; accessed 16-Jänner-2024]. 2024. URL: <https://developer-old.gnome.org/NetworkManager/stable/nmcli.html>.
- [15] Liquid Web. *What is Systemctl? An In-Depth Overview*. [Online; accessed 16-Jänner-2024]. 2023. URL: <https://www.liquidweb.com/kb/what-is-systemctl-an-in-depth-overview/>.
- [16] Bitneo. *Visual Studio 2019*. [Online; accessed 15-March-2024]. 2024. URL: <https://www.biteno.com/was-ist-visual-studio/#:~:text=Visual%20Studio%20ist%20eine%20leistungsstarke,Webseiten%2C%20Webdienste%20und%20Apps%20entwickeln..>
- [17] Microsoft. *Microsoft.Data.Sqlite*. [Online; accessed 15-March-2024]. 2024. URL: <https://learn.microsoft.com/de-de/dotnet/standard/data/sqlite/extensions>.
- [18] GitHub. *MQTTnet*. [Online; accessed 15-March-2024]. 2024. URL: <https://github.com/dotnet/MQTTnet/wiki>.
- [19] Microsoft. *ASP.NET*. [Online; accessed 15-March-2024]. 2024. URL: <https://learn.microsoft.com/de-de/aspnet/overview>.
- [20] Ionos. *ASP.NET*. [Online; accessed 15-March-2024]. 2024. URL: <https://www.ionos.at/digitalguide/websites/web-entwicklung/was-ist-aspnet/>.

- [21] Intelligent modernisieren. *Szenen*. [Online; accessed 15-March-2024]. 2024. URL: <https://www.intelligent-modernisieren.de/faq/modernisierung/smart-home/was-sind-smart-home-szenen.html>.
- [22] Kinsta. *HTTP GET/POST erklärt*. [Online; accessed 15-March-2024]. 2024. URL: <https://kinsta.com/de/wissensdatenbank/was-ist-eine-http-anfrage/>.
- [23] Wikipedia. *Deadlock (Informatik)* — *Wikipedia, die freie Enzyklopädie*. [Online; Stand 15. März 2024]. 2023. URL: [https://de.wikipedia.org/w/index.php?title=Deadlock_\(Informatik\)&oldid=232971988%7D](https://de.wikipedia.org/w/index.php?title=Deadlock_(Informatik)&oldid=232971988%7D).
- [24] Crowdstrike. *CRUD - Create, Read, Update und Delete*. [Online; accessed 15-March-2024]. 2024. URL: <https://www.crowdstrike.com/cybersecurity-101/observability/crud/>.
- [25] IT-Visions. *ADO Mapper*. [Online; accessed 15-March-2024]. 2024. URL: https://www.it-visions.de/glossar/alle/6/Active_Data_Objects_NET_ADONET.aspx.
- [26] Microsoft. *TypeScript*. [Online; Stand 8. Februar 2024]. URL: <https://www.typescriptlang.org/docs/>.
- [27] Wikipedia. *TypeScript* — *Wikipedia*. [Online; Stand 8. Februar 2024]. 2024. URL: <https://de.wikipedia.org/w/index.php?title=TypeScript&oldid=241738750>.
- [28] *ts-node* — *GitHub*. [Online; Stand 8. Februar 2024]. URL: <https://github.com/TypeStrong/ts-node>.
- [29] Wikipedia contributors. *Express.js* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 10-February-2024]. 2024. URL: <https://en.wikipedia.org/w/index.php?title=Express.js&oldid=1194751246>.
- [30] Express.js contributors. *Middleware* — *Express.js*. [Online; accessed 10-February-2024]. 2024. URL: <https://expressjs.com/en/guide/using-middleware.html>.

- [31] Kinsta contributors. *What is Express.js? Everything You Should Know*. [Online; accessed 10-February-2024]. 2024. URL: [**https://kinsta.com/knowledgebase/what-is-express-js/**](https://kinsta.com/knowledgebase/what-is-express-js/).
- [32] Express.js contributors. *Routing — Express.js*. [Online; accessed 11-February-2024]. 2024. URL: [**https://expressjs.com/en/starter/basic-routing.html**](https://expressjs.com/en/starter/basic-routing.html).
- [33] Wikipedia contributors. *Npm — Wikipedia, The Free Encyclopedia*. [Online; accessed 10-February-2024]. 2024. URL: [**https://en.wikipedia.org/w/index.php?title=Npm&oldid=1201452139**](https://en.wikipedia.org/w/index.php?title=Npm&oldid=1201452139).
- [34] Npm contributors. *Express — npm*. [Online; accessed 10-February-2024]. 2024. URL: [**https://www.npmjs.com/package/express**](https://www.npmjs.com/package/express).
- [35] Bun contributors. *Build an HTTP server using Express and Bun*. [Online; accessed 10-February-2024]. 2024. URL: [**https://bun.sh/guides/ecosystem/express**](https://bun.sh/guides/ecosystem/express).
- [36] Express.js contributors. *Hello world example — Express.js*. [Online; accessed 10-February-2024]. 2024. URL: [**https://expressjs.com/en/starter/hello-world.html**](https://expressjs.com/en/starter/hello-world.html).
- [37] Socket.IO contributors. *Documentation V4 — Socket.IO*. [Online; accessed 11-February-2024]. 2024. URL: [**https://socket.io/docs/v4**](https://socket.io/docs/v4).
- [38] Wikipedia contributors. *Socket.IO — Wikipedia, The Free Encyclopedia*. [Online; accessed 11-February-2024]. 2023. URL: [**https://en.wikipedia.org/w/index.php?title=Socket.IO&oldid=1180985241**](https://en.wikipedia.org/w/index.php?title=Socket.IO&oldid=1180985241).
- [39] Socket.IO contributors. *Protocol V4 — Socket.IO*. [Online; accessed 11-February-2024]. 2024. URL: [**https://github.com/socketio/socket.io-protocol**](https://github.com/socketio/socket.io-protocol).
- [40] Socket.IO contributors. *Socket.IO — GitHub*. [Online; accessed 11-February-2024]. 2024. URL: [**https://github.com/socketio/socket.io**](https://github.com/socketio/socket.io).
- [41] Socket.IO contributors. *Server Initialization — Socket.IO*. [Online; accessed 11-February-2024]. 2024. URL: [**https://socket.io/docs/v4/server-initialization/**](https://socket.io/docs/v4/server-initialization/).

- [42] Socket.IO contributors. *Client Initialization* — *Socket.IO*. [Online; accessed 11-February-2024]. 2024. URL: <https://socket.io/docs/v4/client-initialization/>.
- [43] Socket.IO contributors. *Client Socket instance* — *Socket.IO*. [Online; accessed 11-February-2024]. 2024. URL: <https://socket.io/docs/v4/client-socket-instance/>.
- [44] Next.js. *Next.js Documentation*. [Online; accessed 12-February-2024]. 2024. URL: <https://nextjs.org/docs>.
- [45] Next.js. *Next.js*. [Online; accessed 12-February-2024]. 2024. URL: <https://nextjs.org/>.
- [46] Next.js. *Next.js GitHub Repository*. [Online; accessed 21-February-2024]. 2024. URL: <https://github.com/vercel/next.js>.
- [47] Vercel. *Vercel*. [Online; accessed 21-February-2024]. 2024. URL: <https://vercel.com>.
- [48] Juan Salas. *What is NextJS? (Uses, Features, and Examples in 2023)*. [Online; accessed 21-February-2024]. 2024. URL: <https://www.goalto.io/blog/nextjs-uses-features-examples>.
- [49] Wikipedia contributors. *React (software)* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 28-February-2024]. 2024. URL: [https://en.wikipedia.org/w/index.php?title=React_\(software\)&oldid=1210239939](https://en.wikipedia.org/w/index.php?title=React_(software)&oldid=1210239939).
- [50] Evgeniy Fetisov und Anastasiya Talochka. *Top 30 Companies Using React JS Development*. [Online; accessed 28-February-2024]. 2024. URL: <https://jaydevs.com/top-companies-using-react-js/>.
- [51] Wilde-IT. *React*. [Online; accessed 28-February-2024]. 2024. URL: <https://www.wilde-it.com/react/>.
- [52] Sascha Thattil. *Vorteile von React.JS für die Webentwicklung*. [Online; accessed 28-February-2024]. 2024. URL: <https://www.software-developer-india.com/de/vorteile-von-react-js-fuer-die-webentwicklung/>.
- [53] React. *JSX*. [Online; accessed 05-March-2024]. 2024. URL: <https://legacy.reactjs.org/docs/introducing-jsx.html>.

- [54] Adam Wathan u. a. *Tailwind CSS*. [Online; accessed 5-March-2024]. 2024. URL: **<https://tailwindcss.com/>**.
- [55] Wikipedia contributors. *Tailwind CSS — Wikipedia, The Free Encyclopedia*. [Online; accessed 5-March-2024]. 2024. URL: **https://en.wikipedia.org/w/index.php?title=Tailwind_CSS&oldid=1206918069**.
- [56] Soham De Roy. *What is Tailwind CSS? A Beginner's Guide*. [Online; accessed 5-March-2024]. 2022. URL: **<https://www.freecodecamp.org/news/what-is-tailwind-css-a-beginners-guide/>**.
- [57] Bun. *Bun Documentation*. [Online; accessed 10-March-2024]. 2024. URL: **<https://bun.sh/docs>**.
- [58] Sumner Jarred, Partovi Ashcon und McDonnell Colin. *Bun 1.0*. [Online; accessed 11-March-2024]. 2024. URL: **<https://bun.sh/blog/bun-v1.0>**.